

Grafentheorie: van eerste definities tot onderzoek

Een boek dat begint bij "wat is een graaf" en eindigt bij grafenminoren en spectrale

36 sections · approximately 50,068 words

Johan De Clercq

Generated from the markdown sources

Contents

Hoofdstuk 1 — Wat een graaf is	3
Hoofdstuk 2 — Representaties	8
Hoofdstuk 3 — Graad	13
Hoofdstuk 4 — Wandelingen, paden, samenhang	17
Hoofdstuk 5 — Isomorfie	21
Hoofdstuk 6 — Bomen	25
Hoofdstuk 7 — Opspannende bomen en de formule van Cayley	29
Hoofdstuk 8 — Doorlopen	33
Hoofdstuk 9 — Minimale opspannende bomen	37
Hoofdstuk 10 — Kortste paden	41
Hoofdstuk 11 — Afstand tussen alle paren	46
Hoofdstuk 12 — Samenhang en de stelling van Menger	51
Hoofdstuk 13 — Max-stroom min-snedes	55
Hoofdstuk 14 — Koppeling	59
Hoofdstuk 15 — Kleuring	64
Hoofdstuk 16 — Bipartiete grafen	68
Hoofdstuk 17 — Vlakheid	71
Hoofdstuk 18 — De vijf- en de vierkleurenstelling	76
Hoofdstuk 19 — Perfecte en koordale grafen	79
Hoofdstuk 20 — Hamiltoniciteit	83
Hoofdstuk 21 — Kliken, onafhankelijke verzamelingen, overdekkingen	87
Hoofdstuk 22 — NP-moeilijkheid	90
Hoofdstuk 23 — Leven met hardheid	94
Hoofdstuk 24 — De probabilistische methode	98
Hoofdstuk 25 — Toevalsgraf en	102
Hoofdstuk 26 — De reuzencomponent	106
Hoofdstuk 27 — Extremale grafentheorie	110
Hoofdstuk 28 — Ramsey-theorie	114
Hoofdstuk 29 — Spectrale grafentheorie	117
Hoofdstuk 30 — De Laplaciaan	121
Hoofdstuk 31 — Minoren en boombreedte	125
Hoofdstuk 32 — Expanders, en waar je hierna heen gaat	130
Bijlage A — Notatie	134
Bijlage B — Woordenlijst	137
Bijlage C — Verder lezen	140
Bijlage E — Oplossingen bij de oefeningen	143

Hoofdstuk 1 — Wat een graaf is

Een graaf is een verzameling dingen en een verzameling paren van die dingen. Dat is de hele definitie, en bijna alles wat moeilijk is in dit boek komt voort uit hoe weinig ze zegt.

De definitie

Een **graaf** G is een paar (V, E) waarbij V een eindige verzameling **knopen** is en E een verzameling ongeordende paren van verschillende knopen: de **kanten**.

Lees dat nog eens over, en let op wat het uitsluit. E is een *verzameling*, dus er bestaat niet zoiets als twee kanten tussen hetzelfde paar — je hebt de kant, of je hebt hem niet. De paren bestaan uit *verschillende* knopen, dus geen knoop verbindt zichzelf. De paren zijn *ongeordend*, dus een kant van u naar v is hetzelfde object als een kant van v naar u . En V is *eindig*, wat in dit boek overal geldt en wat echt onderzoek vaak laat vallen.

Een graaf die aan dit alles voldoet heet **enkelvoudig**, en tenzij een hoofdstuk anders zegt, is elke graaf hier enkelvoudig. De drie versoepelingen hebben elk een naam:

- herhaalde kanten toestaan geeft een **multigraaf**;
- een kant van een knoop naar zichzelf toestaan geeft een **lus**;
- de paren geordend maken geeft een **gerichte graaf**, of **digraaf**.

Hoofdstuk 10 en 13 hebben richtingen en gewichten nodig en voeren ze daar in. Alles ervoor redt het zonder.

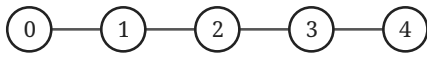
Twee getallen komen zo vaak voor dat ze één letter krijgen: $n = |V|$ en $m = |E|$. Zie je $O(n + m)$ in dit boek — en dat zul je voortdurend — dan is dat de grootte van de graaf, en van niets anders.

```
from graphs.core import Graph

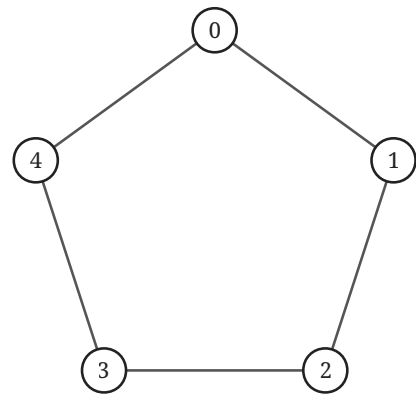
g = Graph(4, [(0, 1), (1, 2), (2, 3), (3, 0)])
print(g.n, g.m)                # 4 4
print(sorted(g.edges()))       # [(0, 1), (0, 3), (1, 2), (2, 3)]
print(g.neighbours(1))         # {0, 2}
```

De knopen heten hier 0, 1, 2, 3. Niets dwingt daartoe; knopen kunnen steden zijn, mensen, of webpagina's. Dit boek gebruikt overal gehele getallen omdat ze in arrays indexeren, en hoofdstuk 2 gaat erover waarom dat meer uitmaakt dan het klinkt.

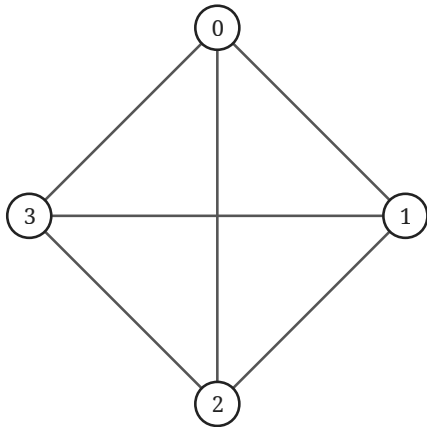
De tekening is de graaf niet



P5 - a path: 5 vertices, 4 edges



C5 - odd cycle, $\chi = 3$, not bipartite



K4 - complete, planar, $\chi = 4$

Drie van de vijf hieronder genoemde grafen, getekend. Elke prent is *een* keuze — de graaf levert geen posities, en hoofdstuk 17 gaat over wat die keuze wel en niet overleeft.

Teken de graaf hierboven en je krijgt een vierkant. Teken hem opnieuw met de knopen in een andere volgorde en je krijgt een vlinderdas met een kruising in het midden. Beide tekeningen zijn correct, want **een graaf heeft geen meetkunde**. Er zijn geen posities, geen lengtes, geen hoeken, en geen begrip van de ene kant die de andere kruist. Een tekening voegt dat allemaal toe, en niets ervan is gegeven.

Dit is de meest voorkomende beginnersfout, en ze overleeft ruimschoots het beginnersstadium: redeneren over een graaf met behulp van een eigenschap van de tekening die je ervan maakte. De prent heeft een links en een rechts. De graaf niet.

Het onderscheid is geen muggenzifterij, want twee gevallen doen er later toe en ze trekken in tegengestelde richting:

- **Vlakheid** (hoofdstuk 17) is de vraag of *een of andere* tekening kruisingen vermijdt. Dat is een eigenschap van de graaf, juist omdat het over alle tekeningen kwantificeert. De tekening die je toevallig maakte zegt niets.
- **Grafenopmaak**, het probleem om een tekening te maken die een mens kan lezen, is helemaal geen grafentheorie. Het is optimalisatie over een ruimte waarvan de graaf niets weet.

Houd het verschil scherp en hoofdstuk 17 is eenvoudig. Vervraag het en de stelling van Kuratowski lijkt over plaatjes te gaan.

Wat een graaf niet kan zeggen

De definitie is karig, en het is de moeite waard te benoemen wat ze weglaat, want naar een graaf grijpen wanneer je een van deze dingen nodig hebt, is hoe modellen misgaan.

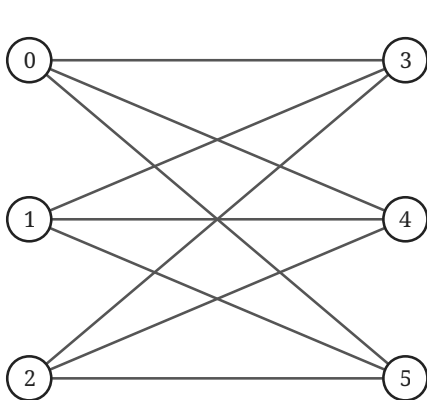
Relaties tussen drie of meer dingen tegelijk. Een kant verbindt precies twee knopen. Is je relatie werkelijk driedelig — drie auteurs op één artikel, drie reagentia in één reactie — dan gaat er informatie verloren als je haar als drie paarsgewijze kanten codeert, en je krijgt haar niet terug. De driehoek $\{a, b\}$, $\{b, c\}$, $\{a, c\}$ is niet te onderscheiden van drie afzonderlijke samenwerkingen die elkaar nooit ontmoeten. Wat je wilt is een **hypergraaf**, waarin een kant een willekeurige deelverzameling is.

Volgorde of veelvoud. Twee vluchten tussen dezelfde steden zijn één kant. Doet de tweede vlucht ertoe, dan heb je een multigraaf nodig, en de standaardresultaten veranderen: het handdruklemma van hoofdstuk 3 blijft overeind, de karakterisering van bomen in hoofdstuk 6 niet.

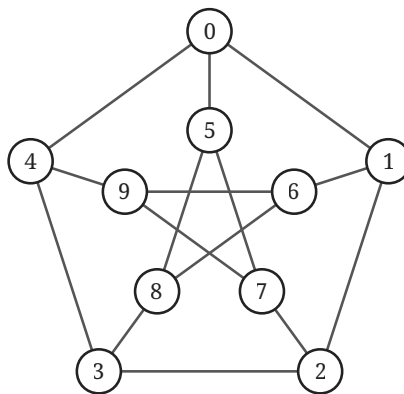
Iets over de knopen zelf. Een graaf weet niet dat knoop 3 een persoon is. Attributen leven buiten de structuur, in een woordenboek dat je ernaast draagt. Dat is een voordeel — elke stelling in dit boek geldt ongeacht wat de knopen betekenen — maar het betekent dat een graaf alleen zelden een volledig model van iets is.

Richting, tenzij je erom vraagt. "Alice volgt Bob" is niet symmetrisch, en dat modelleren met een ongerichte kant is een leugen die zichzelf niet aankondigt. Ruwweg de helft van de resultaten in dit boek heeft een gerichte tegenhanger; sommige zijn moeilijker (hoofdstuk 12), sommige eenvoudiger, en sommige onwaar.

Een eerste familie grafen



$K_{3,3}$ - bipartite, and not planar



The Petersen graph - the standard counterexample

$K_{3,3}$ getekend als twee kolommen, wat zijn bipartietheid meteen zichtbaar maakt; de Petersen-graaf in zijn gebruikelijke vijfhoek-met-pentagram-opmaak. Geen van beide tekeningen wordt door de graaf afgedwongen.

Vijf grafen komen zo vaak voor dat ze een naam krijgen, en je zou alle vijf voor je moeten kunnen zien.

Naam	Notatie	n	m
Volledige graaf	K_n	n	$n(n-1)/2$
Lege graaf	—	n	0
Pad	P_n	n	$n - 1$
Cykel	C_n ($n \geq 3$)	n	n
Volledig bipartiet	$K_{\{a,b\}}$	$a + b$	ab

```
from graphs.core import complete, cycle, complete_bipartite, petersen

print(complete(5).m)           # 10 = 5*4/2
print(cycle(7).m)              # 7
print(complete_bipartite(3, 3).m) # 9
print(petersen().degree_sequence()) # [3, 3, 3, 3, 3, 3, 3, 3, 3]
```

De laatste is de **Petersen-graaf**, en die verdient zijn plaats als het standaard tegenvoorbeeld van dit boek. Hij is 3-regulier, hij is niet vlak, hij is niet Hamiltoniaans, en zijn chromatisch getal is 3. Vrijwel elk plausibel klinkend vermoeden dat een lezer in de hoofdstukken 15 tot 20 verzint, sneuvelt op de Petersen-graaf, en dat is de efficiëntste reden om hem nu al uit het hoofd te leren.

Probeer het

Overtuig jezelf ervan dat de tekening geen informatie draagt, door dezelfde graaf op twee manieren te bouwen en te controleren dat ze gelijk zijn als *gelabelde* grafen:

```
python -c "
from graphs.core import Graph, cycle
square = Graph(4, [(0,1),(1,2),(2,3),(3,0)])
bowtie = Graph(4, [(3,0),(2,3),(0,1),(1,2)])
print('same graph:', square == bowtie)
print('is it C_4:', square == cycle(4))
"
```

```
same graph: True
is it C_4: True
```

De kantenlijsten zijn in verschillende volgorde geschreven en de ene werd omschreven als een vierkant en de andere als een vlinderdas. Het is hetzelfde object, en `Graph.__eq__` trekt zich niets aan van hoe je het tekende.

Probeer nu de moeilijkere versie, waarin de *labels* verschillen:

```
python -c "
from graphs.core import Graph, cycle
relabelled = Graph(4, [(0,2),(2,1),(1,3),(3,0)])
print('equal as labelled graphs:', relabelled == cycle(4))
"
```

equal as labelled graphs: False

Beide zijn viercyclen. Ze zijn niet gelijk, want gelijkheid van gelabelde grafen vraagt of *dezelfde paren* verbonden zijn, en dat zijn ze hier niet. De relatie die je eigenlijk wilde is isomorfie, en die is lastig genoeg om hoofdstuk 5 helemaal voor zich te krijgen.

Oefeningen

1. K_n heeft $n(n-1)/2$ kanten. Verifieer dit voor $n = 6$ door te tellen, en controleer het tegen `complete(6).m`.
2. De Petersen-graaf is 3-regulier op 10 knopen. Hoeveel kanten heeft hij, zonder ze een voor een te tellen?
3. Schrijf twee verschillende kantenlijsten die *dezelfde gelabelde graaf* opleveren, en één die een andere gelabelde graaf op dezelfde knopen geeft.
4. Drie auteurs schrijven samen één artikel. Leg uit wat er verloren gaat wanneer dit als drie paarsgewijze kanten wordt gemodelleerd, en noem de structuur die het niet verliest.

Oplossingen in Bijlage E.

Kernpunten

- Een graaf is een eindige knopenverzameling en een verzameling ongeordende paren. Enkelvoudig als standaard: geen lussen, geen herhaalde kanten, geen richtingen.
- n en m zijn de aantallen knopen en kanten, en worden vanaf hier zonder inleiding gebruikt.
- Een graaf heeft geen meetkunde. Elk argument dat afhangt van je tekening is geen argument over de graaf.
- Alleen paarsgewijs, ongeordend, zonder labels: heeft je model drieledige relaties, veelvoud van kanten, of richting nodig, zeg dat dan expliciet, want de standaarddefinitie laat het stilzwijgend vallen.
- Leer de Petersen-graaf vroeg kennen. Hij weerlegt meer gissingen dan welke andere graaf in dit boek ook.

Hoofdstuk 2 — Representaties

Een graaf is een abstract object. Een graaf in het geheugen is een concreet object, en het gat daartussen is waar elk algoritme in dit boek zijn looptijd vandaan haalt.

Er zijn drie standaardmanieren om een graaf op te slaan, en de schoolboekvergelijking ertussen is een tabel met asymptotiek. Dit hoofdstuk geeft je die tabel, meet haar vervolgens, en de meting spreekt de tabel op drie plaatsen tegen. Alle drie de tegenspraken zijn eerlijk — de asymptotiek klopt, en ze is óók niet wat er gebeurt bij de omvang waarop je werkelijk draait.

De drie representaties

Kantenlijst. Sla de paren op en verder niets. $O(m)$ ruimte, en elke vraag behalve "hoeveel kanten" vereist een scan. Het is de juiste keuze voor invoer, uitvoer, en niets anders. Het algoritme van Kruskal in hoofdstuk 9 is de enige plaats in dit boek waar het echt de natuurlijke structuur is, omdat dat algoritme als eerste daad de kanten toch al sorteert.

Adjacentiellijst. Voor elke knoop de verzameling van zijn burens. $O(n + m)$ ruimte. Eén knoops burens aflopen kost $O(\deg(v))$ — optimaal, want dat is de grootte van het antwoord. Vragen of een specifieke kant bestaat kost $O(\deg(v))$ in een gelinkte lijst, of $O(1)$ verwacht in een hashverzameling, en dat laatste gebruikt `graphs.core.Graph`.

Adjacenciematrix. Een $n \times n$ array van bits, symmetrisch voor een ongerichte graaf. $O(n^2)$ ruimte, ongeacht hoe weinig kanten er zijn. Kantvragen zijn één array-index. Eén knoops burens aflopen kost $O(n)$, want je moet elke plaats in de rij bekijken, ook de nullen.

```
from graphs.core import Graph
from graphs.matrix import MatrixGraph

g = Graph(4, [(0, 1), (1, 2), (2, 3)])
mg = MatrixGraph.of(g)
print(g.has_edge(0, 1), mg.has_edge(0, 1))      # True True
print(sorted(g.neighbours(1)), sorted(mg.neighbours(1))) # [0, 2] [0, 2]
```

`MatrixGraph` slaat elke rij op als één Python-geheel getal dat als bitmasker dienstdoet. Die keuze maakt de metingen hieronder interessant, en wordt aan het eind eerlijk besproken.

De tabel die iedereen opschrijft:

	kantenlijst	adjacentiellijst	adjacenciematrix
ruimte	$O(m)$	$O(n + m)$	$O(n^2)$
<code>has_edge(u, v)</code>	$O(m)$	$O(1)$ verwacht	$O(1)$
$N(v)$ aflopen	$O(m)$	$O(\deg v)$	$O(n)$
kant toevoegen	$O(1)$	$O(1)$	$O(1)$

Meet het nu.

Wat er werkelijk gebeurt

```
python scripts/bench_representations.py
```

`n = 600`, times in microseconds per operation, best of 3

density	m	edge query		scan neighbours		count triangles	
		list	matrix	list	matrix	list	matrix
0.01	1800	0.088	0.180	0.2	54.6	6	56
0.05	8958	0.089	0.189	0.5	57.1	55	65
0.20	35872	0.095	0.202	1.3	62.7	796	89
0.50	89755	0.104	0.202	2.7	69.3	6619	138

Absolute tijden horen bij deze machine en verschuiven enkele procenten tussen runs; de verhoudingen zijn stabiel. Drie dingen hier zijn niet wat de tabel voorspelt.

De adjacentielijst wint de kantvraag, bij elke dichtheid. De tabel zegt dat beide $O(1)$ zijn; de meting zegt dat de lijst ongeveer twee keer zo snel is. Een gehashte lidmaatschapstest is werkelijk één hash en één probe. De `rows[u] >> v & 1` van de matrix is geen enkele machine-instructie, want `rows[u]` is een Python-geheel getal van willekeurige precisie, en het naar rechts schuiven ervan met `v` raakt elk 30-bits cijfer onder `v`. De kantvraag van de matrix is $O(n)$ in vermomming.

Dat is een bewering over schaalgedrag, dus die vraagt om een schaalmeting in plaats van een bewering:

edge query as `n` grows, `p = 0.05` fixed:

n	bits/row	list	matrix	ratio
64	64	0.084	0.144	1.72x
256	256	0.093	0.163	1.76x
1024	1024	0.098	0.199	2.02x
4096	4096	0.106	0.267	2.52x
16384	16384	0.118	0.674	5.70x

De kosten van de lijst zijn vlak — 0,084 tot 0,118 microseconde over een 256-voudige toename van `n`. Die van de matrix groeien gestaag, en bij `n = 16384` is de structuur met "constante tijd" **ongeveer 6 keer trager dan die met "lineaire tijd"** bij precies de operatie die ze had moeten winnen.

Houd de dichtheid vast wanneer je dit probeert. De eerste versie van deze meting liet `p` per ongeluk met `n` meevariëren, waardoor de grote grafen bijna leeg waren; de rijen waren toen kleine gehele getallen, de schuifoperatie was goedkoop, en de tijden kwamen er vlak uit. Het effect verdween volledig en de verkeerde conclusie leek goed onderbouwd. Wat de rijen duur maakt is de positie van het *hoogste* gezette bit, niet hoeveel bits gezet zijn.

De matrix wint het tellen van driehoeken met een factor 48, en met meer naarmate de dichtheid stijgt. Dit is de operatie waarvoor een matrix bestaat. Driehoeken door een knoop tellen betekent buurverzamelingen snijden, en twee bitmaskers snijden is `&` — 64 knopen per machinewoord, zonder enige interpreteroverhead per element:

```
def common_neighbours(self, u: int, v: int) -> int:
    return (self.rows[u] & self.rows[v]).bit_count()
```

Bij $p = 0,5$ is dat 138 microseconden tegen 6619 voor de lijst. Het geheel getal van willekeurige precisie dat kantvragen traag maakte, is precies wat dit snel maakt: het werk gebeurt binnen één C-operatie in plaats van binnen een Python-lus. Hoofdstuk 21 gebruikt dit, en het is het verschil tussen een klikzoektocht die eindigt en een die dat niet doet.

De matrix gebruikt 22 keer minder geheugen dan de lijst, op een ijle graaf.

```
memory, n = 600, p = 0.05:
  adjacency list 1,344,576 bytes
  bitset matrix  61,704 bytes
```

$O(n^2)$ dat $O(n + m)$ met een factor twintig verslaat lijkt onmogelijk, en de asymptotiek is niet fout — ze is bij $n = 600$ alleen niet de baas. Een Python-set draagt ruwweg twee kilobyte overhead voordat hij iets bevat, en er zijn er 600. Een geheel getal van 600 bits is ongeveer 100 byte. Het omslagpunt waar $O(n^2)$ werkelijk verliest ligt ver rechts van de meeste grafen die je tegenkomt, en heb je ooit een adjacentielijst gekozen "om geheugen te besparen" op een graaf van een paar duizend knopen, dan is het de moeite waard te meten wat je bespaarde.

De tegenspraak juist lezen

Niets hiervan maakt de asymptotische tabel onjuist. Het maakt haar onvolledig op een specifieke manier, en de specifieke punten doen ertoe:

- De trage kantvraag van de matrix is een feit over **een rij coderen als één groot geheel getal**. Een echte bitset — numpy, C, of bitarray — indexeert direct het bevattende woord en is werkelijk $O(1)$. Neem je één implementatie mee uit dit hoofdstuk, dan deze: "bitset" en "Python-geheel-getal-als-bitset" hebben verschillende complexiteit, en maar één van beide komt overeen met het schoolboek.
- De geheugenkosten van de lijst zijn een feit over **de overhead van Python-verzamelingen**, niet over adjacentielijsten. In C zou een adjacentielijst bij $p = 0,05$ werkelijk kleiner zijn.
- De winst van de matrix bij driehoeken tellen is **geen** artefact. Het is het echte asymptotische voordeel — $n/64$ woorden in plaats van $\deg(v)$ geïnterpreteerde operaties — en het blijft in elke taal overeind.

De algemene les is de les die dit boek zal herhalen: een asymptotische grens vertelt je de vorm van een kromme, niet haar positie. Twee $O(1)$ -operaties kunnen zesvoudig verschillen, en welke wint kan omkeren naarmate n groeit.

Kiezen

Gebruik voor alles in deel I tot en met III de adjacentielijst. Doorlopen, kortste paden, stromen en koppelingen zijn allemaal $O(n + m)$ of erger in m , en ze lopen allemaal buurverzamelingen af, wat de beste operatie van de lijst is en de slechtste van de matrix met twee ordes van grootte.

Grijp naar een matrix wanneer je **verzamelingsoperaties op buurverzamelingen** doet — driehoeken tellen, klikzoektocht (hoofdstuk 21), gemeenschappelijke-buurgelijkenis — of wanneer de graaf werkelijk dicht is, dat wil zeggen m een constante fractie van n^2 is en niet slechts groot.

Grijp naar een kantenlijst wanneer de kanten het onderwerp zijn: ze sorteren (hoofdstuk 9), ze streamen, of ze naar schijf schrijven.

Probeer het

Kijk hoe de twee representaties het oneens zijn over wie sneller is, op dezelfde graaf, enkel afhankelijk van de gestelde vraag:

```
python -c "
import sys, time, random
sys.path.insert(0, '.')
from graphs.generate import random_graph
from graphs.matrix import MatrixGraph
g = random_graph(800, 0.3, random.Random(1))
mg = MatrixGraph.of(g)

t = time.perf_counter(); sum(1 for _ in g.neighbours(5)); scan_list =
time.perf_counter() - t
t = time.perf_counter(); sum(1 for _ in mg.neighbours(5)); scan_mat =
time.perf_counter() - t
print(f'scan one neighbourhood: list {scan_list*1e6:.0f}us, matrix
{scan_mat*1e6:.0f}us')

t = time.perf_counter(); mg.common_neighbours(5, 6); tri_mat = time.perf_counter()
- t
t = time.perf_counter(); len(g.neighbours(5) & g.neighbours(6)); tri_list =
time.perf_counter() - t
print(f'intersect two:          list {tri_list*1e6:.0f}us, matrix {tri_mat*1e6:.0f}
us')
"
```

De eerste regel bevoordeelt de lijst met ruime marge en de tweede de matrix. Geen van beide representaties is beter; ze beantwoorden verschillende vragen goed, en de enige manier om te weten welke je nodig hebt, is weten welke vraag je stelt.

Oefeningen

1. Hoeveel kanten zijn er gemiddeld bij $n = 1000$ en $p = 0,01$, en hoeveel plaatsen heeft de adjacenciematrix?
2. Welke representatie zou je kiezen om driehoeken door een gegeven knoop te tellen, en waarom?
3. `MatrixGraph.has_edge` ziet eruit als een bittest in constante tijd. Leg uit waarom dat in deze implementatie niet zo is.
4. In de code van dit boek gebruikte de bitsetmatrix bij $n = 600$ *minder* geheugen dan de adjacencielijst. Geef de reden, en zeg of dat in C ook zou gelden.

Oplossingen in Bijlage E.

Kernpunten

- Kantenlijst voor invoer en uitvoer, adjacencielijst voor doorlopen, adjacenciematrix voor verzamelingsoperaties op burens en voor dichte grafen.
- n en m begrenzen alles, maar constanten bepalen echte programma's. De $O(1)$ -kantvraag van de matrix mat ongeveer 6 keer trager dan die van de lijst bij $n = 16384$, want een Python-geheel getal is geen machinewoord.

- De matrix versloeg de lijst op geheugen met een factor 22 bij $n = 600$. $O(n^2)$ betekent niet "groot" bij de omvang die de meeste grafen werkelijk hebben.
- Buurverzamelingen snijden met bitmaskers is de enige echte, taalonafhankelijke asymptotische winst, en hoofdstuk 21 hangt ervan af.
- Spreekt een meting een grens tegen, houd dan de andere variabelen vast en meet opnieuw voordat je een van beide gelooft. De dichtheid met n laten meevariëren verborg het belangrijkste effect van dit hoofdstuk volledig.

Hoofdstuk 3 — Graad

De graad van een knoop is hoeveel kanten hem raken. Het is het eerste getal dat je aan een graaf kunt hangen, het goedkoopste om te berekenen, en er volgt verrassend veel uit alleen dat.

Het handdruklemma

Schrijf $\deg(v)$ voor het aantal burens van v , en $\Delta(G)$ en $\delta(G)$ voor de grootste en kleinste graad in G .

Stelling (handdruklemma). Voor elke graaf geldt $\sum_v \deg(v) = 2m$.

Bewijs. Tel de paren (v, e) waarbij v een knoop is, e een kant, en v een eindpunt van e . Tel je per knoop, dan komt elke v in $\deg(v)$ zulke paren voor, wat $\sum_v \deg(v)$ geeft. Tel je per kant, dan heeft elke kant precies twee eindpunten, wat $2m$ geeft. De twee tellingen gaan over dezelfde verzameling, dus ze zijn gelijk. ■

Dat is een **dubbeltelling**: één verzameling, twee manieren om haar te turven. Het is de meest herbruikbare bewijstechniek in dit boek, en ze keert terug in hoofdstuk 17 voor de formule van Euler en in hoofdstuk 27 voor de stelling van Mantel.

```
from graphs.core import petersen

g = petersen()
print(sum(g.degree(v) for v in g.vertices()), 2 * g.m)  # 30 30
```

De werkelijke inhoud van het lemma is een pariteitsuitspraak, en dit is de vorm die je daadwerkelijk zult gebruiken:

Gevolg. In elke graaf is het aantal knopen van oneven graad even.

Bewijs. Splits de som in het handdruklemma in termen van even en van oneven graad. De even termen sommeren tot een even getal, en het totaal $2m$ is even, dus de oneven termen moeten ook tot een even getal sommeren. Een som van oneven getallen is even precies wanneer er een even aantal van is. ■

Daarom kun je geen feest houden waarop elk van negen mensen precies drie anderen de hand schudt, en daarom moet een 3-reguliere graaf een even aantal knopen hebben. Beide uitspraken klinken alsof ze werk vergen; beide zijn één regel.

Twee knopen delen altijd een graad

Stelling. Elke graaf met $n \geq 2$ heeft twee knopen van dezelfde graad.

Bewijs. Graden liggen in $\{0, 1, \dots, n-1\}$, wat n waarden zijn voor n knopen — nog geen tegenspraak. Maar 0 en $n-1$ kunnen niet allebei voorkomen: een knoop van graad $n-1$ grenst aan alles, dus niets heeft graad 0 . Dus de graden liggen werkelijk in een verzameling van grootte $n-1$, en het duivenhokprincipe maakt het af. ■

De voorwaarde $n \geq 2$ doet echt werk, en het is precies het soort dat bij het overschrijven sneuvelt. Op één knoop is de bewering onwaar — er is maar één knoop, dus geen paar om te vergelijken. De verificatie legt de voorwaarde expliciet vast in plaats van op de lezer te vertrouwen:

```
@theorem("Any graph with  $n \geq 2$  has two vertices of equal degree", chapter=3)
def two_vertices_share_a_degree(g: Graph) -> bool | None:
    if g.n < 2:
        return None          # not a pass -- the hypothesis fails, so this graph
    says nothing
    degrees = [g.degree(v) for v in g.vertices()]
    return len(set(degrees)) < len(degrees)
```

Dat er `None` teruggegeven wordt in plaats van `True` doet ertoe. `True` zou betekenen "de stelling gold hier", en dat is een leugen over een graaf waarover de stelling niets zegt.

Graadrijen

De **graadrij** is de lijst van graden, gebruikelijk niet-stijgend geordend. Het is een invariant: knopen hernoemen kan haar niet veranderen. Ze is dus een snelle manier om te bewijzen dat twee grafen *niet* isomorf zijn — en, zoals hoofdstuk 5 laat zien, een ernstig onvolledige manier om te bewijzen dat ze het wel zijn.

De interessante vraag loopt de andere kant op. Gegeven een lijst getallen, bestaat er een graaf met die graden? Zo'n lijst heet **grafisch**.

Sommige falen om eenvoudige redenen. $[1, 1, 1]$ heeft een oneven som, dus het handdruklemma doodt haar. $[5, 1, 1, 1]$ heeft een knoop die vijf burens wil in een graaf met vier knopen. Maar $[3, 3, 3, 1]$ heeft een even som en geen te grote plaats, en is nog steeds niet grafisch — en inzien waarom vergt een argument in plaats van een waarneming.

Havel–Hakimi

Het algoritme is gulzig en het bewijs is het algoritme.

Stelling (Havel–Hakimi). Zij $d_1 \geq d_2 \geq \dots \geq d_n$ met $d_1 \geq 1$. De rij is grafisch dan en slechts dan als de rij die ontstaat door d_1 te verwijderen en één af te trekken van elk van de volgende d_1 plaatsen, grafisch is.

Bewijs. De ene richting is eenvoudig: gegeven een graaf voor de gereduceerde rij, voeg een nieuwe knoop toe verbonden met de d_1 knopen waarvan de graad verlaagd werd, en je hebt een graaf voor de oorspronkelijke rij.

De andere richting is de inhoud. Stel dat G de oorspronkelijke rij realiseert, met v de knoop van graad d_1 . Grenst v aan precies de d_1 knopen van eerstvolgend hoogste graad, verwijder hem dan en je bent klaar. Zo niet, dan zijn er knopen x en y met $\deg(x) \geq \deg(y)$, waarbij v aan y grenst maar niet aan x . Omdat $\deg(x) \geq \deg(y)$ en x meer met v 's niet-buren verbonden is dan y , moet er een knoop z zijn die aan x grenst maar niet aan y . Verwijder nu de kanten vy en xz , en voeg vx en yz toe. Elke graad blijft gelijk, en v grenst nu aan x in plaats van aan y . Herhaal; elke verwisseling verhoogt strikt het aantal knopen van hoge graad dat aan v grenst, dus het proces eindigt in een realisatie van de vereiste vorm. ■

Dat **twee-verwisselingsargument** — ruil een paar kanten voor een paar dat alle graden behoudt — is het onthouden waard. Het is dezelfde zet die de stelling over vermeerderende paden in hoofdstuk 14 bewijst en de voorwaarde van Ore in hoofdstuk 20.

Het algoritme is de stelling, herhaald toegepast:

```
from graphs.degree import is_graphical_havel_hakimi, realise

print(is_graphical_havel_hakimi([3, 3, 3, 1])) # False
print(is_graphical_havel_hakimi([3, 3, 2, 2, 1, 1])) # True
print(sorted(realise([3, 3, 2, 2, 1, 1]).edges()))
# [(0, 1), (0, 4), (0, 5), (1, 2), (1, 3), (2, 3)]
```

Omdat het bewijs constructief is, geeft `realise` je een werkelijke graaf en niet een ja.

Erdős–Gallai

Er bestaat ook een niet-constructieve karakterisering, en die gebruik je wanneer je een criterium wilt in plaats van een graaf.

Stelling (Erdős–Gallai). Een niet-stijgende rij niet-negatieve gehele getallen met even som is grafisch dan en slechts dan als voor elke k van 1 tot n geldt:

$$\sum_{i \leq k} d_i \leq k(k-1) + \sum_{i > k} \min(d_i, k).$$

De ongelijkheid zegt iets dat je kunt lezen: de k knopen van hoogste graad moeten hun graad kwijt kunnen, ofwel onderling — hoogstens $k(k-1)$ kanteindpunten — ofwel door naar de rest te reiken, waar elke overblijvende knoop i er hoogstens $\min(d_i, k)$ kan opnemen. Kunnen ze dat niet, dan is de rij onmogelijk.

Twee stellingen, twee algoritmen, en geen reden om aan te nemen dat ze het eens zijn. Daarom controleert de verificatie beide tegen uitputtend zoeken en niet tegen elkaar:

```
held      ch 3 Havel-Hakimi agrees with brute-force realisability (52 graphs)
held      ch 3 Erdos-Gallai agrees with Havel-Hakimi (52 graphs)
held      ch 3 Havel-Hakimi's construction really has the requested degrees (52
graphs)
```

De eerste regel is degene die telt. Havel–Hakimi tegen Erdős–Gallai controleren zou alleen vaststellen dat twee implementaties van mij het eens zijn, wat ze kunnen zijn terwijl ze allebei fout zijn. Controleren tegen "probeer elke graaf op n knopen en kijk" kan zo niet falen.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.degree import is_graphical_havel_hakimi, is_graphical_erdos_gallai,
is_graphical_bruteforce
for s in ([3,3,3,3], [3,3,3,1], [1,1,1], [5,1,1,1,1,1], [2,2,2]):
    hh = is_graphical_havel_hakimi(list(s))
    eg = is_graphical_erdos_gallai(list(s))
    bf = is_graphical_bruteforce(list(s))
    print(f'{str(s):<18} Havel-Hakimi={hh!s:<6} Erdos-Gallai={eg!s:<6}
exhaustive={bf}')
"
```

[3, 3, 3, 3]	Havel-Hakimi=True	Erdos-Gallai=True	exhaustive=True
[3, 3, 3, 1]	Havel-Hakimi=False	Erdos-Gallai=False	exhaustive=False
[1, 1, 1]	Havel-Hakimi=False	Erdos-Gallai=False	exhaustive=False
[5, 1, 1, 1, 1, 1]	Havel-Hakimi=True	Erdos-Gallai=True	exhaustive=True
[2, 2, 2]	Havel-Hakimi=True	Erdos-Gallai=True	exhaustive=True

Let op de vierde: [5, 1, 1, 1, 1, 1] is grafisch op **zes** knopen — het is de ster $K_{1,5}$ — ook al zou het op vijf onmogelijk zijn. Graadrijen dragen n met zich mee, en dat vergeten is de meest voorkomende manier om deze paragraaf verkeerd te lezen.

Oefeningen

1. Kunnen negen mensen elk precies drie anderen de hand schudden? Verantwoord je antwoord met het handdruklemma.
2. Is [4, 3, 2, 1, 0] grafisch? Voer Havel–Hakimi met de hand uit.
3. Wat is de som van de graden van de Petersen-graaf, en wat zegt dat over zijn aantal kanten?
4. De stelling "twee knopen delen een graad" vereist $n \geq 2$. Geef de graaf op één knoop en leg precies uit welke stap van het bewijs faalt.

Oplossingen in Bijlage E.

Kernpunten

- Het handdruklemma is een dubbeltelling, en dubbeltelling is de werkpaardtechniek van dit boek.
- Het aantal knopen van oneven graad is altijd even. De meeste pariteitsargumenten over grafen herleiden hiertoe.
- De graadrij is een invariant, dus ze kan isomorfie weerleggen maar nooit bewijzen.
- Havel–Hakimi beslist realiseerbaarheid door constructie; Erdős–Gallai beslist haar door ongelijkheid. Beide worden hier bewezen, en beide worden tegen uitputtend zoeken gecontroleerd en niet tegen elkaar.
- Heeft een stelling een voorwaarde als $n \geq 2$, codeer die dan als "dit geval zegt niets" en niet als "dit geval slaagde".

Hoofdstuk 4 — Wandelingen, paden, samenhang

Dit hoofdstuk is woordenschat, en het loont de moeite er precies in te zijn, want drie woorden die beginners door elkaar gebruiken zijn niet onderling verwisselbaar, en de helft van de verwarring in latere bewijzen komt voort uit het vervagen ervan.

Drie woorden

Een **wandeling** is een rij knopen $v_0, v_1, \dots, v_k$ waarbij opeenvolgende knopen aangrenzend zijn. Herhalingen zijn toegestaan, zowel van knopen als van kanten. De **lengte** is k , het aantal doorlopen kanten — niet het aantal knopen, dat $k + 1$ is.

Een **pad** is een wandeling zonder herhaalde knoop.

Een **cykel** is een wandeling van lengte minstens 3 die begint en eindigt in dezelfde knoop, zonder andere herhalingen.

Het verband tussen de eerste twee is het lemma waarop al het overige rust:

Lemma. Bestaat er een wandeling van u naar v , dan bestaat er een pad van u naar v .

Bewijs. Neem een wandeling van u naar v van minimale lengte. Zou ze een knoop w herhalen, dan kon het stuk tussen de twee bezoeken aan w geschrapt worden, wat een kortere wandeling van u naar v oplevert — in tegenspraak met de minimaliteit. Dus ze herhaalt geen knoop, en is een pad. ■

Daarom mag je altijd "pad" aannemen wanneer een argument je een "wandeling" aanreikt, en vanaf hoofdstuk 9 gebeurt dat zonder commentaar. Het is ook de reden dat de twee begrippen werkelijk verschillen: wandelingen zijn eenvoudig te tellen en paden niet, wat het onderwerp is van de volgende paragraaf.

Wandelingen tellen

Wandelingen hebben een nette algebraïsche beschrijving die paden opvallend ontberen.

Stelling. Zij A de adjacentiematrix van G . De (u, v) -plaats van A^k is het aantal wandelingen van lengte precies k van u naar v .

Bewijs. Inductie naar k . Voor $k = 1$ is de bewering de definitie van A . Stel dat ze geldt voor k . Een wandeling van lengte $k+1$ van u naar v is een wandeling van lengte k van u naar een t , gevolgd door een kant tv . Sommeren over t geeft $(A^{k+1})_{uv} = \sum_t (A^k)_{ut} \cdot A_{tv}$, en dat is precies de definitie van matrixvermenigvuldiging. ■

```
from graphs.core import cycle, complete
from graphs.algorithms import walk_counts

for row in walk_counts(cycle(4), 2):
    print(row)
```

```
[2, 0, 2, 0]
[0, 2, 0, 2]
[2, 0, 2, 0]
[0, 2, 0, 2]
```

Lees het. Vanuit knoop 0 zijn er twee wandelingen van lengte 2 terug naar knoop 0 (naar een van beide burens en terug), twee naar knoop 2 (langs beide kanten rond), en geen naar de knopen 1 en 3 — want C_4 is bipartiet, en een wandeling van even lengte kan niet tussen de twee zijden oversteken. De nullen in die matrix zijn de stelling van hoofdstuk 16, drie hoofdstukken te vroeg zichtbaar.

Probeer nu dezelfde stelling voor paden op te schrijven. Dat lukt niet: er is geen matrixoperatie die paden telt, en ze tellen is $\#P$ -volledig. De kloof tussen "wandelingen zijn lineaire algebra" en "paden zijn onhandelbaar" is een van de scherpste scheidslijnen in het vak, en hoofdstuk 29 is wat er gebeurt als je de kant van de lineaire algebra serieus neemt.

Samenhang

u en v zijn **verbonden** wanneer een pad ze samenvoegt. Die relatie is reflexief (de wandeling van lengte nul), symmetrisch (draai het pad om) en transitief (plak twee wandelingen aan elkaar en pas dan het bovenstaande lemma toe om een pad te krijgen) — ze is dus een equivalentierelatie, en haar klassen zijn de **samenhangscomponenten**.

Dat "dus" is de hele reden dat het lemma bewezen moest worden. Twee paden aan elkaar plakken geeft geen pad; het geeft een wandeling. Zonder het wandeling-naar-padlemma faalt transitiviteit en zijn componenten niet welgedefinieerd.

```
from graphs.core import Graph
from graphs.algorithms import components, is_connected

g = Graph(7, [(0, 1), (1, 2), (2, 0), (3, 4), (5, 6)])
print(components(g))      # [{0, 1, 2}, {3, 4}, {5, 6}]
print(is_connected(g))    # False
```

Een graaf is **samenhangend** wanneer hij precies één component heeft. Twee conventies moeten uitgesproken worden in plaats van aangenomen, want teksten verschillen erin en bewijzen hangen er stilzwijgend van af:

- De graaf op één knoop is samenhangend. Daarover bestaat geen discussie.
- De lege graaf — helemaal geen knopen — geldt in dit boek als samenhangend. Hij heeft nul componenten, niet één, dus dit is een conventie en geen gevolg. Ze is gekozen omdat ze "elke graaf is de disjuncte vereniging van zijn componenten" waar maakt zonder uitzondering, en omdat de kantentelling van hoofdstuk 6 dan uitkomt. Sommige teksten noemen de lege graaf niet samenhangend; lees je er zo een, ga dan na welke stellingen er een voorwaarde bij krijgen.

Het kantenbudget

Stelling. Een samenhangende graaf op $n \geq 1$ knopen heeft $m \geq n - 1$.

Bewijs. Inductie naar n . Voor $n = 1$ volstaat $m \geq 0$. Voor de stap: neem een samenhangende G op n knopen. Heeft elke knoop graad minstens 2, dan is $2m = \sum \deg(v) \geq 2n$, dus $m \geq n > n - 1$ en we zijn klaar. Anders heeft een knoop v graad hoogstens 1; omdat G samenhangend is en $n \geq 2$, is zijn graad precies 1. Verwijder v en zijn kant. Het resultaat is samenhangend op $n - 1$ knopen, heeft dus per inductie minstens $n - 2$ kanten, en G heeft er minstens $n - 1$. ■

Het complementaire feit — een graaf met $m \geq n$ bevat een cykel — is dat van hoofdstuk 6, en samen leggen ze bomen precies vast.

Een tweede klein resultaat dat voortdurend gebruikt wordt:

Lemma. Eén kant verwijderen verhoogt het aantal componenten met hoogstens één.

Bewijs. uv verwijderen kan alleen knopen scheiden waarvan elk pad uv gebruikte. Twee knopen die in $G - uv$ nog door een pad verbonden zijn, blijven in dezelfde component. Elke knoop bereikt in $G - uv$ nog u of v , want een pad ernaartoe in G dat uv gebruikte kan afgekapt worden bij de eerste van u , v die het tegenkomt. De component die uv bevat valt dus in hoogstens twee stukken uiteen, en geen andere component verandert. ■

Een kant waarvan verwijdering het aantal *wel* verhoogt heet een **brug**, en hoofdstuk 12 karakteriseert ze: een kant is een brug precies wanneer ze op geen enkele cykel ligt.

Afstand

Voor u, v in dezelfde component is $d(u, v)$ de lengte van een kortste pad ertussen. Het is een metriek: niet-negatief, nul alleen op de diagonaal, symmetrisch, en voldoende aan de driehoeksongelijkheid $d(u, w) \leq d(u, v) + d(v, w)$ — plak aan elkaar en kort in.

```
from graphs.core import path
from graphs.algorithms import distances

print(distances(path(5), 0))  # {0: 0, 1: 1, 2: 2, 3: 3, 4: 4}
```

Breedte-eerst zoeken berekent ze alle in $O(n + m)$, en hoofdstuk 8 legt uit waarom dat klopt. De **diameter** is de grootste eindige afstand; de **excentriciteit** van v is de grootste afstand vanaf v ; de **straal** is de kleinste excentriciteit. Hoofdstuk 11 berekent ze allemaal, en hoofdstuk 30 verbindt de diameter met het Laplace-spectrum, wat een veel minder voor de hand liggend verband is dan het klinkt.

Probeer het

Bevestig de matrixmachtstelling tegen een directe telling, op een graaf die klein genoeg is om met de hand na te gaan:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete
from graphs.algorithms import walk_counts
w = walk_counts(complete(4), 3)
print('closed walks of length 3 at each vertex:', [w[i][i] for i in range(4)])
"
```

```
closed walks of length 3 at each vertex: [6, 6, 6, 6]
```

Zes klopt, en je ziet waarom: een gesloten wandeling van lengte 3 vanuit v in K_4 moet twee verschillende andere knopen bezoeken, en er zijn $3 \times 2 = 6$ geordende manieren om die te kiezen. Merk op dat dit elke driehoek **twee keer** telt — één keer per richting — en dat is waarom de standaardformule voor het aantal driehoeken $\text{trace}(A^3)/6$ is: drie startpunten, twee richtingen.

Oefeningen

1. Geef in C_5 een wandeling die geen pad is, en een pad dat geen cykel is.
2. Waaraan is $(A^2)_{\{vv\}}$ gelijk, en waarom?
3. Wat is het minimale aantal kanten in een samenhangende graaf op n knopen, en welke grafen bereiken het?
4. Twee paden aan elkaar plakken hoeft geen pad te geven. Leg uit waarom samenhang toch een equivalentierelatie is.

Oplossingen in Bijlage E.

Kernpunten

- Wandeling, pad en cykel zijn drie verschillende dingen. Een wandeling herhaalt; een pad niet; een cykel sluit.
- Elke wandeling bevat een pad met dezelfde eindpunten. Dat maakt samenhang tot een equivalentierelatie, en het wordt na dit hoofdstuk overal stilzwijgend gebruikt.
- $(A^k)_{\{uv\}}$ telt wandelingen van lengte k . Voor paden bestaat niets vergelijkbaars, en dat gat is geen gat in onze kennis — paden tellen is $\#P$ -volledig.
- Samenhangende grafen hebben $m \geq n - 1$; een kant verwijderen splitst hoogstens één component in tweeën.
- Spreek je conventie voor de lege graaf één keer uit en houd je eraan. Dit boek noemt hem samenhangend.

Hoofdstuk 5 — Isomorfie

Hoofdstuk 1 liet twee viercykels zien die niet gelijk waren. Het was overduidelijk dezelfde graaf. Die twee zinnen met elkaar verzoenen is waar dit hoofdstuk voor dient, en de verzoening blijkt een van de vreemdste problemen in de informatica te zijn.

De definitie

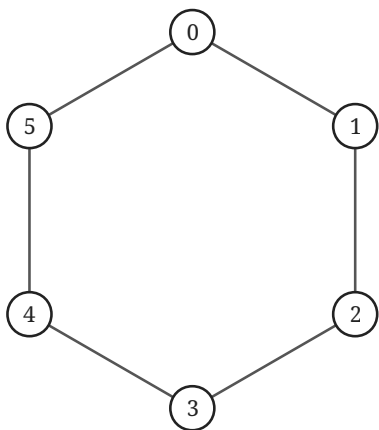
Een **isomorfie** van G naar H is een bijectie $\varphi : V(G) \rightarrow V(H)$ zodat $uv \in E(G)$ dan en slechts dan als $\varphi(u)\varphi(v) \in E(H)$. Bestaat er een, dan schrijven we $G \cong H$.

De definitie is onopvallend. Wat opvallend is, is dat niemand weet hoe moeilijk het is om haar te beslissen.

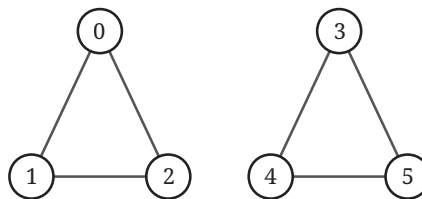
Bijna elk natuurlijk probleem in dit boek zit ofwel in P — kortste paden, koppeling, vlakheid — of is NP -volledig — kleuring, Hamiltoniciteit, kliek. Grafenisomorfie zit in NP , is niet bekend in P , en wordt *ook niet vermoed* NP -volledig te zijn, want als het dat was, zou de polynomiale hiërarchie instorten. Het hoort bij een kleine klasse problemen waarvan vermoed wordt dat ze werkelijk tussenin liggen. Babai's algoritme uit 2016 loopt in quasipolynomiale tijd, $\exp((\log n)^{O(1)})$, wat sneller is dan elke exponentiële en trager dan elke polynomiale functie. Dat is de huidige stand, en het is een vreemde plaats voor een probleem dat zo eenvoudig te formuleren is.

Invarianten, en wat ze niet kunnen

Een **invariant** is iets dat door isomorfie behouden blijft: n , m , de graadrij, het aantal driehoeken, de multiverzameling van componentgroottes. Elke invariant geeft een **eenzijdige test**. Verschillen twee grafen in enige invariant, dan zijn ze zeker niet isomorf. Komen ze overeen in alle invarianten die je controleerde, dan heb je niets geleerd.



C_6 - even cycle, bipartite



Two triangles - same degrees as C_6 , not isomorphic

De twee grafen waar dit hoofdstuk om draait: beide 2-regulier op zes knopen, met identieke graadrijen, en niet isomorf. De ene is samenhangend en de andere niet.

Die asymmetrie is het hele praktische verhaal, en het loont haar als een bewering te formuleren die de verificatie in beide richtingen kan controleren:

held	ch 5	Isomorphic graphs have equal degree sequences (52 graphs)
refuted	ch 5	Equal degree sequences imply isomorphic (1 graphs)

De tweede regel is een *succes*. Ze staat geregistreerd als een stelling waarvan verwacht wordt dat ze faalt, en de verificatie kleurt rood zodra er geen tegenvoorbeeld meer gevonden wordt. Het tegenvoorbeeld is het kleinste dat er is:

```
from graphs.iso import cospectral_mates, canonical

c6, two_triangles = cospectral_mates()
print(c6.degree_sequence())           # [2, 2, 2, 2, 2, 2]
print(two_triangles.degree_sequence()) # [2, 2, 2, 2, 2, 2]
print(canonical(c6) == canonical(two_triangles)) # False
```

De zeszykel en twee disjuncte driehoeken zijn beide 2-regulier op zes knopen. Identieke graadrijen, dezelfde n , dezelfde m , en niet isomorf — de ene is samenhangend en de andere niet.

Kleurverfijning, en haar beruchte blinde vlek

De standaard snelle heuristiek is **kleurverfijning**, ook eendimensionale Weisfeiler–Leman genoemd. Geef elke knoop dezelfde kleur. Herkleur elke knoop herhaaldelijk met het paar (zijn huidige kleur, de gesorteerde multiverzameling kleuren van zijn burens). Stop wanneer de partitie niet meer verandert.

```
def wl_colours(g, rounds=None):
    colour = [0] * g.n
    for _ in range(rounds if rounds is not None else g.n):
        signature = [
            (colour[v], tuple(sorted(colour[w] for w in g.neighbours(v))))
            for v in g.vertices()
        ]
        relabel = {s: i for i, s in enumerate(sorted(set(signature)))}
        new = [relabel[s] for s in signature]
        if new == colour:
            break
        colour = new
    return colour
```

Het loopt in bijna lineaire tijd en het scheidt vrijwel elk paar grafen dat je tegenkomt. Het is ook **correct maar niet volledig**: isomorfe grafen krijgen altijd dezelfde kleurmultipliciteit, dus een verschil bewijst niet-isomorfie, maar overeenstemming bewijst niets.

De blinde vlek is precies het paar hierboven:

```
from graphs.iso import wl_signature, wl_distinguishes

print(wl_signature(c6))           # ((0, 6),)
print(wl_signature(two_triangles)) # ((0, 6),)
print(wl_distinguishes(c6, two_triangles)) # False
```

Beide signaturen zeggen "zes knopen, allemaal kleur 0". Verfijning kan niet beginnen, want in een reguliere graaf ziet elke knoop er identiek uit aan elke andere: dezelfde kleur, dezelfde multiverzameling buurkleuren, voor altijd. De partitie is stabiel bij ronde nul.

Dit is geen tekortkoming van de implementatie. **Eendimensionale Weisfeiler–Leman kan geen twee reguliere grafen van dezelfde grootte en graad onderscheiden**, en daar zijn er heel veel van. De oplossing is k -dimensionale WL, die k -tallen knopen kleurt in plaats van knopen; 2-WL scheidt C_6 van twee driehoeken. Maar voor elke k bestaan er grafen waarop k -WL faalt, een resultaat van Cai, Fürer en Immerman dat de hele aanpak als route naar een polynomiaal algoritme afsloot.

Wat wel werkt

Combineer ze: eerst verfijning als goedkoop filter, brute kracht alleen wanneer verfijning onbeslist blijft.

```
def is_isomorphic(g, h):
    if wl_distinguishes(g, h):
        return False          # cheap, and decisive when it fires
    return canonical(g) == canonical(h)  # expensive, and always right
```

`canonical` neemt de lexicografisch kleinste kantenverzameling over alle $n!$ hernoemingen. Het is correct per constructie en het is geen werktuig — het is een definitie die je kunt uitvoeren. Dit is wat dat kost, bij het opsommen van elke graaf op n knopen op isomorfie na:

n=1:	1 graphs up to isomorphism	(0.00s)
n=2:	2 graphs up to isomorphism	(0.00s)
n=3:	4 graphs up to isomorphism	(0.00s)
n=4:	11 graphs up to isomorphism	(0.01s)
n=5:	34 graphs up to isomorphism	(0.53s)
n=6:	156 graphs up to isomorphism	(71.82s)

Eén knoop erbij kost **135 keer zoveel tijd**. Twee factoren stapelen zich op: het aantal gelabelde grafen is $2^{\binom{n}{2}}$, wat bij elke stap met 2^n vermenigvuldigt, en de canonieke vorm kost $n!$. Daarom draait de verificatie standaard uitputtend tot $n = 5$ en heeft ze een expliciete `--exhaustive`-vlag nodig voor $n = 6$, en daarom is $n = 7$ — 1044 isomorfieklassen, een getal dat klein oogt — met deze methode volstrekt onbereikbaar.

Echte isomorfiesoftware (`nauty`, `bliss`, `traces`) doet verfijning met individualisatie: verfijn, en wanneer de partitie stabiliseert zonder alles te scheiden, kies één knoop, geef die een unieke kleur, verfijn opnieuw, en backtrack over die keuze. In de praktijk is dat snel op vrijwel alles, ook op grafen die speciaal geconstrueerd zijn om het te verslaan. Het slechtste geval blijft exponentieel.

De aantallen zelf — 1, 2, 4, 11, 34, 156, 1044 — zijn het herkennen waard. Het is OEIS A000088, en dat de brute-force-opsommer van dit boek ze reproduceert, is een controle op `canonical` die geen stelling van mij kon leveren.

Probeer het

Kijk hoe de goedkope test faalt en de dure slaagt op hetzelfde paar:

```
python -c "
import sys, time; sys.path.insert(0, '.')
from graphs.iso import cospectral_mates, wl_distinguishes, canonical
a, b = cospectral_mates()
print('same degree sequence: ', a.degree_sequence() == b.degree_sequence())
print('refinement separates: ', wl_distinguishes(a, b))
print('actually isomorphic: ', canonical(a) == canonical(b))
"
```

```
same degree sequence:  True
refinement separates:  False
actually isomorphic:   False
```

Drie regels, en de middelste is het onderwerp van dit hoofdstuk: een test die zegt "ik kan het niet zien" is geen test die "ja" zegt.

Oefeningen

1. Zijn C_6 en twee disjuncte driehoeken isomorf? Geef een reden die geen enkele berekening vergt.
2. De graadrij is een invariant. Leg in één zin uit waarom dat haar bruikbaar maakt om isomorfie te weerleggen en onbruikbaar om haar te bewijzen.
3. Waarom slaagt kleurverfijning er niet in om twee reguliere grafen van dezelfde graad en grootte te scheiden?
4. Hoeveel grafen op 4 knopen zijn er op isomorfie na? Controleer je antwoord tegen `all_graphs_up_to_iso(4)`.

Oplossingen in Bijlage E.

Kernpunten

- Isomorfie is hernoemen. Gelijkheid van gelabelde grafen is een andere en veel sterkere relatie.
- Het probleem zit in **NP**, is niet bekend in **P**, en wordt vermoed niet **NP**-volledig te zijn — een werkelijk ongebruikelijke status. Babai's quasipolynomiale algoritme is het snelst bekende.
- Invarianten zijn eenzijdig. Een verschil weerlegt isomorfie; overeenstemming bewijst niets.
- Kleurverfijning is snel, correct, en blind voor reguliere grafen van gelijke graad. C_6 tegen twee driehoeken is het kleinste getuigenis, en geen enkele k repareert de aanpak in het algemeen.
- Canonieke vormen met brute kracht kosten $n!$ en houden op bruikbaar te zijn rond $n = 6$: 0,53 seconde bij $n = 5$, 71,8 bij $n = 6$.

Hoofdstuk 6 — Bomen

Een boom is de kleinste samenhangende graaf op zijn knopen, de grootste cykelvrije, en de enige waarin elk paar knopen op precies één manier verbonden is. Dat zijn drie verschillende zinnen die hetzelfde object beschrijven, en dit hoofdstuk bewijst dat ze het zijn.

Vijf definities

Stelling. Voor een graaf G op $n \geq 1$ knopen zijn de volgende uitspraken equivalent:

1. G is samenhangend en cykelvrij.
2. G is samenhangend en $m = n - 1$.
3. G is cykelvrij en $m = n - 1$.
4. Elke twee knopen zijn door precies één pad verbonden.
5. G is samenhangend, en elke kant verwijderen maakt hem onsamenhangend.

Een graaf die aan een van deze voldoet heet een **boom**. Een cykelvrije graaf — een disjuncte vereniging van bomen — is een **bos**.

Bewijs. We tonen $1 \implies 4 \implies 5 \implies 2 \implies 3 \implies 1$.

1 $\implies$ 4. Samenhang geeft minstens één pad tussen elke u en v . Waren er twee verschillende paden P en Q , neem dan x als de eerste knoop waarna ze uiteenlopen en y als de eerstvolgende knoop die ze delen. Het stuk van P van x naar y en het omgekeerde van Q van y naar x vormen samen een gesloten wandeling zonder herhaalde knoop behalve de eindpunten — een cykel, in tegenspraak met cykelvrijheid.

4 $\implies$ 5. Unieke paden geven samenhang meteen. Zou G na verwijdering van uv samenhangend blijven, dan bestond er een pad van u naar v dat uv vermijdt, wat samen met de kant uv twee verschillende u - v -paden geeft.

5 $\implies$ 2. Inductie naar n . Voor $n = 1$ is $m = 0$. Voor $n \geq 2$: kies een kant uv . Hem verwijderen laat precies twee componenten achter (het lemma van hoofdstuk 4 begrenst de splitsing op twee, en ze is minstens twee per aanname), zeg G_1 en G_2 met $n_1 + n_2 = n$. Beide erven eigenschap 5, dus hebben per inductie $n_i - 1$ kanten. Dan is $m = (n_1 - 1) + (n_2 - 1) + 1 = n - 1$.

2 $\implies$ 3. Stel dat G samenhangend is met $m = n - 1$ en een cykel bevat. Een kant van die cykel verwijderen houdt de graaf samenhangend — de twee eindpunten zijn nog steeds langs de lange weg verbonden — en laat $n - 2$ kanten op n knopen achter, in tegenspraak met de grens $m \geq n - 1$ van hoofdstuk 4 voor samenhangende grafen.

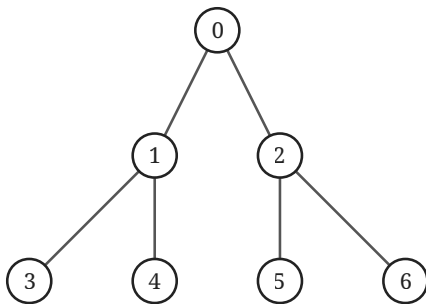
3 $\implies$ 1. Zij G cykelvrij met componenten $C_1, \dots, C_k$, elk van grootte n_i . Elke component is samenhangend en cykelvrij, dus heeft per 1 $\implies$ 2 (al bewezen via 4 en 5) $n_i - 1$ kanten. Sommeren geeft $m = n - k$. Omdat $m = n - 1$ volgt $k = 1$, dus G is samenhangend. ■

Vijf eigenschappen, één object. In de praktijk gebruik je die welke het goedkoopst is voor het argument dat voorligt — meestal 2 om te tellen, 4 voor constructies, en 5 wanneer je iets moet weglaten.

De verificatie controleert de equivalentie van 1 en 2 zonder een van beide als definitie te gebruiken: cykelvrijheid wordt getest door cyclen op te sommen, niet via de snelweg $m = n - 1$, want die gebruiken zou de bewering circulair maken.

held	ch 6	Tree iff connected and acyclic iff connected with $m = n - 1$ (52
graphs)		
held	ch 6	A tree has a unique path between any two vertices (7 graphs)
held	ch 6	Removing any edge of a tree disconnects it (7 graphs)

Bladeren



A tree: 7 vertices, 6 edges, 4 leaves

Zeven knopen, zes kanten, vier bladeren — en geworteld getekend, hoewel een boom geen wortel heeft tot je er een kiest.

Een **blad** is een knoop van graad 1.

Stelling. Elke boom op $n \geq 2$ knopen heeft minstens twee bladeren.

Bewijs. Neem een langste pad $P = v_0 v_1 \dots v_k$ in G ; omdat $n \geq 2$ en G samenhangend is, geldt $k \geq 1$. Elke buur van v_0 ligt op P — anders kon P verlengd worden, in tegenspraak met maximaliteit. En v_0 heeft geen andere buur op P dan v_1 , want $v_0 v_i$ met $i \geq 2$ zou een cykel sluiten. Dus $\deg(v_0) = 1$, en hetzelfde argument geldt bij v_k . ■

De langste-padtruc is het waard uit dit bewijs te lichten. Een maximaal object nemen en constateren dat zijn maximaliteit de uiteinden vastlegt, is een van de weinige werkelijk algemene zinnen in de grafentheorie; hij keert terug in hoofdstuk 20 voor Hamiltoniciteit en in hoofdstuk 15 voor degeneratie-ordeningen.

```

from graphs.core import Graph
from graphs.algorithms import is_tree

t = Graph(7, [(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)])
print(is_tree(t), t.n, t.m)                # True 7 6
print([v for v in t.vertices() if t.degree(v) == 1])  # [3, 4, 5, 6]
  
```

Bladeren zijn wat inductie op bomen laat werken. Vrijwel elk bewijs over bomen in dit boek gaat door een blad te verwijderen, de inductiehypothese op de kleinere boom toe te passen, en het blad terug te zetten — en de bovenstaande stelling is wat garandeert dat er altijd een blad is om te verwijderen.

Kanten toevoegen en verwijderen

Twee complementaire feiten, beide onmiddellijk uit de equivalenties, en beide voortdurend gebruikt in hoofdstuk 9:

Lemma (de verwisselingsfeiten). Zij T een boom.

- Een kant toevoegen die er nog niet is, creëert **precies één** cykel.
- Een willekeurige kant verwijderen splitst T in **precies twee** componenten.

Bewijs. Voor het eerste: T heeft per eigenschap 4 een uniek pad P van u naar v , dus uv toevoegen creëert de cykel $P + uv$. Elke cykel door de nieuwe kant moet een $u-v$ -pad in T gebruiken, en dat is er maar één, dus deze cykel is uniek. Elke cykel die niet door de nieuwe kant gaat, zou een cykel in T zijn. Voor het tweede: eigenschap 5 geeft minstens twee componenten, en het lemma van hoofdstuk 4 geeft er hoogstens twee. ■

Zet ze naast elkaar en je krijgt de **verwisselingseigenschap**: voeg je een kant toe aan een boom en verwijder je vervolgens een willekeurige kant van de ontstane cykel, dan heb je weer een boom. Die enkele waarneming is de motor van beide minimale-opspannendeboomalgoritmen in hoofdstuk 9, en van het twee-verwisselingsargument dat je al zag in hoofdstuk 3.

Bomen zijn bipartiet

Gevolg. Elke boom is bipartiet.

Bewijs. Een boom heeft geen cycli, dus in het bijzonder geen oneven cycli, en de stelling van hoofdstuk 16 geeft bipartietheid. Of rechtstreeks: wortel de boom waar dan ook en kleur elke knoop naar de pariteit van zijn diepte. Aangrenzende knopen verschillen precies één in diepte, dus ze verschillen in pariteit. ■

Het rechtstreekse argument is het bruikbare, want het geeft je ook de tweekleuring expliciet: het is de pariteit van de afstand tot de wortel.

Probeer het

Overtuig jezelf met de hand van de verwisselingseigenschap — voeg een kant toe en kijk hoe er precies één cykel verschijnt:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.algorithms import is_tree, is_connected

t = Graph(5, [(0,1),(1,2),(2,3),(3,4)])
print('a path is a tree:', is_tree(t), 'm =', t.m, 'n-1 =', t.n - 1)
t.add_edge(0, 4)
print('after adding 0-4: is_tree =', is_tree(t), 'm =', t.m)
t.remove_edge(2, 3)
print('after removing 2-3 from the cycle: is_tree =', is_tree(t), 'connected =',
is_connected(t))
"
```

```
a path is a tree: True m = 4 n-1 = 4
after adding 0-4: is_tree = False m = 5
after removing 2-3 from the cycle: is_tree = True connected = True
```

Een kant toevoegen brak de boom; een *andere* kant van de ontstane cykel verwijderen herstelde er een. Dat is de verwisselingseigenschap, en hoofdstuk 9 maakt er een algoritme van.

Oefeningen

1. Een boom heeft 12 knopen. Hoeveel kanten heeft hij?
2. Wat is het kleinste aantal bladeren dat een boom op $n \geq 2$ knopen kan hebben, en welke boom bereikt het?
3. Je voegt één kant toe aan een boom. Hoeveel cykels bevat het resultaat?
4. Elke boom is bipartiet. Is elke bipartiete graaf een boom? Geef een getuigenis.

Oplossingen in Bijlage E.

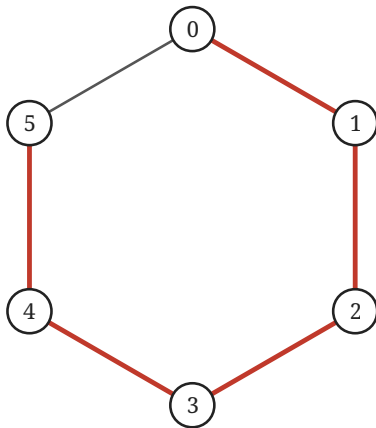
Kernpunten

- Vijf definities van een boom, allemaal equivalent. Gebruik die welke het bewijs het kortst maakt: $m = n - 1$ om te tellen, unieke paden om te construeren, "elke kant is een brug" voor verwijderingsargumenten.
- Elke boom met minstens twee knopen heeft minstens twee bladeren, bewezen met de langste-padtruc — een zet die door het hele boek terugkeert.
- Een kant toevoegen creëert precies één cykel; een kant verwijderen creëert precies twee componenten. Samen geven ze de verwisselingseigenschap, die hoofdstuk 9 aandrijft.
- Inductie op bomen betekent een blad verwijderen. De twee-bladerenstelling is wat dat altijd mogelijk maakt.

Hoofdstuk 7 — Opspannende bomen en de formule van Cayley

Een **opspannende boom** van G is een deelgraaf die een boom is en elke knoop bevat. Het is de goedkoopste manier om een graaf samenhangend te houden, en tellen hoeveel een graaf er heeft blijkt een veel rijkere vraag dan het lijkt.

Bestaan



A spanning tree of C_6 (red): 5 of the 6 edges

Een opspannende boom van C_6 : verwijder een willekeurige kant van de cykel en wat overblijft is een boom op alle zes de knopen. C_6 heeft precies zes opspannende bomen, één per verwijderde kant.

Stelling. Elke samenhangende graaf heeft een opspannende boom.

Bewijs. Neem onder alle samenhangende opspannende deelgrafen van G er een met de minste kanten; noem hem T . Bevatte T een cykel, dan zou een kant van die cykel verwijderen hem samenhangend laten (hoofdstuk 6) en minder kanten gebruiken, in tegenspraak met minimaliteit. Dus T is samenhangend en cykelvrij: een boom. ■

Dat is het **extremale argument**, en het is na de langste-padtruc de tweede algemene zet die een naam verdient. Neem het extreme object, en toon dan dat elk mankement je verder zou laten gaan. De hoofdstukken 13 en 27 gebruiken het opnieuw.

Het bewijs is geen algoritme — "neem een minimale" is geen procedure — maar er volgen meteen twee algoritmen uit, en hoofdstuk 8 geeft beide: de breedte-eerstboom en de diepte-eerstboom van een samenhangende graaf zijn opspannende bomen, elk berekend in $O(n + m)$.

Ze tellen

Hoeveel opspannende bomen heeft een graaf? Voor K_5 zegt brute kracht 125. Voor K_6 , 1296. Dat zijn 5^3 en 6^4 .

Stelling (Cayley, 1889). De volledige graaf K_n heeft precies $n^{(n-2)}$ gelabelde opspannende bomen — equivalent: er zijn $n^{(n-2)}$ verschillende bomen op een vaste knopenverzameling van grootte n .

De uitputtende telling is het ermee eens, en dat is de moeite waard voordat je enig bewijs vertrouwt:

K_2:	cayley	$n^{(n-2)}$	=	1	enumerated	=	1
K_3:	cayley	$n^{(n-2)}$	=	3	enumerated	=	3
K_4:	cayley	$n^{(n-2)}$	=	16	enumerated	=	16
K_5:	cayley	$n^{(n-2)}$	=	125	enumerated	=	125
K_6:	cayley	$n^{(n-2)}$	=	1296	enumerated	=	1296

Het woord **gelabeld** doet cruciaal werk. Op isomorfie na zijn er maar 6 bomen op 6 knopen; er zijn 1296 bomen *op de knopenverzameling* $\{0, \dots, 5\}$. Het onderscheid uit hoofdstuk 5 tussen gelijkheid en isomorfie is precies het verschil, en de twee door elkaar halen laat de formule van Cayley absurd lijken.

De Prüfer-bijectie

Er zijn verschillende bewijzen. Dit is het beste, want het telt de bomen niet alleen — het *benoemt* ze.

Stelling (Prüfer). Voor $n \geq 2$ bestaat er een bijectie tussen gelabelde bomen op $\{0, \dots, n-1\}$ en rijen van lengte $n - 2$ over $\{0, \dots, n-1\}$.

Omdat er $n^{(n-2)}$ zulke rijen zijn, volgt de formule van Cayley onmiddellijk.

Coderen. Zolang er meer dan twee knopen over zijn, zoek het blad met het kleinste label, noteer zijn unieke buur, en verwijder het blad. Stop wanneer er twee knopen resten.

```
def to_prufer(t):
    degree = [t.degree(v) for v in t.vertices()]
    neighbours = [set(t.neighbours(v)) for v in t.vertices()]
    seq = []
    for _ in range(t.n - 2):
        leaf = min(v for v in t.vertices() if degree[v] == 1)
        parent = next(iter(neighbours[leaf]))
        seq.append(parent)
        degree[leaf] = 0
        neighbours[parent].discard(leaf)
        degree[parent] -= 1
    return seq
```

Decoderen. Gegeven een rij, bereken de graad van elke knoop als één plus het aantal keren dat hij voorkomt. Neem dan herhaaldelijk de knoop met het kleinste label van graad 1 die nog niet gebruikt is, verbind hem met de volgende plaats in de rij, en verlaag beide graden. Verbind ten slotte de twee overgebleven knopen van graad 1.

Bewijs dat deze elkaars inverse zijn. De sleutelwaarneming is dat **een knoop precies $\deg(v) - 1$ keer in de rij voorkomt**. Een blad komt er dus nooit in voor, en een inwendige knoop minstens één keer. De decodeerder kan dus het bij elke stap verwijderde blad identificeren — het is het kleinste label dat niet in de resterende rij voorkomt en nog niet verbruikt is — en dat is precies het blad dat de codeerder koos. Per inductie naar n

maakt elke stap van de decodeerder de overeenkomstige stap van de codeerder ongedaan. Beide afbeeldingen zijn dus welgedefinieerd en elkaars inverse, dus elk is een bijectie. ■

Die graadwaarneming is het hele bewijs, en het is ook de snelste manier om informatie uit een Prüfer-rij te lezen zonder haar te decoderen:

```
from graphs.core import Graph, path
from graphs.generate import to_prufer, from_prufer

star = Graph(5, [(0, 1), (0, 2), (0, 3), (0, 4)])
print(to_prufer(star))          # [0, 0, 0]
print(to_prufer(path(5)))      # [1, 2, 3]
print(sorted(from_prufer([0, 0, 0]).edges()))
# [(0, 1), (0, 2), (0, 3), (0, 4)]
```

Het middelpunt van de ster heeft graad 4 en komt $4 - 1 = 3$ keer voor. De twee uiteinden van het pad hebben graad 1 en komen nooit voor. Je kunt de graadrij er rechtstreeks van aflezen.

Een gevolg dat rechtstreeks bewijzen omslachtig zou zijn, valt er gratis uit: **het aantal gelabelde bomen op n knopen met voorgeschreven graden $d_1, \dots, d_n$ is de multinomiaalcoëfficiënt $(n-2)! / \prod (d_i - 1)!$** — want dat is precies hoeveel rijen elke knoop i precies $d_i - 1$ keer laten voorkomen.

Tellen voor algemene grafen

De formule van Cayley behandelt K_n . Voor een willekeurige graaf is het antwoord de **matrix-boomstelling**: het aantal opspannende bomen is gelijk aan elke cofactor van de Laplace-matrix. Het rekent in $O(n^3)$ met een determinant, zonder iets op te sommen.

Die stelling heeft de Laplaciaan nodig en wacht dus tot hoofdstuk 30. Het is nu al de moeite waard te vermelden als de grootste kloof tussen wat dit hoofdstuk kan en wat mogelijk is: `spanning_trees` in de bibliotheek van dit boek somt $C(m, n-1)$ deelverzamelingen op en test ze elk, en daarom stopt de Cayley-controle hierboven bij K_6 . De matrix-boomstelling haalt de 1296 van K_6 uit een 5×5 -determinant.

Probeer het

Kijk hoe de bijectie heen en terug loopt op een willekeurige boom, en controleer de graadwaarneming:

```
python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.generate import random_tree, to_prufer, from_prufer
rng = random.Random(11)
t = random_tree(8, rng)
seq = to_prufer(t)
print('prufer sequence:', seq)
print('degrees:         ', [t.degree(v) for v in t.vertices()])
print('appearances+1:   ', [seq.count(v) + 1 for v in t.vertices()])
print('round trips:     ', sorted(from_prufer(seq).edges()) == sorted(t.edges()))
"
```

```

prufer sequence: [7, 7, 7, 3, 2, 7]
degrees:         [1, 1, 2, 2, 1, 1, 1, 5]
appearances+1:   [1, 1, 2, 2, 1, 1, 1, 5]
round trips:      True

```

De tweede en derde regel zijn identiek, en dat is het lemma waar het hele bewijs om draait.

Oefeningen

1. Hoeveel gelabelde opspannende bomen heeft K_4 ? Controleer tegen de formule van Cayley.
2. Bereken met de hand de Prüfer-rij van het pad $0-1-2-3$.
3. Een knoop komt $\deg(v) - 1$ keer voor in een Prüfer-rij. Wat zegt dat over de labels die er nooit in voorkomen?
4. Er zijn 1296 gelabelde bomen op 6 knopen maar slechts 6 op isomorfie na. Verklaar het verschil in één zin.

Oplossingen in Bijlage E.

Kernpunten

- Elke samenhangende graaf heeft een opspannende boom, via het extremale argument: neem een minimale samenhangende opspannende deelgraaf en toon dat hij geen cykel kan bevatten.
- K_n heeft n^{n-2} **gelabelde** opspannende bomen. Op isomorfie na is het aantal veel kleiner, en de twee door elkaar halen laat de formule fout lijken.
- De Prüfer-bijectie bewijst de formule van Cayley door elke boom van een naam te voorzien. Het lemma dat haar draagt: knoop v komt precies $\deg(v) - 1$ keer voor.
- Boomtellingen met voorgeschreven graden vallen zonder extra werk uit dezelfde bijectie, als een multinomiaalcoëfficiënt.
- Opspannende bomen van een algemene graaf tellen is de matrix-boomstelling, $O(n^3)$ via een determinant. De opsommer van dit hoofdstuk is exponentieel en stopt bij K_6 .

Hoofdstuk 8 — Doorlopen

Breedte-eerst en diepte-eerst zoeken bezoeken dezelfde knopen in $O(n + m)$ tijd. Ze verschillen alleen in welke knoop ze als volgende nemen — een wachtrij tegenover een stapel — en dat ene verschil geeft ze volstrekt verschillende toepassingen.

De twee volgordes

```
from graphs.core import Graph
from graphs.algorithms import bfs_order, dfs_order, distances

g = Graph(6, [(0, 1), (0, 2), (1, 3), (2, 3), (3, 4), (4, 5)])
print(bfs_order(g, 0))    # [0, 1, 2, 3, 4, 5]
print(dfs_order(g, 0))    # [0, 1, 3, 2, 4, 5]
print(distances(g, 0))    # {0: 0, 1: 1, 2: 1, 3: 2, 4: 3, 5: 4}
```

Breedte-eerst bezoekt 2 vóór 3, omdat 2 dichterbij de bron ligt. Diepte-eerst bezoekt 3 vóór 2, omdat het eerste pad zo ver mogelijk volgt voordat het terugkeert. Beide kosten $O(n + m)$: elke knoop komt één keer in de houder, en elke kant wordt twee keer bekeken — één keer vanaf elk eindpunt.

Het enige structurele verschil in de code is de houder:

```
queue.popleft()    # BFS: first in, first out
stack.pop()        # DFS: last in, first out
```

Merk op dat `dfs_order` hier **iteratief** is. De recursieve versie is korter en is wat de meeste teksten afdrukken, maar de standaardrecursielimiet van Python is 1000, dus een padgraaf op tienduizend knopen laat haar crashen. De iteratieve versie kent die grens niet, en dit boek gebruikt haar overal.

Waarom breedte-eerst kortste paden vindt

De correctheid van BFS is het enige in dit hoofdstuk dat werkelijk bewijs vergt.

Stelling. BFS vanuit s kent elke bereikbare v de waarde $d(s, v)$ toe, de lengte van een kortste pad.

Bewijs. Zij $\text{dist}[v]$ de waarde die BFS toekent. We tonen $\text{dist}[v] = d(s, v)$ per inductie naar $d(s, v)$.

Ten eerste geldt altijd $\text{dist}[v] \geq d(s, v)$, want BFS zet $\text{dist}[w] = \text{dist}[v] + 1$ alleen wanneer vw een kant is, dus de toegekende waarden tekenen een werkelijke wandeling vanaf s uit.

Voor de andere richting: stel $d(s, v) = k$ en dat elke knoop op werkelijke afstand $< k$ correct gelabeld is. Neem een kortste $s-v$ -pad en zij u de knoop ervoor op dat pad, dus $d(s, u) = k-1$ en per inductie $\text{dist}[u] = k-1$. De wachtrij wordt in niet-dalende volgorde van dist verwerkt — dat is de invariant die BFS laat werken, en ze geldt omdat we alleen waarden toevoegen die één groter zijn dan de waarde die verwerkt wordt. Dus u verlaat de rij voordat enige knoop van afstand k of meer verwerkt wordt, en op dat moment is v ofwel al gelabeld met iets $\leq k$, ofwel krijgt hij k . Hoe dan ook $\text{dist}[v] \leq k$. ■

De invariant — **de wachtrij bevat op elk moment hoogstens twee verschillende afstandswaarden, en die zijn opeenvolgend** — is het ding om te onthouden. Ze faalt zodra kanten verschillende gewichten hebben, en dat is precies waarom hoofdstuk 10 Dijkstra en een prioriteitswachtrij nodig heeft in plaats van een gewone rij.

De verificatie controleert de stelling tegen uitputtend opgesomde paden en niet tegen een tweede kortstepadroutine:

held	ch 8	BFS distances equal true shortest-path lengths (52 graphs)
held	ch 8	DFS and BFS reach exactly the same vertices (52 graphs)

Zoekbomen, en de kanten die ze achterlaten

Beide zoekmethoden bouwen een **zoekboom**: onthoud voor elke knoop behalve de bron de kant waarlangs hij het eerst ontdekt werd. Omdat elke knoop precies één keer ontdekt wordt, zijn dat $n - 1$ kanten op een samenhangende graaf, en hij is samenhangend, dus het is een opspannende boom — de bestaansstelling van hoofdstuk 7, nu constructief gemaakt.

Elke kant van de graaf is vervolgens ofwel een **boomkant** ofwel niet. De kanten die het niet zijn classificeren is waar de twee zoekmethoden het scherpst uiteenlopen, en het is de grondslag van hoofdstuk 12.

Voor **diepte-eerst op een ongerichte graaf** is elke niet-boomkant een **terugkant**: ze verbindt een knoop met een van zijn eigen voorouders in de boom.

Bewijs. Beschouw een kant uv en stel dat u het eerst ontdekt wordt. DFS keert niet terug uit u voordat elke knoop die via onbezochte knopen vanuit u bereikbaar is, afgehandeld is — in het bijzonder wordt v ontdekt tijdens de verkenning van u , dus v is een afstammeling van u . Dus uv verbindt een knoop met een voorouder. ■

Er zijn geen kruiskanten in ongericht DFS. Dat is een sterke uitspraak, en ze maakt DFS het juiste gereedschap om bruggen, snijknopen en biconnexe componenten te vinden: de enige manier om "boven" je huidige positie te komen is een terugkant, dus bijhouden welke hoogste voorouder vanuit elke deelboom bereikbaar is, vertelt je precies welke boomkanten bruggen zijn. Hoofdstuk 12 bouwt dat algoritme.

Voor **breedte-eerst op een ongerichte graaf** gaat het overeenkomstige feit over niveaus: elke kant verbindt knopen waarvan de afstanden tot de bron **hoogstens één** verschillen. Een kant naar een knoop twee niveaus lager zou het pad hebben verkort. Elke niet-boomkant ligt dus ofwel binnen een niveau ofwel tussen opeenvolgende niveaus — en een kant *binnen* een niveau is precies een oneven cykel die erop wacht gevonden te worden, wat het tweekleuringsalgoritme van hoofdstuk 16 is.

Welke je gebruikt

Probleem	Zoekmethode	Waarom
Kortste paden, ongewogen	BFS	De niveau-invariant is de afstand
Samenhangscomponenten	beide	Beide bereiken dezelfde verzameling
Bipartietheid / oneven cykel	BFS	Een kant binnen een niveau benoemt de oneven cykel
Bruggen, snijknopen	DFS	Geen kruiskanten, dus terugkanten vertellen alles
Topologische ordening (gericht)	DFS	Afhandelsonderzoek, omgekeerd
Cykeldetectie	beide	Een terugkant in DFS; een kant binnen of terug in BFS

Het enige geval waarin de keuze een fout is in plaats van een voorkeur, is kortste paden. DFS vindt *een* pad, en mensen gebruiken het daar ook voor, maar het pad dat het vindt kan willekeurig veel langer zijn dan het kortste — op een graaf met een lange omweg neemt DFS die vrolijk.

Probeer het

Kijk hoe DFS een pad teruggeeft dat veel langer is dan het kortste, op een graaf die ontworpen is om het te misleiden:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.algorithms import bfs_order, dfs_order, distances
# 0 and 7 are adjacent; there is also a long way round through 1..6
g = Graph(8, [(0,7)] + [(i, i+1) for i in range(7)])
print('bfs order:', bfs_order(g, 0))
print('dfs order:', dfs_order(g, 0))
print('true distance 0 to 7:', distances(g, 0)[7])
print('position of 7 in dfs:', dfs_order(g, 0).index(7))
"
```

```
bfs order: [0, 1, 7, 2, 6, 3, 5, 4]
dfs order: [0, 1, 2, 3, 4, 5, 6, 7]
true distance 0 to 7: 1
position of 7 in dfs: 7
```

BFS bereikt knoop 7 als tweede, omdat hij één stap verwijderd is. DFS bereikt hem als laatste, na eerst de hele omweg van zeven kanten te hebben afgelegd. Beide zijn correcte doorlopen; slechts één ervan weet iets over afstand.

Oefeningen

1. Je hebt kortste paden nodig in een ongewogen graaf. BFS of DFS, en waarom is de andere fout in plaats van slechts trager?
2. Wat is de looptijd van beide zoekmethoden, en waarom is die dezelfde?
3. In een BFS vind je een kant tussen twee knopen in hetzelfde niveau. Wat heb je gevonden?

4. Waarom schrijft dit boek DFS iteratief in plaats van recursief?

Oplossingen in Bijlage E.

Kernpunten

- BFS en DFS verschillen alleen door wachtrij tegenover stapel, en beide lopen in $O(n + m)$.
- BFS berekent exacte kortstepadaafstanden op ongewogen grafen. De invariant is dat de wachtrij hoogstens twee opeenvolgende afstandswaarden bevat — en dat is precies wat ongelijke kantgewichten kapotmaken, waarom hoofdstuk 10 bestaat.
- Beide bouwen opspannende bomen, wat de bestaansstelling van hoofdstuk 7 constructief maakt.
- Ongericht DFS heeft **geen kruiskanten**: elke niet-boomkant is een terugkant. Dat maakt het tot het gereedschap voor bruggen en snijknopen.
- Ongericht BFS legt elke kant binnen een niveau of tussen opeenvolgende niveaus. Een kant binnen een niveau is een oneven cykel.
- Schrijf DFS iteratief. De recursieve vorm sterft aan de grens van 1000 frames van Python op een graaf die enkel lang is.

Hoofdstuk 9 — Minimale opspannende bomen

Geef de kanten gewichten en vraag om de goedkoopste opspannende boom. Twee gulzige algoritmen lossen het op, en gulzige algoritmen werken bijna nooit — dus de interessante inhoud is de ene eigenschap die beide correct maakt.

De snede-eigenschap

Een **snede** is een verdeling van de knopen in twee niet-lege delen $(S, V \setminus S)$. Een kant **kruist** de snede wanneer ze in elk deel één eindpunt heeft.

Stelling (snede-eigenschap). Zij $(S, V \setminus S)$ een willekeurige snede, en zij e een kruisende kant van strikt minimaal gewicht onder de kruisende kanten. Dan behoort e tot elke minimale opspannende boom.

Bewijs. Zij T een opspannende boom die $e = uv$ niet bevat. e aan T toevoegen creëert precies één cykel (het verwisselingsfeit uit hoofdstuk 6). Die cykel kruist de snede bij e , en een cykel kruist elke snede een even aantal keren, dus hij kruist bij nog een kant $f \neq e$. Nu is $T + e - f$ weer een opspannende boom, en omdat $w(e) < w(f)$ door de strikte minimaliteit van e , is hij strikt lichter dan T . Dus T was niet minimaal. ■

Twee dingen in dat bewijs verdienen hun plaats. Het feit dat een cykel een snede **een even aantal keren** kruist — loop de cykel rond en telkens als je S verlaat moet je terugkomen — wordt opnieuw gebruikt in de hoofdstukken 13 en 17. En het verwisselingsfeit uit hoofdstuk 6 is wat $T + e - f$ tot een boom maakt, precies de reden waarom hoofdstuk 6 de moeite nam het te bewijzen.

Het woord **strikt** draagt gewicht, en het weglaten ervan is de standaardfout. Delen twee kruisende kanten het minimum, dan hoeft geen van beide in elke MOB te zitten; elk zit in *een* MOB. De stelling zoals ze er staat is precies juist, en de versie zonder "strikt" is onwaar.

Er is een tegenhanger, op dezelfde manier bewezen:

Stelling (cykeleigenschap). Is e de unieke zwaarste kant van een cykel, dan behoort e tot geen enkele minimale opspannende boom.

Kruskal

Sorteer de kanten op gewicht en voeg elke kant toe tenzij ze een cykel zou sluiten.

```
def kruskal(g):
    uf = UnionFind(g.n)
    chosen = []
    for u, v, _ in sorted(g.edges(), key=lambda e: e[2]):
        if uf.union(u, v):
            chosen.append((u, v))
    return chosen
```

Correctheid. Wanneer Kruskal een kant $e = uv$ aanvaardt, zij S de verzameling knopen die al via gekozen kanten met u verbonden zijn. Elke eerder bekeken kant die $(S, V \setminus S)$ kruist werd verworpen, en kanten

worden in volgorde van gewicht bekeken, dus `e` is een kruisende kant van minimaal gewicht. De snede-eigenschap is van toepassing. ■

De looptijd is $O(m \log m)$ en zit volledig in het sorteren; de union-find-operaties zijn effectief constant. Merk op dat Kruskal op een **onsamenhangende** graaf een minimaal opspannend **bos** teruggeeft — één boom per component — in plaats van te falen. Dat is meestal wat je wilt, en het is goed te weten dat je dat krijgt.

Union-find is de ondersteunende structuur: `find` geeft de vertegenwoordiger van een verzameling, `union` voegt twee verzamelingen samen en meldt of ze al dezelfde waren. Met padcompressie en samenvoegen op grootte kosten m operaties $O(m \alpha(n))$, waarbij α de inverse Ackermann-functie is — hoogstens 4 voor elke n die in het waarneembare heelal past. Dit boek behandelt haar als constant en zegt er ronduit bij dat ze dat niet is.

Prim

Laat één boom groeien vanuit een startknoop, en neem steeds de goedkoopste kant die hem verlaat.

```
def prim(g, source=0):
    seen = {source}
    frontier = [(g.weight(source, x), source, x) for x in g.neighbours(source)]
    heapq.heapify(frontier)
    chosen = []
    while frontier:
        _, u, v = heapq.heappop(frontier)
        if v in seen:
            continue
        seen.add(v)
        chosen.append((min(u, v), max(u, v)))
        for x in g.neighbours(v):
            if x not in seen:
                heapq.heappush(frontier, (g.weight(v, x), v, x))
    return chosen
```

Correctheid. Bij elke stap is `seen` één zijde van een snede en neemt het algoritme een kruisende kant van minimaal gewicht. De snede-eigenschap is rechtstreeks van toepassing. ■

$O(m \log n)$ met een binaire heap. Anders dan Kruskal spant Prim alleen de component van de bron — hij heeft geen manier om naar een andere te springen.

De twee algoritmen maken werkelijk verschillende keuzes over waarover ze gulzig zijn: Kruskal is gulzig over *alle* kanten globaal, Prim over de kanten die één groeiende boom verlaten. Dat beide werken, is de snede-eigenschap toegepast op twee verschillende families sneden.

De MOB is niet uniek

Hier is het misverstand waarvoor dit hoofdstuk bestaat. "De" minimale opspannende boom suggereert dat er één is. Er is precies één minimaal **gewicht**; er kunnen vele bomen zijn die het bereiken.

De verificatie registreert dit als een stelling waarvan verwacht wordt dat ze weerlegd wordt:

refuted	ch 9	Kruskal and Prim always choose the same edges (5 graphs)
held	ch 9	Kruskal and Prim both achieve the true minimum weight (91 graphs)

De tweede regel is de stelling; de eerste is het misverstand, en het wordt weerlegd in plaats van bewezen.

Die controle aan de praat krijgen vergde een correctie die het optekenen waard is. Bij de eerste poging kwamen de gewichten uit 1..20, en de twee algoritmen waren het eens op **alle 79 grafen** — de bewering hield stand, en zag eruit als een stelling. Met gewichten uit 1..2, zodat gelijke gewichten vaak voorkomen, waren ze het oneens op **66 van de 3000**. Gelijke gewichten zijn het hele mechanisme, en een willekeurige familie zonder gelijke gewichten kan het verschijnsel helemaal niet zien. Zijn alle gewichten verschillend, dan is de MOB werkelijk uniek — een gevolg van de snede-eigenschap met haar striktheid intact.

```
from graphs.weighted import WeightedGraph
from graphs.mst import kruskal, prim, brute_force_mst, spanning_trees

wg = WeightedGraph(5, [(0,1,1), (0,2,3), (1,2,2), (1,3,6), (2,3,4), (3,4,5), (2,4,7)])
print(sorted(kruskal(wg)))           # [(0, 1), (1, 2), (2, 3), (3, 4)]
print(brute_force_mst(wg)[1])        # 12
print(len(spanning_trees(wg.graph))) # 21
```

Eenentwintig opspannende bomen, één van gewicht 12, en beide algoritmen vinden hem.

Verificatie zonder circulariteit

Kruskal tegen Prim controleren zou alleen vaststellen dat twee van mijn gulzige implementaties het eens zijn — wat ze kunnen zijn terwijl ze allebei fout zijn, want ze delen dezelfde redenering rond de snede-eigenschap en dezelfde auteur. Daarom somt de verificatie **elke** opspannende boom op en neemt ze de goedkoopste:

```
def brute_force_mst(g):
    best = None
    for tree in spanning_trees(g.graph):
        w = g.subgraph_weight(tree)
        if best is None or w < best[1]:
            best = (tree, w)
    return best
```

$C(m, n-1)$ deelverzamelingen, dus het is begrensd op zeven knopen. Dat volstaat: een fout in een gulzig algoritme laat zich op kleine grafen zien, want de gulzige keuze is lokaal fout of helemaal niet.

Probeer het

Kijk hoe de twee algoritmen het oneens zijn over de boom terwijl ze het eens zijn over het gewicht:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.weighted import WeightedGraph
from graphs.mst import kruskal, prim
# C_4 with every weight 1, so every spanning tree is minimum
g = WeightedGraph(4, [(0,2,1),(0,3,1),(1,2,1),(1,3,1)])
k = sorted(kruskal(g))
p = sorted((min(u,v), max(u,v)) for u,v in prim(g, source=1))
print('kruskal:', k, 'weight', g.subgraph_weight(k))
print('prim:    ', p, 'weight', g.subgraph_weight(p))
print('same weight:', g.subgraph_weight(k) == g.subgraph_weight(p), ' same
edges:', k == p)
"
```

```
kruskal: [(0, 2), (0, 3), (1, 2)] weight 3
prim:    [(0, 2), (1, 2), (1, 3)] weight 3
same weight: True    same edges: False
```

Verschillende bomen, identieke kosten. Beide zijn minimale opspannende bomen; geen van beide is *de* minimale opspannende boom.

Vier knopen is het kleinste geval waarop dit kan gebeuren, gevonden door uitputtend zoeken en niet door te gissen — en de eerste twee grafen die ik gokte gaven beide algoritmen die het eens waren.

Oefeningen

1. Formuleer de snede-eigenschap, en zeg welk woord erin het gewicht draagt.
2. Geef een graaf met twee verschillende minimale opspannende bomen van gelijk gewicht.
3. Wat geeft Kruskal terug wanneer de invoergraaf onsaamenhangend is?
4. Onder welke voorwaarde is de minimale opspannende boom uniek?

Oplossingen in Bijlage E.

Kernpunten

- De snede-eigenschap is het hele onderwerp: de strikt lichtste kant over een willekeurige snede zit in elke MOB. Beide algoritmen zijn gevolgen ervan.
- "Strikt" is geen versiering. Zonder dat woord is de stelling onwaar, en met gelijke gewichten houdt de MOB op uniek te zijn.
- Kruskal is globaal gulzig en geeft een bos op onsaamenhangende invoer; Prim is gulzig vanuit één groeiende boom en spant alleen de component van de bron.
- Union-find kost $O(\alpha(n))$ per operatie, wat niet constant is en hoogstens 4.
- Verschillende gewichten $\implies$ unieke MOB. Gelijke gewichten $\implies$ vele. Een willekeurige testfamilie met breed gespreide gewichten laat je dit nooit zien: 0 meningsverschillen op 79 grafen bij gewichten 1..20, 66 op 3000 bij gewichten 1..2.

Hoofdstuk 10 — Kortste paden

Geef de kanten gewichten en breedte-eerst zoeken houdt op te werken. Dit hoofdstuk gaat over waarom, wat ervoor in de plaats komt, en wat een negatief gewicht met de hele vraag doet.

Richtingen, en waarom ze nu pas komen

Alles tot hier was ongericht. Vanaf nu heeft het boek **bogen** nodig, en de reden is specifiek: een negatief gewicht op een ongerichte kant is betekenisloos. Je zou hem kunnen oversteken, terugsteken, en opnieuw oversteken, en het totaal onbegrensd laten dalen. Negatieve gewichten hebben alleen zin wanneer de kant één kant op gaat, dus zodra dit hoofdstuk ze toelaat, moet het richting toelaten.

```
from graphs.digraph import Digraph

d = Digraph(5, [(0,1,4), (0,2,1), (2,1,2), (1,3,1), (2,3,5), (3,4,3)])
```

Een ongerichte gewogen graaf is het bijzondere geval waarin elke boog een tweelingboog heeft. `Digraph.of(g)` bouwt hem, en elk resultaat hieronder geldt dan onveranderd.

Dijkstra

Houd een verzameling **afgehandelde** knopen bij waarvan de afstand definitief is. Neem herhaaldelijk de niet-afgehandelde knoop met de kleinste voorlopige afstand, handel hem af, en relaxeer zijn uitgaande bogen.

```
def dijkstra(d, source):
    dist = {source: 0.0}
    heap = [(0.0, source)]
    settled = set()
    while heap:
        du, u = heapq.heappop(heap)
        if u in settled:
            continue
        settled.add(u)
        for v in d.successors(u):
            alt = du + d.weight(u, v)
            if alt < dist.get(v, INF):
                dist[v] = alt
                heapq.heappush(heap, (alt, v))
    return dist
```

```
print(dijkstra(d, 0))    # {0: 0.0, 1: 3.0, 2: 1.0, 3: 4.0, 4: 7.0}
```

De route naar 1 kost 3 en niet de 4 van de rechtstreekse boog, want via 2 kost het 1 + 2.

Stelling. Met niet-negatieve gewichten handelt Dijkstra elke bereikbare knoop af met zijn werkelijke kortste afstand.

Bewijs. Per inductie naar de volgorde van afhandelen. Stel dat elke eerder afgehandelde knoop zijn werkelijke afstand heeft, en zij `u` de knoop die nu afgehandeld wordt met voorlopige waarde `du`. Elk pad van `s` naar `u`

moet op enig moment de afgehandelde verzameling verlaten: zij xy de eerste boog die dat doet, met x afgehandeld. Dat pad kost dan minstens $\text{dist}[x] + w(x,y)$, wat minstens de voorlopige waarde van y is, wat minstens du is omdat u als kleinste gekozen werd. **De laatste ongelijkheid vereist dat elk resterend booggewicht niet-negatief is** — de rest van het pad van y naar u kan er alleen bij optellen. Dus geen pad verslaat du . ■

De voorwaarde wordt op precies één plaats gebruikt, en dat is de plaats waar ze faalt.

Wat een negatieve boog werkelijk doet

De gebruikelijke formulering is "Dijkstra faalt op negatieve gewichten", wat waar is maar niet informatief — en de naïeve verklaring, "zodra een knoop is afgehandeld wordt hij nooit meer verbeterd", is niet helemaal wat er in deze implementatie gebeurt. Kijk:

```
d = Digraph(4, [(0,1,-1), (0,2,-1), (1,3,-1), (2,1,-1)])
print(dijkstra(d, 0))      # {0: 0.0, 1: -2.0, 2: -1.0, 3: -2.0}
print(bellman_ford(d, 0)[0]) # {0: 0.0, 1: -2.0, 2: -1.0, 3: -3.0}
```

Kijk naar knoop 1. Dijkstra meldt -2 , en dat is **correct** — de verbetering via 2 werd gevonden en vastgelegd. Het is knoop 3 die fout is: -2 in plaats van -3 .

Dat is de werkelijke faalwijze. Knoop 1 werd afgehandeld bij -1 , en zijn bogen werden toen gerelaxeerd. Toen de betere waarde -2 later arriveerde, werd $\text{dist}[1]$ bijgewerkt, maar 1 zat al in `settled`, dus hem opnieuw uit de heap halen deed niets en **de verbetering plantte zich nooit voort naar 3**. De fout is niet dat afgehandelde knopen niet kunnen verbeteren; het is dat hun verbetering niet kan reizen.

Dit is de kleinste graaf waarop het gebeurt — vier knopen en vier bogen, alle van gewicht -1 , gevonden door uitputtend zoeken en niet door constructie. De verificatie registreert het als een stelling waarvan verwacht wordt dat ze weerlegd wordt:

held	ch10	Dijkstra is correct when weights are non-negative (120 graphs)
refuted	ch10	Dijkstra is correct when weights may be negative (3 graphs)

Merk op dat grafen die een negatieve cykel *bevatten* uit die tweede familie zijn uitgesloten. De bewering wordt weerlegd op grafen waar een correct antwoord bestaat en Dijkstra het niet vindt, wat een scherpere uitspraak is dan "het gaat stuk wanneer het probleem slecht gesteld is".

Bellman–Ford

Geef het afhandelen helemaal op. Relaxeer gewoon elke boog, $n - 1$ keer.

```
def bellman_ford(d, source):
    dist = {v: INF for v in d.vertices()}
    dist[source] = 0.0
    for _ in range(d.n - 1):
        for u, v, w in d.arcs():
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in d.arcs():
        if dist[u] + w < dist[v]:
            return dist, True          # a negative cycle is reachable
    return dist, False
```

Correctheid. Na k ronden is $\text{dist}[v]$ hoogstens het gewicht van het lichtste pad van s naar v dat hoogstens k bogen gebruikt. Dat volgt meteen per inductie: ronde k relaxeert de laatste boog van zo'n pad, waarvan het beginstuk door ronde $k-1$ is afgehandeld. Een kortste pad zonder negatieve cykel is enkelvoudig, gebruikt dus hoogstens $n - 1$ bogen, en $n - 1$ rondes volstaan. ■

$O(nm)$, wat veel slechter is dan Dijkstra's $O(m \log n)$ — de prijs voor het aankunnen van negatieve bogen.

Negatieve cyclen

Is een negatieve cykel bereikbaar, dan bestaat er geen kortste pad: ga nog eens rond en het totaal daalt. De vraag wordt slecht gesteld, niet slechts moeilijk.

De laatste lus van Bellman–Ford is precies de test. Na $n - 1$ rondes is geen verdere verbetering mogelijk tenzij een pad meer dan $n - 1$ bogen met winst kan gebruiken, en dat vereist een negatieve cykel.

```
neg = Digraph(3, [(0,1,1), (1,2,-3), (2,0,1)])
print(bellman_ford(neg, 0)[1])    # True
```

De verificatie controleert dit tegen een onafhankelijke opsomming van elke enkelvoudige cykel, en niet tegen Floyd–Warshall, dat de rekenkunde van dit boek deelt:

```
held      ch10  Bellman-Ford flags exactly the reachable negative cycles (300
graphs)
```

Bereikbaar hoort bij de uitspraak. Een negatieve cykel in een verre hoek van de graaf maakt de afstanden vanuit s niet slecht gedefinieerd, en Bellman–Ford vanuit s negeert hem terecht. Floyd–Warshall in hoofdstuk 11 ziet de hele graaf en beantwoordt dus een net andere vraag.

Eén gevolg is het vermelden waard, want het verklaart waarom mensen erom geven: met negatieve bogen toegestaan is het vinden van het *kortste enkelvoudige pad* NP-moeilijk — een Hamiltoniaans pad is een kortste enkelvoudig pad bij de juiste gewichten (hoofdstuk 20). Bellman–Ford lost dat niet op. Het lost de kortste *wandeling* op, en die valt samen met het kortste enkelvoudige pad precies wanneer geen negatieve cykel bereikbaar is.

Kiezen

Situatie	Gebruik	Kosten
Ongewogen	BFS	$O(n + m)$
Niet-negatieve gewichten	Dijkstra	$O(m \log n)$
Willekeurige gewichten, één bron	Bellman–Ford	$O(nm)$
Willekeurige gewichten, alle paren	Floyd–Warshall (h. 11)	$O(n^3)$
Negatieve cykel bereikbaar	geen — de vraag is slecht gesteld	—

BFS is Dijkstra met alle gewichten gelijk aan 1, en de prioriteitswachtrij valt terug op een gewone wachtrij omdat de invariant uit hoofdstuk 8 — hoogstens twee opeenvolgende afstandswaarden onderweg — weer geldt.

Probeer het

Kijk hoe de verbetering zich niet voortplant, knoop voor knoop:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.digraph import Digraph
from graphs.paths import dijkstra, bellman_ford, brute_force_shortest
d = Digraph(4, [(0,1,-1), (0,2,-1), (1,3,-1), (2,1,-1)])
dj = dijkstra(d, 0)
bf = bellman_ford(d, 0)[0]
for t in range(4):
    print(f' to {t}: dijkstra={dj[t]:>5} bellman={bf[t]:>5}
true={brute_force_shortest(d,0,t):>5}')
"
```

```
to 0: dijkstra= 0.0 bellman= 0.0 true= 0.0
to 1: dijkstra= -2.0 bellman= -2.0 true= -2
to 2: dijkstra= -1.0 bellman= -1.0 true= -1
to 3: dijkstra= -2.0 bellman= -3.0 true= -3
```

Drie van de vier zijn juist. Alleen de knoop *stroomafwaarts* van de late verbetering is fout, en daarom is deze fout makkelijk te missen op een graaf die je niet uitputtend hebt gecontroleerd.

Oefeningen

1. Dijkstra's bewijs gebruikt niet-negativiteit in precies één ongelijkheid. Welke?
2. Wat is de looptijd van Bellman–Ford, en waarom is die slechter dan die van Dijkstra?
3. Wat detecteert de n -de relaxatieronde van Bellman–Ford?
4. Een negatieve cykel is bereikbaar vanuit s . Wat is de kortste wandeling van s naar een knoop op die cykel?

Oplossingen in Bijlage E.

Kernpunten

- Negatieve gewichten dwingen richting af, en daarom stapt het boek hier over op gerichte grafen.
- Dijkstra's bewijs gebruikt niet-negativiteit in precies één ongelijkheid, en dat is precies waar het breekt.
- Dijkstra's werkelijke falen is niet dat afgehandelde knopen nooit verbeteren — dat kunnen ze — maar dat de verbetering zich niet voorbij hen kan voortplanten. Het kleinste getuigenis heeft vier knopen.
- Bellman–Ford relaxeert alles $n - 1$ keer, hanteert negatieve bogen in $O(nm)$, en zijn n -de ronde is precies een test op een bereikbare negatieve cykel.
- Met een bereikbare negatieve cykel bestaat er geen kortste wandeling. Kortste *enkelvoudige* paden met negatieve gewichten is NP-moeilijk, en geen algoritme hier probeert het.

Hoofdstuk 11 — Afstand tussen alle paren

Dijkstra vanuit elke knoop draaien kost $O(nm \log n)$. Op een dichte graaf bestaat er iets eenvoudigers en, opmerkelijk genoeg, snellers — en het is vier regels.

Floyd–Warshall

Het idee is een andere inductie dan elk algoritme tot nu toe. In plaats van naar buiten te groeien vanuit een bron, beperk je welke knopen een pad **doorheen** mag gaan.

Zij $d_k(u, v)$ de lengte van het kortste u – v -pad waarvan alle inwendige knopen in $\{0, \dots, k-1\}$ liggen. Dan is d_0 gewoon het booggewicht, en

$$d_{k+1}(u, v) = \min(d_k(u, v), d_k(u, k) + d_k(k, v))$$

want een pad dat k mag gebruiken doet dat ofwel niet — de eerste term — ofwel wel, en dan gaat het precies één keer door k en valt het uiteen in twee paden die dat niet doen.

Dat "precies één keer" vraagt om rechtvaardiging: een kortste pad bezoekt geen knoop twee keer, mits er geen negatieve cykel is. De recursie is dus geldig onder dezelfde voorwaarde die Bellman–Ford nodig heeft, en faalt zonder haar op dezelfde manier.

```
def floyd_warshall(d):
    n = d.n
    dist = [[INF] * n for _ in range(n)]
    for v in range(n):
        dist[v][v] = 0.0
    for u, v, w in d.arcs():
        dist[u][v] = min(dist[u][v], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

$O(n^3)$, drie geneste lussen, geen prioriteitswachtrij en helemaal geen datastructuur. **De k -lus moet de buitenste zijn.** Dat is het enige wat je goed moet krijgen: k indexeert de inductie, en i en j doorlopen binnen een vaste fase. Verwissel de lussen en je krijgt een algoritme dat soms correcte antwoorden geeft, en dat is de ergste soort fout.

```
d = Digraph(5, [(0,1,4), (0,2,1), (2,1,2), (1,3,1), (2,3,5), (3,4,3)])
for row in floyd_warshall(d):
    print(row)
```

```
0 [0, 3, 1, 4, 7]
1 [inf, 0, inf, 1, 4]
2 [inf, 2, 0, 3, 6]
3 [inf, inf, inf, 0, 3]
4 [inf, inf, inf, inf, 0]
```

Rij 0 komt exact overeen met de Dijkstra-uitvoer van hoofdstuk 10. De oneindigheden zijn eerlijk: deze digraaf heeft geen bogen terug naar 0, dus niets bereikt hem.

Negatieve gewichten, en een andere vraag

Floyd–Warshall hanteert negatieve bogen zonder aanpassing, wat Dijkstra niet kan. En het detecteert negatieve cyclen anders dan Bellman–Ford, op een manier die precisie verdient:

```
neg = Digraph(3, [(0,1,1), (1,2,-3), (2,0,1)])
print([floyd_warshall(neg)[v][v] for v in range(3)]) # [-1, -1, -2]
```

Een **negatieve plaats op de diagonaal** betekent dat v op een negatieve cykel ligt. Bellman–Ford vanuit een bron s meldt of een negatieve cykel *bereikbaar is vanuit* s ; Floyd–Warshall meldt welke knopen er *op* liggen. Dat zijn verschillende vragen, en beide zijn nuttig:

- De versie van Bellman–Ford vertelt je dat je afstanden vanuit s betekenisloos zijn.
- Die van Floyd–Warshall vertelt je welk deel van de graaf het probleem is.

De -2 bij knoop 2 is geen afstand. Zodra er een negatieve cykel bestaat, zijn de getallen in de matrix wat het vaste aantal relaxatieronden toevallig opleverde, en alleen hun teken is betekenisvol.

De metriek

Voor een ongerichte graaf met niet-negatieve gewichten is d een echte **metriek**: niet-negatief, nul precies op de diagonaal, symmetrisch, en voldoende aan de driehoeksongelijkheid. De verificatie controleert die laatste rechtstreeks:

```
held      ch11 Graph distance satisfies the triangle inequality (120 graphs)
held      ch11 Floyd-Warshall agrees with enumerating every simple path (120
graphs)
```

Symmetrie is degene die faalt voor digrafen, en ze faalt grondig — $d(u,v)$ kan eindig zijn terwijl $d(v,u)$ oneindig is, zoals rij 1 van de matrix hierboven laat zien. Een gerichte graaf geeft een **quasimetriek**, en elke meetkundige intuïtie die je uit metrische ruimten meeneemt, moet daaraan getoetst worden.

Uit de alle-parenmatrix volgen drie standaardgrootheden:

- de **excentriciteit** van v is $\max_u d(v,u)$;
- de **diameter** is de grootste excentriciteit;
- de **straal** is de kleinste excentriciteit, en een knoop die haar bereikt is een **centrum**.

$\text{straal} \leq \text{diameter} \leq 2 \cdot \text{straal}$. De rechterongelijkheid is de driehoeksongelijkheid toegepast via een centrum c : $d(u,v) \leq d(u,c) + d(c,v) \leq 2 \cdot \text{straal}$. Beide grenzen worden bereikt — door respectievelijk een cykel en een pad — dus geen van beide kan verbeterd worden.

Welk algoritme

	tijd	negatieve bogen	het best wanneer
BFS vanuit elke knoop	$O(nm)$	nee (ongewogen)	ongewogen
Dijkstra vanuit elke knoop	$O(nm \log n)$	nee	ijl, $m \ll n^2$
Floyd–Warshall	$O(n^3)$	ja	dicht, of negatieve bogen
Johnson	$O(nm \log n)$	ja	ijl én negatieve bogen

Het omslagpunt ligt bij $m \approx n^2 / \log n$. Eronder wint herhaald Dijkstra; erboven Floyd–Warshall, dat bovendien op constanten ruim wint omdat het drie lussen over een vlakke array zijn zonder allocatie.

Het algoritme van Johnson is het enige gat dat dit boek in kortste paden openlaat. Het herweegt de graaf met een Bellman–Ford-potentiaal zodat alle gewichten niet-negatief worden en kortste paden behouden blijven, en draait dan n keer Dijkstra. Het is het juiste antwoord voor ijle grafen met negatieve bogen, en het is een werkelijk vernuftige truc in plaats van een routineuze combinatie — de potentiaalfunctie is $h(v) = d(s, v)$ vanuit een nieuwe knoop die met alles verbonden is op gewicht 0, en de herwogen boog $w'(u, v) = w(u, v) + h(u) - h(v)$ is niet-negatief precies omdat h aan de driehoeksongelijkheid voldoet.

Probeer het

Breek de lusvolgorde met opzet en kijk hoe vaak het zichtbaar wordt:


```
python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.digraph import Digraph, INF, random_digraph
from graphs.paths import floyd_warshall, brute_force_shortest

def wrong(d):
    n = d.n
    dist = [[INF]*n for _ in range(n)]
    for v in range(n): dist[v][v] = 0.0
    for u, v, w in d.arcs(): dist[u][v] = min(dist[u][v], w)
    for i in range(n):          # i outermost -- the classic mistake
        for j in range(n):
            for k in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

# a plain path: the broken version gets this right
d = Digraph(4, [(0,1,1), (1,2,1), (2,3,1)])
print('path graph, 0->3:  correct', floyd_warshall(d)[0][3], ' broken', wrong(d)[0][3])

# and one where it does not
d = Digraph(4, [(0,3,2), (2,1,1), (3,0,1), (3,2,4)])
print('witness,    0->1:  correct', floyd_warshall(d)[0][1], ' broken', wrong(d)[0][1],
      ' true', brute_force_shortest(d, 0, 1))

rng = random.Random(1); bad = 0
for _ in range(4000):
    g = random_digraph(rng.randint(3, 6), 0.4, rng)
    if any(floyd_warshall(g)[i][j] != wrong(g)[i][j] for i in g.vertices() for j in g.vertices()):
        bad += 1
print(f'broken version differs on {bad}/4000 random digraphs')
"
```

```
path graph, 0->3:  correct 3  broken 3
witness,    0->1:  correct 7  broken inf
broken version differs on 942/4000 random digraphs
```

Dit is de gevaarlijke soort fout, en de getallen zeggen waarom. Op een padgraaf heeft de kapotte versie gelijk. Op een willekeurig gekozen digraaf heeft ze ongeveer **een kwart van de tijd** ongelijk. Een implementatie die op elke invoer faalde, zou door de eerste test die iemand schreef gevangen worden; een die op de eenvoudige gevallen klopt en op 24% van de rest faalt, doorstaat een met de hand gecontroleerd voorbeeld en gaat de deur uit.

Het getuigenis toont het mechanisme. De echte route $0 \rightarrow 3 \rightarrow 2 \rightarrow 1$ kost $2 + 4 + 1 = 7$. Met i als buitenste lus wordt de plaats `dist[0][1]` definitief gemaakt terwijl `dist[3][1]` nog oneindig is, omdat rij 3 nog niet verwerkt is — en niets bezoekt rij 0 ooit opnieuw. De juiste volgorde maakt *alle* paden door knoop k definitief voordat ze naar $k+1$ gaat, dus geen rij kan achterblijven.

Oefeningen

1. Wat is de looptijd van Floyd–Warshall, en welke datastructuur heeft het nodig?

2. Waarom moet de k -lus de buitenste zijn?
3. Wat betekent een negatieve plaats op de diagonaal van de uitvoer?
4. Geef een digraaf op twee knopen waarin $d(u, v)$ eindig is en $d(v, u)$ oneindig.

Oplossingen in Bijlage E.

Kernpunten

- Floyd–Warshall induceert op *welke knopen een pad mag doorkruisen*, niet op afstand tot een bron. Daarom doet het alle paren tegelijk.
- k moet de buitenste lus zijn. Zij indexeert de inductie; i en j niet.
- $O(n^3)$, negatieve bogen toegestaan, geen datastructuren. Op dichte grafen verslaat het n keer Dijkstra op zowel asymptotiek als constanten.
- Een negatieve plaats op de diagonaal betekent dat die knoop op een negatieve cykel ligt — een andere vraag dan Bellman–Fords "is er een bereikbaar vanuit s ", en beide zijn het stellen waard.
- Ongerichte afstand is een metriek; gerichte afstand slechts een quasimetriek, en asymmetrie is geen randgeval.

Hoofdstuk 12 — Samenhang en de stelling van Menger

Hoe moeilijk is het om een graaf uiteen te laten vallen? Er zijn twee antwoorden — tel de knopen die je moet verwijderen, of tel de disjuncte paden die je moet vernietigen — en de stelling van Menger zegt dat het hetzelfde getal is. Het is de eerste **min-max-stelling** in dit boek, en het patroon dat ze vestigt loopt door de volgende twee hoofdstukken en door het grootste deel van de combinatorische optimalisatie.

Twee maten

Een verzameling $S \subseteq V \setminus \{s, t\}$ is een **s - t -knopensnede** wanneer haar verwijdering geen s - t -pad overlaat. Een verzameling kanten is een **s - t -kantensnede** wanneer haar verwijdering hetzelfde doet.

Twee paden zijn **inwendig disjunct** wanneer ze geen knoop delen behalve s en t , en **kantdisjunct** wanneer ze geen kant delen.

Eén richting is gratis, en het loont te zien waarom ze gratis is:

Lemma (zwakke dualiteit). Het aantal paarsgewijs inwendig disjuncte s - t -paden is hoogstens de grootte van elke s - t -knopensnede.

Bewijs. Elk s - t -pad moet een knoop van de snede bevatten, anders zou het de verwijdering overleven. Inwendig disjuncte paden kunnen zo'n knoop niet delen. Dus de snede heeft minstens één knoop per pad. ■

Dat argument geeft $\max \leq \min$ gratis en vertelt je niets over de vraag of de grens scherp is. **De inhoud van elke min-max-stelling is de andere richting** — en er is geen algemene reden ze te verwachten. Er zijn genoeg natuurlijke paren grootheden die aan zwakke dualiteit voldoen met een echte kloof; $\omega(G) \leq \chi(G)$ uit hoofdstuk 15 is er een, en hoofdstuk 19 gaat over de grafen waar ze sluit.

De stelling

Stelling (Menger, 1927). Voor niet-aangrenzende s en t is het maximale aantal paarsgewijs inwendig disjuncte s - t -paden gelijk aan de minimale grootte van een s - t -knopensnede.

De kantversie geldt voor alle $s \neq t$: het maximale aantal kantdisjuncte s - t -paden is gelijk aan de minimale grootte van een s - t -kantensnede.

De voorwaarde **niet-aangrenzend** in de knopenversie is geen muggenzifterij. Zijn s en t aangrenzend, dan scheidt geen verzameling *andere* knopen ze, dus de minimale snede bestaat niet — de grootheid rechts is ongedefinieerd in plaats van groot. De `vertex_connectivity` van dit boek geeft in dat geval oneindig terug, wat een uitspraak over de definitie is en niet een rekenkundig gemak.

Bewijs (kantversie), per inductie naar het aantal kanten. Zij k de minimale kantensnedegrootte. Zwakke dualiteit geeft hoogstens k disjuncte paden, dus we hebben er k nodig.

Kies een minimale snede F met $|F| = k$ en neem een kant $e \in F$. In $G - e$ heeft de minimale s - t -snede grootte $k - 1$: hoogstens dat, want $F - e$ werkt, en minstens dat, want e aan een kleinere snede toevoegen zou F verslaan. Per inductie heeft $G - e$ $k - 1$ kantdisjuncte paden, en die vermijden e . e weer toevoegen

vraagt om nog één pad dat ze alle vermijdt — dat bestaat precies omdat e in elke minimale snede zat. Het volledige argument vergt zorg voor het geval waarin samentrekken in plaats van verwijderen nodig is, en de nette moderne route is die hieronder. ■

De nette route: het is een stroomprobleem

In plaats van die inductie af te maken, merk op dat Menger **max-stroom min-snede is met elke capaciteit op 1**, en hoofdstuk 13 bewijst dat. Dit is geen ontwijking; het is de juiste indeling, want één algoritme levert dan drie stellingen.

Voor de **kantversie** vervang je elke kant door twee bogen met eenheidscapaciteit:

```
def edge_connectivity(g, s, t):
    net = FlowNetwork(g.n)
    for u, v in g.edges():
        net.add_arc(u, v, 1.0)
        net.add_arc(v, u, 1.0)
    return net.max_flow(s, t)[0]
```

Een stroom van waarde k valt uiteen in k eenheidspaden, en eenheidscapaciteit dwingt ze kantdisjunct te zijn. Een snede van capaciteit k is een verzameling van k kanten.

Voor de **knopenversie** is de truc **knoopsplitsing**: vervang elke knoop v door $v_{in} \rightarrow v_{out}$ met capaciteit 1, en leid elke kant naar v_{in} en uit v_{out} .

```
def vertex_connectivity(g, s, t):
    net = FlowNetwork(2 * g.n)
    for v in g.vertices():
        net.add_arc(2*v, 2*v + 1, INF if v in (s, t) else 1.0)
    for u, v in g.edges():
        net.add_arc(2*u + 1, 2*v, INF)
        net.add_arc(2*v + 1, 2*u, INF)
    return net.max_flow(2*s + 1, 2*t)[0]
```

De inwendige boog begrenst hoeveel stroom *door* v gaat op 1, dus paden kunnen geen knoop delen. De kanten krijgen oneindige capaciteit omdat de stelling knopen telt en geen kanten, en s en t krijgen oneindige capaciteit omdat ze niet verwijderd mogen worden.

Knoopsplitsing is een algemene techniek en geen eenmalige truc: telkens wanneer een beperking op knopen ligt in plaats van op kanten, splits je de knoop en zet je de beperking op de inwendige boog.

Beide worden gecontroleerd tegen uitpuittend verwijderen — een volstrekt andere berekening:

```
held      ch12 Menger (edge form): max edge-disjoint s-t paths = min s-t edge
cut (49 graphs)
held      ch12 Menger (vertex form): max internally-disjoint paths = min s-t
vertex cut (51 graphs)
```

Globale samenhang

De **knopensamenhang** $\kappa(G)$ is het kleinste aantal knopen waarvan verwijdering G onsaamenhangend maakt of tot één knoop reduceert; de **kantensamenhang** $\lambda(G)$ is de kantentegenhanger. Een graaf is **k -samenhangend** wanneer $\kappa(G) \geq k$.

Stelling (Whitney). $\kappa(G) \leq \lambda(G) \leq \delta(G)$.

Bewijs. De rechterongelijkheid: alle kanten bij een knoop van minimale graad verwijderen isoleert hem. De linker: neem een minimale kantensnede en kies van elke snedekant één eindpunt aan de bronzijde. Die knopen verwijderen vernietigt elk s - t -pad, dus $\kappa \leq \lambda$. ■

Beide ongelijkheden kunnen strikt zijn, en de Petersen-graaf is deze keer niet het getuigenis — hij heeft $\kappa = \lambda = \delta = 3$. Neem in plaats daarvan twee driehoeken verbonden door één lang pad: $\delta = 2$, $\lambda = 1$, $\kappa = 1$.

Een bruikbare herformulering, die Menger op elk paar tegelijk toepast:

Gevolg. G is k -samenhangend dan en slechts dan als elk paar knopen verbonden is door k inwendig disjuncte paden.

Dat is de vorm om te onthouden, want ze zet een uitspraak over het *vernietigen* van de graaf om in een uitspraak over *routes erdoorheen* — en daarom is k -samenhang het juiste robuustheidsbegrip voor een netwerk.

Probeer het

Bevestig beide vormen op de Petersen-graaf, waarvan de samenhang in elke zin 3 is:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import petersen, cycle
from graphs.flow import edge_connectivity, vertex_connectivity,
brute_force_edge_cut, brute_force_vertex_cut

p = petersen()
print('petersen, vertices 0 and 2 (non-adjacent):')
print('  edge-disjoint paths ', edge_connectivity(p, 0, 2), ' exhaustive edge
cut ', brute_force_edge_cut(p, 0, 2))
print('  internally-disjoint ', vertex_connectivity(p, 0, 2), ' exhaustive vertex
cut', brute_force_vertex_cut(p, 0, 2))
c = cycle(6)
print('C_6, opposite vertices:')
print('  edge-disjoint paths ', edge_connectivity(c, 0, 3), ' exhaustive edge
cut ', brute_force_edge_cut(c, 0, 3))
"
```

```
petersen, vertices 0 and 2 (non-adjacent):
  edge-disjoint paths    3.0  exhaustive edge cut    3.0
  internally-disjoint    3.0  exhaustive vertex cut 3.0
C_6, opposite vertices:
  edge-disjoint paths    2.0  exhaustive edge cut    2.0
```

Een cykel geeft 2 in beide richtingen, wat het kleinste interessante geval is: twee routes rond, en je moet twee kanten doorknippen om ze te stoppen.

Oefeningen

1. Formuleer zwakke dualiteit voor knopensneden, en leg uit waarom ze op zichzelf niets bewijst.
2. Waarom vereist de knopenversie van de stelling van Menger dat s en t niet aangrenzend zijn?
3. Geef een graaf waarin $\kappa(G) < \lambda(G) < \delta(G)$ — of leg uit waarom een van die ongelijkheden niet strikt kan zijn.
4. Waarom krijgt in de knoopsplitsingsconstructie de inwendige boog capaciteit 1 en de kantbogen oneindig?

Oplossingen in Bijlage E.

Kernpunten

- Menger is een min-max-stelling: disjuncte paden tegenover snedegrootte, in zowel een knopen- als een kantversie.
- Zwakke dualiteit ($\max \leq \min$) is gratis en bewijst niets. De stelling is de andere ongelijkheid, en er is geen algemene reden waarom zo'n grens scherp zou zijn.
- De knopenversie vereist dat s en t niet aangrenzend zijn, want anders bestaat er helemaal geen snede.
- Beide vormen zijn max-stroom min-snede met eenheidscapaciteiten. Knoopsplitsing — $v_{in} \rightarrow v_{out}$ met capaciteit 1 — is de algemene manier om een beperking van kanten naar knopen te verplaatsen.
- $\kappa(G) \leq \lambda(G) \leq \delta(G)$, en G is k -samenhangend precies wanneer elk paar k inwendig disjuncte paden heeft.

Hoofdstuk 13 — Max-stroom min-snede

Dit is de stelling waar het vorige hoofdstuk naar doorverwees, en die waar het volgende zich toe herleidt. Het is het centrale algoritmische resultaat in de grafentheorie, en het bewijs is korter dan zijn reputatie doet vermoeden.

De opzet

Een **stroomnetwerk** is een digraaf met een niet-negatieve **capaciteit** $c(u, v)$ op elke boog, een **bron** s , en een **put** t . Een **stroom** f kent elke boog een waarde toe met

- **capaciteit:** $0 \leq f(u, v) \leq c(u, v)$;
- **behoud:** voor elke v behalve s en t is de instroom gelijk aan de uitstroom.

De **waarde** van een stroom is de netto hoeveelheid die s verlaat. Een **snede** is een verdeling (S, T) met $s \in S$ en $t \in T$; haar **capaciteit** is de totale capaciteit van bogen van S naar T . Bogen die van T naar S teruglopen dragen niets bij — een punt dat makkelijk verkeerd gaat en waarvan het bewijs hieronder afhangt.

Opnieuw zwakke dualiteit, en opnieuw gratis:

Lemma. De waarde van elke stroom is hoogstens de capaciteit van elke snede.

Bewijs. Alle stroom die s verlaat moet uiteindelijk van S naar T oversteken, en de overstekende bogen dragen hoogstens hun capaciteit. ■

Het restnetwerk, en waarom omkeringen ertoe doen

Het algoritme luidt: zoek een pad van s naar t met vrije capaciteit, duw er zoveel doorheen als het aankan, herhaal. Naïef uitgevoerd loopt dit vast — een vroege gulzige keuze kan een betere latere blokkeren.

De oplossing is het **restnetwerk**. Houd naast elke boog (u, v) die f draagt een omgekeerde boog (v, u) bij met restcapaciteit f . Stroom langs de omgekeerde boog duwen *annuleert* stroom op de voorwaartse.

```
def add_arc(self, u, v, c):
    self.cap[(u, v)] = self.cap.get((u, v), 0.0) + c
    self.cap.setdefault((v, u), 0.0) # the reverse arc, initially empty
```

Die ene regel is wat de hele methode laat werken. Zonder omgekeerde bogen kan het algoritme geen beslissing herzien, en gulzig paden duwen is dan werkelijk fout. Mét ze leidt elk vermeerderend pad ofwel nieuwe stroom ofwel *herleidt* het oude stroom, en de zoektocht hoeft nooit terug te keren.

Een **vermeerderend pad** is een willekeurig s – t -pad in het restnetwerk. Het algoritme — de **methode van Ford–Fulkerson** — is: vermeerder tot er geen meer bestaat.

Edmonds–Karp

Ford–Fulkerson zegt niet welk vermeerderend pad je moet nemen, en die keuze doet meer terzake dan het lijkt. Neem ze willekeurig, en op een netwerk met irrationale capaciteiten kan de methode eeuwig doorlopen en naar

een waarde onder het maximum convergeren. Met gehele capaciteiten termineert ze, maar ze kan tijd kosten evenredig met de *waarde* van de stroom, wat exponentieel is in de invoergrootte.

Neem kortste vermeerderende paden — BFS, geen DFS — en het wordt Edmonds–Karp, met looptijd $O(nm^2)$ ongeacht de capaciteiten. De grens volgt uit het feit dat de BFS-afstand van s naar t in het restnetwerk nooit daalt en na elke $O(m)$ vermeerderingen strikt moet stijgen.

```
def _shortest_augmenting_path(self, residual, source, sink):
    parent = {source: source}
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in self.adj[u]:
            if v not in parent and residual.get((u, v), 0.0) > 1e-12:
                parent[v] = u
                if v == sink:
                    return parent
                queue.append(v)
    return None
```

De keuze van houder is het hele verschil tussen een algoritme met een polynomiale grens en een zonder. Het is hetzelfde onderscheid tussen `deque` en stapel als in hoofdstuk 8, en hier doet het meer terzake.

De stelling

Stelling (Ford–Fulkerson, Elias–Feinstein–Shannon, 1956). De maximale stroomwaarde is gelijk aan de minimale snedecapaciteit.

Bewijs. Zwakke dualiteit geeft $\max \leq \min$. Voor de omgekeerde richting: laat het algoritme volledig aflopen en zij S de verzameling knopen die vanuit s bereikbaar zijn in het uiteindelijke restnetwerk. Dan geldt $t \notin S$, want anders zou er nog een vermeerderend pad zijn. Dus (S, T) is een snede.

Elke boog van S naar T is **verzadigd** — had ze vrije restcapaciteit, dan zou haar kop in S liggen. Elke boog van T naar S draagt **nul** stroom — droeg ze iets, dan zou haar omgekeerde restboog haar staart in S plaatsen.

Dus de netto stroom over de snede is gelijk aan de totale capaciteit van de snede, en de waarde van de stroom is daaraan gelijk. Deze stroomwaarde is dus gelijk aan deze snedecapaciteit, en per zwakke dualiteit zijn beide optimaal. ■

Het bewijs is constructief: het beweert niet slechts dat er een passende snede bestaat, het overhandigt er een. Dat is wat `min_cut` teruggeeft, en de verificatie controleert de constructie apart van het getal, want het zijn verschillende beweringen:

held	ch13	Max-flow equals min-cut (80 graphs)
held	ch13	The cut the algorithm reports really has the flow's value (80 graphs)
held	ch13	Integer capacities give an integer maximum flow (80 graphs)

De tweede regel is niet overbodig. Een correcte maximale waarde met een verkeerd uitgelezen snede is een plausibele fout, en alleen het getal controleren zou haar missen.

Geheeltalligheid

Gevolg. Zijn alle capaciteiten geheel, dan is een maximale stroom geheel.

Bewijs. Elke vermeerdering duwt de flessenhals van het pad, die geheel is wanneer alle restcapaciteiten dat zijn. Per inductie blijven ze de hele tijd geheel. ■

Dit gevolg is de reden dat stroom überhaupt combinatorische problemen oplost. Menger in hoofdstuk 12 en koppeling in hoofdstuk 14 hebben allebei nodig dat het antwoord een *verzameling paden* of een *verzameling kanten* is en geen fractionele toewijzing — en geheeltalligheid is wat belooft dat het optimum als zodanig afgelezen kan worden. Een lineair programma met dezelfde beperkingen zou je 0,5 op drie bogen geven en geen manier om af te ronden.

```
net = FlowNetwork(6, [(0,1,16), (0,2,13), (1,2,10), (2,1,4), (1,3,12),
                      (3,2,9), (2,4,14), (4,3,7), (3,5,20), (4,5,4)])
print(net.max_flow(0, 5)[0])          # 23.0
print(sorted(net.min_cut(0, 5)[1]))    # [0, 1, 2, 4]
```

De snede $\{0,1,2,4\}$ tegenover $\{3,5\}$ heeft capaciteit $12 + 7 + 4 = 23$. Merk op dat de boog $(3,2)$ met capaciteit 9 *achterwaarts* over deze snede loopt en niets bijdraagt — het punt dat aan het begin van dit hoofdstuk werd aangestipt.

Waarom het zich herleidt

Stroom is het werkpaard van de combinatorische optimalisatie omdat zoveel problemen het in vermomming zijn:

Probleem	Codering
Menger, kantvorm	eenheidscapaciteiten (h. 12)
Menger, knooppvorm	eenheidscapaciteiten + knoopsplitsing (h. 12)
Bipartiete koppeling	bron → links → rechts → put, alles eenheid (h. 14)
Knoopdisjuncte routing	knoopsplitsing
Projectselectie, beeldsegmentatie	min-snede rechtstreeks
Honkbaleliminatie	max-stroom-haalbaarheid

De vaardigheid die de moeite waard is, is niet Edmonds–Karp implementeren — je gebruikt een bibliotheek — maar herkennen dat een probleem *een* stroomprobleem is, en dat is meestal een kwestie van vragen wat de capaciteiten zouden moeten zijn.

Probeer het

Kijk hoe een omgekeerde boog zijn werk doet. Eerst een netwerk waarin het gulzige eerste pad een vergissing is:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.flow import FlowNetwork
# 0->1->3 and 0->2->3, plus a crossing arc 1->2
net = FlowNetwork(4, [(0,1,1), (0,2,1), (1,2,1), (1,3,1), (2,3,1)])
v, residual = net.max_flow(0, 3)
print('max flow:', v, ' min cut:', net.brute_force_min_cut(0, 3))
print('flow on the crossing arc 1->2:', net.cap[(1,2)] - residual[(1,2)])
"
```

```
max flow: 2.0  min cut: 2.0
flow on the crossing arc 1->2: 0.0
```

Het maximum is 2 — één eenheid langs elke zijde — en de kruisende boog draagt uiteindelijk niets. Edmonds–Karp vindt dit meteen omdat BFS eerst de paden van twee bogen neemt. Ford–Fulkerson dat eerst het pad $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ van drie bogen zou nemen, zou het via de omgekeerde boog moeten *terugdraaien* — precies de situatie waarvoor omgekeerde bogen bestaan.

Oefeningen

1. Waarom heeft het restnetwerk omgekeerde bogen nodig? Wat gaat er zonder stuk?
2. Ford–Fulkerson bepaalt niet welk vermeerderend pad je neemt. Wat gaat er mis wanneer je slecht kiest?
3. Formuleer het geheeltalligheidsgevolg en leg uit waarom het stroom bruikbaar maakt voor combinatorische problemen.
4. Een boog loopt van de putzijde terug naar de bronzijde van een snede. Hoeveel draagt ze bij aan de capaciteit van de snede?

Oplossingen in Bijlage E.

Kernpunten

- Max-stroom is gelijk aan min-snede, en het bewijs is constructief: de in het restnetwerk bereikbare verzameling van het verzadigde netwerk is een minimale snede.
- Omgekeerde restbogen zijn het mechanisme. Zonder ze kan gulzige vermeerdering een beslissing niet herzien en is ze eenvoudigweg fout.
- Ford–Fulkerson bepaalt niet welk vermeerderend pad. Kortste nemen (BFS) geeft Edmonds–Karp en $O(n^2)$; willekeurige nemen kan op irrationale capaciteiten niet termineren.
- Gehele capaciteiten geven een geheel optimum. Dat gevolg is waarom stroom combinatorische en niet slechts numerieke problemen oplost.
- Bogen die vanaf de putzijde terugkruisen dragen nul bij aan de capaciteit van een snede.

Hoofdstuk 14 — Koppeling

Een **koppeling** is een verzameling kanten waarvan er geen twee een knoop delen. Een grootste vinden is een van de weinige werkelijk moeilijk ogende combinatorische problemen die toch in polynomiale tijd oplosbaar blijken — en op bipartiete grafen is het hoofdstuk 13 met een hoed op.

Vermeerderende paden

Zij M een koppeling. Een knoop is **bedekt** wanneer een kant van M hem raakt, en **vrij** anders. Een **alternerend pad** wisselt af tussen kanten buiten M en kanten binnen M . Een **vermeerderend pad** is een alternerend pad waarvan beide eindpunten vrij zijn.

Keer een vermeerderend pad om — neem zijn niet-koppelingskanten op in M en verwijder zijn koppelingskanten — en je krijgt een koppeling met precies één kant meer, want het pad heeft één niet-koppelingskant meer dan koppelingskanten.

Stelling (Berge, 1957). M is maximaal dan en slechts dan als er geen M -vermeerderend pad bestaat.

Bewijs. Bestaat er een vermeerderend pad, dan geeft omkeren een grotere koppeling, dus M is niet maximaal.

Omgekeerd, stel dat M niet maximaal is en zij N groter. Beschouw het symmetrisch verschil $M \Delta N$ en bekijk de graaf die het vormt. Elke knoop heeft er graad hoogstens 2 — hoogstens één kant uit elke koppeling — dus elke component is een pad of een even cykel, met kanten die afwisselen tussen M en N . Cykels gebruiken er evenveel van elk. Omdat $|N| > |M|$ moet een component meer N -kanten dan M -kanten hebben, en zo'n component kan alleen een pad zijn dat met N -kanten begint en eindigt. Zijn eindpunten zijn vrij in M , dus het is een M -vermeerderend pad. ■

Het symmetrisch-verschilargument is de techniek om te bewaren. Je object vergelijken met een hypothetisch beter object en kijken waar ze verschillen, is hoe vrijwel elk koppelingsresultaat bewezen wordt, en de waarneming dat $M \Delta N$ maximale graad 2 heeft is wat de componenten hanteerbaar maakt.

De stelling van Berge verandert het vinden van een maximale koppeling in: zoek een vermeerderend pad, keer het om, herhaal. Hoogstens $n/2$ iteraties.

Het bipartiete geval, en wat erin verborgen zit

Op een bipartiete graaf is het zoeken naar een vermeerderend pad een rechttoe rechtaan alternerende zoektocht vanaf elke vrije linkerknoop:

```
def try_augment(v, visited):
    for w in sorted(g.neighbours(v)):
        if w in visited:
            continue
        visited.add(w)
        if w not in match or try_augment(match[w], visited):
            match[w] = v
            match[v] = w
            return True
    return False
```

$O(nm)$ in totaal. Hopcroft–Karp verbetert dit tot $O(m\sqrt{n})$ door per fase een maximale *verzameling* kortste vermeerderende paden te vinden.

Die zoektocht is correct op bipartiete grafen en **fout in het algemeen**, en de reden is het bekijken waard. In een graaf met een oneven cykel kan een alternerende wandeling terugkeren naar een knoop met de tegenovergestelde pariteit, en de zoektocht kan niet zien of ze een vermeerderend pad heeft gevonden of rondgelopen is. Edmonds' bloesemalgoritme uit 1965 lost dit op door oneven cyclen — "bloesems" — samen te trekken, en het is het artikel dat het idee introduceerde van polynomiale tijd als de definitie van hanteerbaar.

De verificatie registreert de beperking als een stelling waarvan verwacht wordt dat ze weerlegd wordt:

```
refuted    ch14 Augmenting paths find a maximum matching in any graph (1 graphs)
```

Het getuigenis is C_7 met de linkerzijde opgegeven als $\{0,1,2,6\}$: de routine geeft een koppeling van grootte 2 terug waar er 3 bestaat. Het is de moeite waard op te merken hoe *smal* dat getuigenis is. Op C_3 en C_5 geeft dezelfde routine voor **elke** keuze van linkerzijde het juiste antwoord, dus een test op kleine oneven cyclen zou niets vinden en concluderen dat de code in orde is.

De stelling van Hall

Stelling (Hall, 1935). Een bipartiete graaf met delen L en R heeft een koppeling die elke knoop van L bedekt dan en slechts dan als $|N(S)| \geq |S|$ voor elke $S \subseteq L$.

De voorwaarde is duidelijk noodzakelijk: S heeft $|S|$ verschillende partners nodig, allemaal binnen $N(S)$.

Bewijs van de voldoendeheid. Stel dat geen koppeling L bedekt, en neem een maximale koppeling M met een vrije $u \in L$. Zij Z de verzameling knopen die vanuit u via alternerende paden bereikbaar zijn. Geen knoop van $Z \cap R$ is vrij — dat zou een vermeerderend pad geven, in tegenspraak met maximaliteit — dus elke knoop van $Z \cap R$ is gekoppeld, en wel aan $Z \cap L$. Met $S = Z \cap L$ krijgen we $N(S) = Z \cap R$, en $|Z \cap L| = |Z \cap R| + 1$ vanwege u . Dus $|N(S)| = |S| - 1 < |S|$, in tegenspraak met de voorwaarde. ■

De voorwaarde van Hall controleren vergt naar elke deelverzameling kijken, en de verificatie doet precies dat — $2^{|L|}$ deelverzamelingen. Iets goedkopere zou de stelling aannemen die getest wordt:

```
held      ch14 Hall: every left vertex can be matched iff |N(S)| >= |S| for all
S (120 graphs)
```

De stelling van Hall is een vermomde min-max-stelling, en de vermomming is dun: ze zegt dat de belemmering voor een perfecte koppeling altijd één enkele "te drukke" verzameling is.

De stelling van König

Stelling (König, 1931). In een bipartiete graaf is de grootte van een maximale koppeling gelijk aan de grootte van een minimale knopenoverdekking.

Zwakke dualiteit is gratis — elke kant van een koppeling heeft zijn eigen overdekkingsknoop nodig — en opnieuw is de inhoud de andere richting. Het bewijs is een *constructie*:

```
def konig_cover(g, left, right):
    match = bipartite_matching(g, left)
    unmatched_left = [v for v in left if v not in match]
    # Z: everything reachable from an exposed left vertex by alternating paths
    ...
    return (set(left) - reachable) | (set(right) & reachable)
```

Bewijs dat dit een overdekking van de juiste grootte is. Neem een kant uv met $u \in L$, $v \in R$. Is $u \notin Z$, dan zit u in de overdekking. Is $u \in Z$, dan zit v er ook in — ofwel is uv een niet-koppelingskant, zodat het alternerende pad zich erdoorheen uitbreidt, ofwel is het een koppelingskant en werd u erlangs bereikt. Hoe dan ook is v overdekt. Het is dus een overdekking.

Voor de grootte: elke knoop van $L \setminus Z$ is gekoppeld (een vrije linkerknoop zit per definitie in Z), en elke knoop van $R \cap Z$ is gekoppeld (aangetoond in het bewijs van Hall hierboven). Geen koppelingskant heeft beide eindpunten in de overdekking — dat zou $u \in L \setminus Z$ en $v \in R \cap Z$ vergen, maar $v \in Z$ bereikt langs zijn koppelingskant dwingt $u \in Z$. Dus de overdekking heeft precies één knoop per koppelingskant. ■

De verificatie controleert het getal en de constructie apart, want een juist antwoord uit een verkeerde constructie is een echte faalwijze:

```
held      ch14  Konig: in a bipartite graph, max matching = min vertex cover
(120 graphs)
held      ch14  Konig's construction returns a cover of exactly the matching's
size (120 graphs)
```

König faalt op niet-bipartiete grafen, en het kleinste getuigenis is de driehoek: maximale koppeling 1, minimale knopenoverdekking 2. Hoofdstuk 19 identificeert precies voor welke grafen de gelijkheid geldt, en dat is een veel grotere klasse dan de bipartiete.

De min-max-familie

Drie hoofdstukken, één vorm:

Stelling	max	min
Menger (h. 12)	disjuncte $s-t$ -paden	$s-t$ -snede
Max-stroom min-snede (h. 13)	stroomwaarde	snedecapaciteit
König (h. 14)	koppeling	knopenoverdekking
Hall (h. 14)	koppeling die L verzadigt	schendende verzameling (belemmering)

Alle vier zijn dezelfde stelling vanuit verschillende hoeken bekeken, en alle vier zijn gevallen van dualiteit in de lineaire programmering waarbij het polytoop toevallig gehele hoekpunten heeft. Die laatste zin is de hele reden dat bipartiete koppeling eenvoudig is en algemene koppeling Edmonds vergde: de bipartiete beperkingsmatrix is **totaal unimodulair**, en de algemene niet.

Probeer het

Kijk hoe de constructie van König een overdekking oplevert van dezelfde grootte als de koppeling, op een graaf waar geen van beide voor de hand ligt:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.matching import bipartite_matching, bipartition, matching_size,
konig_cover, min_vertex_cover_bruteforce

# left {0,1,2}, right {3,4,5}
g = Graph(6, [(0,5), (1,3), (1,4), (1,5), (2,5)])
left, right = bipartition(g)
m = bipartite_matching(g, left)
cover = konig_cover(g, left, right)
print('matching size      ', matching_size(m))
print('konig cover       ', sorted(cover), 'size', len(cover))
print('true minimum cover ', min_vertex_cover_bruteforce(g))
print('is a cover         ', all(u in cover or v in cover for u, v in g.edges()))
"
```

```
matching size      2
konig cover       [1, 5] size 2
true minimum cover 2
is a cover        True
```

Vijf kanten overdekt door twee knopen, passend bij een maximale koppeling van grootte 2. De overdekking neemt **één knoop van elke zijde** — 1 van links en 5 van rechts — en welke zijde elk vandaan komt, is precies wat de alternerende-bereikbaarheidsverzameling Z bepaalt. Het is niet "één eindpunt van elke koppelingskant, willekeurig gekozen": hier verkeerd kiezen geeft een verzameling die helemaal geen overdekking is.

Dit voorbeeld kiezen vergde zoeken. De voor de hand liggende kleine bipartiete grafen geven een overdekking die simpelweg de hele linkerzijde is, wat triviaal een overdekking is en niets aantoon.

Oefeningen

1. Definieer een vermeerderend pad, en zeg wat omkeren met de koppelingsgrootte doet.
2. Formuleer de stelling van Berge en noem de techniek die haar bewijs gebruikt.
3. De eenvoudige vermeerderingszoektocht is fout op niet-bipartiete grafen. Welke structuur verslaat haar?
4. De stelling van König zegt dat maximale koppeling gelijk is aan minimale knopenoverdekking. Bereken beide voor een driehoek en verklaar het resultaat.

Oplossingen in Bijlage E.

Kernpunten

- Berge: een koppeling is maximaal dan en slechts dan als er geen vermeerderend pad bestaat. Bewezen met symmetrisch verschil, de techniek om mee te nemen.

- De eenvoudige vermeerderingszoektocht is alleen correct op bipartiete grafen. Oneven cykels verslaan haar, en Edmonds' bloesemalgoritme is de oplossing. Het getuigenis hier is C_7 ; C_3 en C_5 leggen de fout helemaal niet bloot.
- Hall: L kan verzadigd worden dan en slechts dan als geen deelverzameling van L te weinig burens heeft. Haar eerlijk controleren betekent alle $2^{|L|}$ deelverzamelingen bekijken.
- König: bipartiete maximale koppeling is gelijk aan minimale knopenoverdekking, en het bewijs bouwt de overdekking uit alternerende bereikbaarheid. Het faalt op de driehoek.
- Menger, max-stroom min-snedes, König en Hall zijn één stelling in vier vermommingen — LP-dualiteit met een geheelgetalig polytoop.

Hoofdstuk 15 — Kleuring

Een **geldige kleuring** geeft elke knoop een kleur zó dat aangrenzende knopen verschillen. Het **chromatisch getal** $\chi(G)$ is het kleinste aantal kleuren dat volstaat. Het berekenen is NP-moeilijk (hoofdstuk 22), dus het grootste deel van het vak bestaat uit grenzen — en in de grenzen zitten de ideeën.

De voor de hand liggende grenzen

Lemma. $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$.

De ondergrens is onmiddellijk: een klik van grootte ω heeft ω verschillende kleuren nodig. De bovengrens komt van het gulzige algoritme hieronder.

Beide kunnen ver van de waarheid liggen. De kloof aan de linkerkant is het onderwerp van hoofdstuk 19. Die aan de rechterkant is enorm voor een ster: $K_{1,7}$ heeft $\Delta = 7$ en $\chi = 2$.

Gulzig, en de volgorde waarvan het afhangt

Neem de knopen in een of andere volgorde; geef elke knoop de kleinste kleur die geen al gekleurde buur gebruikt.

```
def greedy_colouring(g, order=None):
    colour = {}
    for v in order if order is not None else g.vertices():
        used = {colour[w] for w in g.neighbours(v) if w in colour}
        c = 0
        while c in used:
            c += 1
        colour[v] = c
    return colour
```

Het gebruikt nooit meer dan $\Delta + 1$ kleuren, want een knoop heeft hoogstens Δ gekleurde burens en $\Delta + 1$ kleuren laten er altijd één vrij. Dat bewijst de bovengrens hierboven.

Het resultaat hangt volledig van de volgorde af, en die afhankelijkheid is niet mild. Het standaardgetuigenis is de **kroongraaf**: $K_{n,n}$ met een perfecte koppeling verwijderd. Hij is bipartiet, dus $\chi = 2$. Orden de knopen door tussen de zijden af te wisselen en gulzig gebruikt n kleuren:

```
crown n=3: chi=2 greedy(natural)=2 greedy(interleaved)=3 degeneracy=2
crown n=4: chi=2 greedy(natural)=2 greedy(interleaved)=4 degeneracy=3
crown n=5: chi=2 greedy(natural)=2 greedy(interleaved)=5 degeneracy=4
```

Gulzig dat op een tweekleurbare graaf vijf kleuren gebruikt is zo slecht als een benadering plausibel kan worden, en het is geen gekunstelde pathologie — het is wat er gebeurt wanneer je de knopen toevallig in de verkeerde volgorde verwerkt.

Merk op dat de natuurlijke volgorde het elke keer goed doet. Test je gulzig op kroongrafen zonder over de ordening na te denken, dan concludeer je dat het goed werkt.

Degeneratie: de grens die de moeite waard is

$\Delta + 1$ is zwak omdat ze door de ene slechtste knoop bepaald wordt. De oplossing is vragen wat er gebeurt terwijl je de graaf afpelt.

De **degeneratie** $d(G)$ is de grootste k zodat elke deelgraaf een knoop van graad hoogstens k heeft. Equivalent: verwijder herhaaldelijk een knoop van minimale graad, en $d(G)$ is de grootste graad die je ooit verwijdert.

Stelling. $\chi(G) \leq d(G) + 1$, en $d(G) \leq \Delta(G)$.

Bewijs. Verwijder knopen van minimale graad één voor één en noteer de volgorde; kleur dan gulzig in de **omgekeerde** volgorde. Wanneer elke knoop gekleurd wordt, zijn zijn al gekleurde burens precies die welke er nog waren toen hij verwijderd werd — hoogstens $d(G)$ ervan. Dus $d(G) + 1$ kleuren volstaan. ■

Dit is strikt beter dan $\Delta + 1$ en nooit slechter:

```
star K_1,7: Delta = 7  degeneracy = 1  chi = 2
a tree:      Delta = 3  degeneracy = 1  chi = 2
```

Elk bos heeft degeneratie 1, wat $\chi \leq 2$ geeft — het resultaat van hoofdstuk 6, teruggevonden als bijzonder geval. Elke vlakke graaf heeft degeneratie hoogstens 5 (hoofdstuk 17), wat de zeskleurenstelling gratis oplevert.

```
held      ch15 Greedy colouring uses at most degeneracy + 1 colours (52 graphs)
held      ch15 chi <= degeneracy + 1, which is never worse than Delta + 1 (52
graphs)
```

De stelling van Brooks

$\Delta + 1$ is scherp voor precies twee families, en dat is de hele inhoud van het resultaat.

Stelling (Brooks, 1941). Is G samenhangend en noch een volledige graaf noch een oneven cykel, dan $\chi(G) \leq \Delta(G)$.

De twee uitzonderingen zijn geen versiering. K_n heeft $\Delta = n - 1$ en $\chi = n$. Een oneven cykel heeft $\Delta = 2$ en $\chi = 3$. Laat een van beide uitzonderingen weg en de stelling is onwaar bij K_3 , dat beide is.

De verificatie codeert de uitzonderingen als *falende voorwaarden* in plaats van als bijzondere gevallen die slagen, precies het onderscheid waarvoor hoofdstuk 3 pleitte:

```
if canonical(g) == canonical(complete(g.n)):
    return None # says nothing, rather than "held"
if g.n % 2 == 1 and canonical(g) == canonical(cycle(g.n)):
    return None
return chromatic_number(g) <= max_degree(g)
```

Het bewijs is een gevalsonderscheid naar samenhang dat dit boek niet volledig weergeeft; de leesbare kern is dat een niet-volledige samenhangende graaf met $\Delta \geq 3$ een ordening heeft waarin gulzig bij de laatste knoop

een kleur vrijlaat, verkregen door een opspannende boom in een geschikte knoop te wortelen en naar binnen te kleuren.

Exacte kleuring

$\chi(G)$ wordt hier berekend door $k = 1, 2, 3, \dots$ te proberen met backtracking:

```
def chromatic_number(g):
    for k in range(1, g.n + 1):
        if _colourable(g, k):
            return k
    return g.n
```

Exponentieel, en onvermijdelijk zo tenzij $P = NP$. Bruikbaar tot ongeveer twaalf knopen, wat genoeg is voor de verificatie en voor niets anders. Hoofdstuk 23 behandelt wat je in plaats daarvan doet wanneer je werkelijk een antwoord nodig hebt.

Probeer het

Kijk hoe de ordening ertoe doet, op een graaf die tweekleurbaar is:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.algorithms import chromatic_number, greedy_colouring
from graphs.planar import degeneracy_order

# crown graph: K_{4,4} minus a perfect matching
n = 4
g = Graph(2*n, [(i, n+j) for i in range(n) for j in range(n) if i != j])
interleaved = [x for i in range(n) for x in (i, n+i)]
print('chromatic number      ', chromatic_number(g))
print('greedy, natural order ', max(greedy_colouring(g).values()) + 1)
print('greedy, interleaved   ', max(greedy_colouring(g, interleaved).values()) + 1)
print('greedy, degeneracy    ', max(greedy_colouring(g,
degeneracy_order(g)).values()) + 1)
"
```

```
chromatic number      2
greedy, natural order  2
greedy, interleaved   4
greedy, degeneracy    2
```

Vier kleuren voor een bipartiete graaf, enkel door een slechte volgorde. De degeneratieordening herstelt hier het juiste antwoord, al is dat in het algemeen niet gegarandeerd — ze garandeert alleen $d + 1$, en dat is 4 voor deze graaf.

Oefeningen

1. Toon aan dat $\chi(G) \geq n / \alpha(G)$, waarbij $\alpha(G)$ de grootte is van de grootste onafhankelijke verzameling.
2. Vind een graaf met $\omega(G) = 2$ en $\chi(G) = 3$. Wat is de kleinste zo'n graaf?

3. Bewijs dat elke graaf met degeneratie d hoogstens $d \cdot n$ kanten heeft.
4. De kroongraaf hierboven heeft $\chi = 2$ maar gulzig kan er n kleuren gebruiken. Wat is zijn degeneratie, en waarom is dat geen tegenspraak met de grens $d + 1$?

Oplossingen in Bijlage E.

Kernpunten

- $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$. Beide ongelijkheden kunnen willekeurig ruim zijn.
- Het antwoord van gulzig hangt volledig af van de knoopvolgorde. Op de kroongraaf gebruikt een slechte volgorde n kleuren waar er 2 volstaan — en de natuurlijke volgorde doet het goed, dus een onzorgvuldige test ziet niets verkeerts.
- Degeneratie is de grens om te gebruiken: $\chi \leq d + 1$, nooit slechter dan $\Delta + 1$, en ze levert bossen en vlakke grafen hun grens gratis.
- Brooks: $\chi \leq \Delta$ tenzij de graaf volledig is of een oneven cykel. Beide uitzonderingen zijn nodig en K_3 is ze allebei.
- Exacte χ is NP-moeilijk; de implementatie hier is voor verificatie en niet voor gebruik.

Hoofdstuk 16 — Bipartiete grafen

Een graaf is **bipartiet** wanneer zijn knopen uiteenvallen in twee verzamelingen waarbij elke kant tussen beide oversteekt. Equivalent: $\chi(G) \leq 2$. Dit is de netste karakterisering in het boek, en de enige plaats waar een structurele eigenschap een werkelijk eenvoudige test heeft.

De stelling

Stelling (König, 1916). G is bipartiet dan en slechts dan als hij geen oneven cykel bevat.

Bewijs. Is G bipartiet met delen A en B , dan wisselt elke cykel ertussen af, dus terugkeren naar het startpunt vergt een even aantal stappen.

Omgekeerd, stel dat G geen oneven cykel heeft. Behandel elke component apart. Kies een wortel r en kleur elke knoop naar de pariteit van $d(r, v)$. Stel dat een kant uv twee knopen van gelijke pariteit verbond. Neem kortste paden van r naar u en naar v , en zij x hun laatste gemeenschappelijke knoop. De twee padstukken vanaf x , plus de kant uv , vormen een cykel van lengte $(d(r, u) - d(r, x)) + (d(r, v) - d(r, x)) + 1$, en die is oneven omdat $d(r, u)$ en $d(r, v)$ dezelfde pariteit hebben. Tegenspraak. ■

Het bewijs is een algoritme: BFS, kleur naar de pariteit van het niveau, en elke kant *binnen* een niveau benoemt een oneven cykel. Dat is de waarneming over BFS-niveaus uit hoofdstuk 8, nu aan het werk.

```
def two_colouring(g):
    colour = {}
    for source in g.vertices():
        if source in colour:
            continue
        colour[source] = 0
        queue = deque([source])
        while queue:
            v = queue.popleft()
            for w in g.neighbours(v):
                if w not in colour:
                    colour[w] = 1 - colour[v]
                    queue.append(w)
                elif colour[w] == colour[v]:
                    return None # an edge inside a level: odd cycle
    return colour
```

$O(n + m)$. Vergelijk dat met $\chi(G) \leq 3$, wat NP-volledig is. De sprong in moeilijkheid tussen twee kleuren en drie is de scherpste complexiteitsgrens in de grafentheorie, en hij gebeurt hier.

De controle die circulair zou zijn geweest

"Bipartiet dan en slechts dan als geen oneven cykel" verifiëren is een valstrik. De voor de hand liggende test — `is_bipartite` vergelijken met `has_odd_cycle` — is waardeloos wanneer `has_odd_cycle` geïmplementeerd is als `not is_bipartite`, en zo implementeert een verstandige bibliotheek het nu eenmaal.

Daarom somt de verificatie cyclen rechtstreeks op:

```
def _has_odd_cycle_bruteforce(g):
    for size in range(3, g.n + 1, 2):
        for subset in itertools.combinations(g.vertices(), size):
            first, *rest = subset
            for tail in itertools.permutations(rest):
                walk = (first,) + tail
                if all(g.has_edge(walk[i], walk[(i+1) % size]) for i in range(size)):
                    return True
    return False
```

Exponentieel, en de enige eerlijke manier om precies deze stelling te controleren:

```
held      ch16  Konig: G is bipartite iff it has no odd cycle  (52 graphs)
```

Noodzakelijk is niet voldoende

Bipartiete grafen zijn driehoekvrij. Het omgekeerde faalt, en de verificatie registreert het als een stelling waarvan verwacht wordt dat ze weerlegt wordt:

```
refuted   ch16  Triangle-free does not imply bipartite  (27 graphs)
```

Het getuigenis is C_5 : geen driehoek, maar wel een oneven cykel, dus $\chi = 3$. Dit doet er meer toe dan het lijkt. "Driehoekvrij" is een *lokale* voorwaarde — controleer elk drietal knopen — terwijl bipartietheid *globaal* is. Geen enkele hoeveelheid lokaal controleren stelt haar vast, en hoofdstuk 27 gaat over hoever driehoekvrijheid alleen een graaf beperkt.

```
print(is_bipartite(cycle(5)), is_bipartite(cycle(6))) # False True
print(is_bipartite(petersen()), chromatic_number(petersen())) # False 3
```

De Petersen-graaf heeft omtrek 5 — geen driehoeken, geen vierhoeken — en is nog steeds niet bipartiet.

Waarom bipartiete grafen eenvoudig zijn

Een lange lijst NP-moeilijke problemen wordt polynomiaal op bipartiete grafen:

Probleem	Algemeen	Bipartiet
Maximale koppeling	$O(n^3)$ (bloesems)	$O(m\sqrt{n})$
Minimale knopenoverdekking	NP-moeilijk	= maximale koppeling (König, h. 14)
Grootste onafhankelijke verzameling	NP-moeilijk	n – maximale koppeling
3-kleuring	NP-volledig	triviaal: $\chi \leq 2$

De reden is telkens dezelfde, en het is die uit hoofdstuk 14: de beperkingsmatrix van een bipartiete graaf is **totaal unimodulair**, dus het natuurlijke lineaire programma heeft gehele optimale hoekpunten en LP-dualiteit geeft je de combinatorische min-max-stelling rechtstreeks. Oneven cyclen zijn precies wat die eigenschap vernietigt.

"Bevat geen oneven cykel" is dus geen curiositeit. Het is de structurele reden waarom een hele familie problemen in moeilijkheid instort.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import cycle, petersen, complete_bipartite
from graphs.algorithms import is_bipartite, two_colouring, chromatic_number
for name, g in [('C5', cycle(5)), ('C6', cycle(6)), ('K33',
complete_bipartite(3,3)), ('petersen', petersen())]:
    print(f'{name:<9} bipartite={is_bipartite(g)!s:<6} chi={chromatic_number(g)!s:<6}')
print()
print('C6 two-colouring:', two_colouring(cycle(6)))
print('C5 two-colouring:', two_colouring(cycle(5)))
"
```

```
C5      bipartite=False  chi=3
C6      bipartite=True   chi=2
K33     bipartite=True   chi=2
petersen bipartite=False chi=3

C6 two-colouring: {0: 0, 1: 1, 5: 1, 2: 0, 4: 0, 3: 1}
C5 two-colouring: None
```

Oefeningen

1. Toon aan dat een graaf bipartiet is dan en slechts dan als elke deelgraaf een onafhankelijke verzameling heeft met minstens de helft van zijn knopen.
2. Hoeveel kanten kan een bipartiete graaf op n knopen hebben? Welke graaf bereikt dat?
3. Geef een graaf van omtrek 5 die niet bipartiet is, anders dan C_5 en de Petersen-graaf.
4. Leg uit waarom "driehoekvrij" in $O(n^3)$ gecontroleerd kan worden terwijl "bipartiet" $O(n + m)$ nodig heeft — en waarom de goedkopere test de sterkere eigenschap is.

Oplossingen in Bijlage E.

Kernpunten

- Bipartiet $\iff$ geen oneven cykel $\iff \chi \leq 2$, alle drie testbaar in $O(n + m)$ met de pariteit van BFS-niveaus.
- Een kant binnen een BFS-niveau is een oneven cykel. Het bewijs en het algoritme zijn hetzelfde object.
- Deze stelling verifiëren vergt cyclen onafhankelijk opsommen; de natuurlijke implementatie van "heeft een oneven cykel" is "is niet bipartiet", en die twee tegen elkaar controleren bewijst niets.
- Driehoekvrij impliceert niet bipartiet. C_5 is het kleinste getuigenis; de Petersen-graaf heeft omtrek 5 en faalt nog steeds.
- Bipartietheid maakt koppeling, knopenoverdekking en onafhankelijke verzameling alle polynomiaal, omdat ze de beperkingsmatrix totaal unimodulair maakt. Oneven cyclen breken dat.

Hoofdstuk 17 — Vlakheid

Een graaf is **vlak** wanneer hij in het platte vlak getekend kan worden zonder dat kanten elkaar kruisen. Hoofdstuk 1 hield vol dat een graaf geen meetkunde heeft; vlakheid is de uitzondering die de regel bevestigt, want ze kwantificeert over *alle* tekeningen. De tekening die jij maakte zegt niets. Het bestaan van een goede is een eigenschap van de graaf.

Wat een tekening eigenlijk is

Om over tekeningen te redeneren zonder topologie te bedrijven, vervang je ze door iets eindigs. Een **rotatiesysteem** kent elke knoop een cyclische volgorde van zijn burens toe. Dat is de volledige combinatorische inhoud van een inbedding in een oriënteerbaar oppervlak: geen coördinaten, geen lengtes, alleen "welke kant komt hierna wanneer je om deze knoop draait".

Uit een rotatiesysteem kun je **vlakdelen** mechanisch aflopen. Kom je in v aan langs uv , vertrek dan langs de buur die op u volgt in de cyclische volgorde van v ; ga door tot je terug bent waar je begon.

```
def _trace_faces(g, rotation):
    nxt = {}
    for v, order in rotation.items():
        pos = {w: i for i, w in enumerate(order)}
        for u in order:
            nxt[(u, v)] = (v, order[(pos[u] + 1) % len(order)])
    unvisited, count = set(nxt), 0
    while unvisited:
        arc = next(iter(unvisited)); count += 1
        while arc in unvisited:
            unvisited.discard(arc); arc = nxt[arc]
    return count
```

Zo werkt `is_planar` in dit boek: probeer rotatiesystemen tot er een $n - m + f = 2$ geeft. Het kost $\prod (\deg(v) - 1)!$ — 1024 systemen voor de Petersen-graaf, hopeloos voor K_6 . Echte vlakheidstests zijn $O(n)$ (Hopcroft–Tarjan, of het links-rechtsalgoritme), en dit boek implementeert die niet, omdat hun correctheid een apart betoog van twintig bladzijden is dat je niets over vlakheid zou leren.

De formule van Euler

Stelling (Euler, 1758). Voor een samenhangende vlakke graaf geldt $n - m + f = 2$.

Bewijs. Inductie naar m . Is G een boom, dan $m = n - 1$ en $f = 1$, wat $n - (n-1) + 1 = 2$ geeft. Anders heeft G een cykel; verwijder een van zijn kanten. De graaf blijft samenhangend, en de twee vlakdelen aan weerszijden van die kant versmelten tot één, dus m en f dalen elk met één en $n - m + f$ blijft gelijk. ■

De verificatie kent hier een valstrik die het benoemen waard is: **je kunt de formule van Euler niet controleren door f uit de formule van Euler te berekenen**. De verificatie haalt f op door de vlakdelen van een werkelijke inbedding af te lopen, en controleert pas daarna de identiteit:

```
held      ch17 Euler's formula: n - m + f = 2 for connected planar graphs (30
graphs)
```

Voor onsamenvhangende grafen luidt de formule $n - m + f = 1 + k$ met k componenten. De gebruikelijke formulering neemt samenhang stilzwijgend aan.

De kantengrens, en waarom ze niet volstaat

Gevolg. Een enkelvoudige vlakke graaf met $n \geq 3$ heeft $m \leq 3n - 6$. Is hij bovendien driehoekvrij, dan $m \leq 2n - 4$.

Bewijs. Elk vlakdeel wordt door minstens 3 kanten begrensd, en elke kant grenst aan hoogstens 2 vlakdelen, dus $2m \geq 3f$. Invullen van $f = 2 - n + m$ geeft $2m \geq 3(2 - n + m)$, dus $m \leq 3n - 6$. Driehoekvrij betekent dat elk vlakdeel minstens 4 kanten heeft, wat $2m \geq 4f$ en de tweede grens geeft. ■

Dat is opnieuw een dubbeltelling — de techniek uit hoofdstuk 3, voor de derde keer.

K_5 heeft $n = 5$, $m = 10 > 9$, dus hij is niet vlak. Klaar in één regel.

$K_{3,3}$ heeft $n = 6$, $m = 9 \leq 12$, dus de grens zegt niets. Hij is toch niet vlak, en dat is het punt: **de Euler-grens is noodzakelijk, niet voldoende**. De verificatie registreert het omgekeerde als een stelling waarvan verwacht wordt dat ze weerlegd wordt:

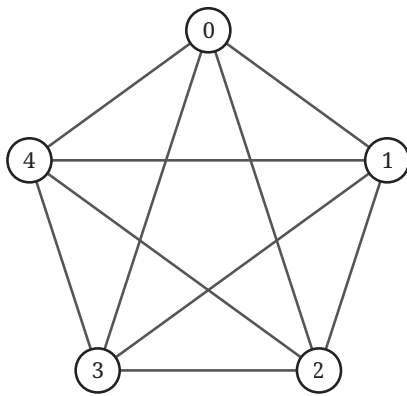
```
refuted   ch17 m <= 3n - 6 implies planar (9 graphs)
```

De driehoekvrije grens vangt hem wel: $K_{3,3}$ is bipartiet, en $9 > 2 \cdot 6 - 4 = 8$. Twee grenzen, en je moet weten welke van toepassing is.

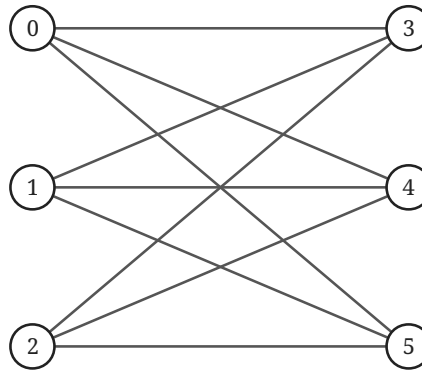
K4	n=4	m=6	planar=True	euler_bound=True	bip_bound=False	faces=4
K5	n=5	m=10	planar=False	euler_bound=False	bip_bound=False	
faces=None						
K33	n=6	m=9	planar=False	euler_bound=True	bip_bound=False	
faces=None						
petersen	n=10	m=15	planar=False	euler_bound=True	bip_bound=True	
faces=None						

De Petersen-graaf doorstaat **beide** grenzen en is nog steeds niet vlak. Geen enkel telargument over n en m zal vlakheid ooit beslissen, want vlakheid gaat niet over hoeveel kanten er zijn.

Kuratowski en Wagner



K5 - the first non-planar graph



K3,3 - bipartite, and not planar

De twee verboden minoren. Elke tekening van beide heeft een kruising, en elke niet-vlakke graaf bevat er een van.

Stelling (Kuratowski, 1930). G is vlak dan en slechts dan als hij geen *onderverdeling* van K_5 of $K_{3,3}$ bevat.

Stelling (Wagner, 1937). G is vlak dan en slechts dan als hij geen K_5 -minor en geen $K_{3,3}$ -minor heeft.

Een **onderverdeling** vervangt kanten door paden; een **minor** ontstaat door knopen te verwijderen, kanten te verwijderen, en kanten samen te trekken. De twee stellingen zijn hier equivalent, al zijn minoren het robuustere begrip en degene waarop hoofdstuk 31 voortbouwt.

Twee verboden grafen, en dat is het volledige antwoord — een opmerkelijk kleine verzameling belemmeringen voor zo'n rijke eigenschap. Hoofdstuk 31 legt uit waarom elke minor-gesloten eigenschap een eindige belemmeringsverzameling heeft, wat Robertson–Seymour is, en waarom die stelling niet-constructief is.

De verificatie controleert Wagner tegen de inbeddingszoektocht — twee volstrekt ongerelateerde berekeningen:

```
held      ch17  Wagner: planar iff no K5 minor and no K3,3 minor  (52 graphs)
```

De Petersen-graaf is de standaardillustratie: trek de vijf spaken samen en je krijgt K_5 .

Gevolgen

Vlakke grafen zijn ijl ($m < 3n$), dus $m = O(n)$ en ze hebben een knoop van graad hoogstens 5 — de gemiddelde graad is onder 6. Dat ene feit geeft:

- **De zeskleurenstelling**, onmiddellijk: degeneratie ≤ 5 , dus $\chi \leq 6$ per hoofdstuk 15.
- **De vijfkleurenstelling**, met een Kempe-ketenargument (hoofdstuk 18).
- **De vierkleurenstelling**, met 633 configuraties en een computer (hoofdstuk 18).

Vlakke grafen hebben ook $O(\sqrt{n})$ -separatoren (Lipton–Tarjan), wat verdeel-en-heersaanpakken erop laat werken en de reden is dat veel NP-moeilijke problemen subexponentiële algoritmen hebben zodra de invoer vlak is.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete, complete_bipartite, petersen, cycle
from graphs.planar import is_planar, euler_bound, bipartite_euler_bound,
planar_face_count
for name, g in [('K4', complete(4)), ('K5', complete(5)),
                ('K3,3', complete_bipartite(3,3)), ('petersen', petersen())]:
    print(f'{name:<9} m<=3n-6: {euler_bound(g)!s:<6} m<=2n-4:
{bipartite_euler_bound(g)!s:<6} planar: {is_planar(g)}')
f = planar_face_count(complete(4))
print()
print('K4 faces from a real embedding:', f, ' n - m + f =', 4 - 6 + f)
"
```

```
K4      m<=3n-6: True   m<=2n-4: False  planar: True
K5      m<=3n-6: False  m<=2n-4: False  planar: False
K3,3    m<=3n-6: True   m<=2n-4: False  planar: False
petersen m<=3n-6: True   m<=2n-4: True   planar: False
```

```
K4 faces from a real embedding: 4  n - m + f = 2
```

K_4 in het vlak getekend heeft vier vlakdelen — drie driehoeken en het buitengebied — en de formule van Euler klopt op een getal dat afgelopen werd in plaats van aangenomen.

Oefeningen

1. Gebruik de formule van Euler om te tonen dat elke vlakke graaf een knoop van graad hoogstens 5 heeft.
2. Toon aan dat K_5 min een willekeurige kant vlak is.
3. De Petersen-graaf doorstaat beide kantengrenzen. Geef een kort argument dat hij toch niet vlak is, zonder Kuratowski aan te roepen.
4. Hoeveel vlakdelen heeft een vlakke inbedding van een boom op n knopen? Controleer tegen de formule van Euler.

Oplossingen in Bijlage E.

Kernpunten

- Vlakheid kwantificeert over alle tekeningen, dus ze is een eigenschap van de graaf ook al is een tekening dat niet.
- Een rotatiesysteem is de volledige combinatorische inhoud van een tekening. Vlakdelen kunnen er mechanisch uit afgelopen worden, en $n - m + f = 2$ karakteriseert vlakheid.

- Controleer de formule van Euler met een vlakdeeltelling uit een werkelijke inbedding. f berekenen door de formule om te schrijven test niets.
- $m \leq 3n - 6$ is noodzakelijk, niet voldoende — $K_{3,3}$ doorstaat haar. De driehoekvrije verfijning $m \leq 2n - 4$ vangt $K_{3,3}$ maar niet de Petersen-graaf. Geen kantentellend argument beslist vlakheid.
- Kuratowski en Wagner: twee verboden structuren, en dat is het hele antwoord.
- Vlakke grafen zijn ijl en 5-degeneraat, wat je de zeskleurenstelling gratis oplevert.

Hoofdstuk 18 — De vijf- en de vierkleurenstelling

Twee stellingen over dezelfde objecten. De ene heeft een bewijs dat je in een bladzijde kunt lezen. De andere heeft een bewijs dat je helemaal niet kunt lezen, en dit hoofdstuk gaat er deels over wat dat betekent.

Zes kleuren, gratis

Hoofdstuk 17 stelde vast dat elke vlakke graaf een knoop van graad hoogstens 5 heeft — anders zou $2m = \sum \deg(v) \geq 6n$ zijn, in tegenspraak met $m \leq 3n - 6$. Vlakke grafen zijn dus 5-degeneraat, en de grens uit hoofdstuk 15 geeft $\chi \leq 6$ onmiddellijk.

held ch18 Every planar graph has a vertex of degree at most 5 (51 graphs)

Dat kostte twee regels. Tot vijf komen kost een bladzijde, en tot vier kwam een eeuw.

Vijf kleuren

Stelling (Heawood, 1890). Elke vlakke graaf is 5-kleurbaar.

Bewijs. Inductie naar n . Neem een knoop v met $\deg(v) \leq 5$ en kleur $G - v$ met 5 kleuren.

Gebruiken de buren van v hoogstens 4 kleuren, dan is er een kleur vrij — klaar.

Anders is $\deg(v) = 5$ en gebruiken zijn buren alle vijf. Leg een vlakke inbedding vast en label de buren $v_1, \dots, v_5$ in rotatievolgorde, waarbij v_i kleur i heeft.

Beschouw de deelgraaf voortgebracht door de kleuren 1 en 3, en zij H de component die v_1 bevat. Is $v_3 \notin H$, verwissel dan de kleuren 1 en 3 in heel H . Dat blijft geldig — H is een volledige component van die deelgraaf, dus geen kant verlaat haar met een kleurconflict — en nu heeft v_1 kleur 3, wat kleur 1 vrijmaakt voor v .

Is $v_3 \in H$, dan is er een pad van v_1 naar v_3 dat alleen de kleuren 1 en 3 gebruikt. Samen met v omsluit dat een gebied. Beschouw nu de kleuren 2 en 4 en de component die v_2 bevat. Elk pad van v_2 naar v_4 zou dat eerste pad moeten kruisen — en in een vlakke inbedding kan dat niet, want kruisingen bestaan er niet. Dus v_4 zit niet in de component van v_2 , en dezelfde verwisseling maakt kleur 2 vrij. ■

Dat is een **Kempe-ketenargument**, en de tweekleurige componentverwisseling is de techniek. Let op waar vlakheid binnenkomt: bij de laatste stap. Een pad kan een ander pad niet kruisen. Dat is de enige topologische invoer, en ze doet al het werk.

Vier kleuren

Stelling (Appel–Haken, 1976). Elke vlakke graaf is 4-kleurbaar.

Kempe publiceerde een bewijs in 1879. Het hield elf jaar stand tot Heawood er een fout in vond — het ketenverwisselingsargument faalt wanneer twee verwisselingen op elkaar inwerken, en dat geval had Kempe niet behandeld. Wat het wrak overleefde is de vijfkleurenstelling hierboven.

Het uiteindelijke bewijs heeft twee delen:

- een **onvermijdbare verzameling**: een lijst van 1936 configuraties (later teruggebracht tot 633) zodat elke vlakke graaf er minstens één moet bevatten;
- **reduceerbaarheid**: een aantoning dat een minimaal tegenvoorbeeld er geen van kan bevatten.

Het tweede deel is waar de computer binnenkomt. Elke configuratie vergt het controleren van een groot aantal kleuringen, en dat met de hand doen voor 633 configuraties is niet haalbaar.

Het bewijs is sindsdien geverifieerd — Robertson, Sanders, Seymour en Thomas vereenvoudigden het in 1997, en Gonthier produceerde in 2005 een volledig machinaal gecontroleerde versie in Coq. Bij die laatste loont het stil te staan: het bewijs is nu strenger geverifieerd dan de meeste bewijzen die mensen schrijven, en het is nog steeds onleesbaar.

Dit boek kan het niet bewijzen. Wat het wel kan, is het controleren:

```
held      ch18  Every planar graph is 4-colourable  (51 graphs)
```

Die regel is geen bewijsmateriaal voor de stelling. Ze zegt dat elke vlakke graaf op hoogstens 6 knopen 4-kleurbaar is, en daar twijfelde nooit iemand aan — het eerste mogelijke tegenvoorbeeld zou enorm zijn. Ze staat om een andere reden in de verificatie: om te vangen dat de uitspraak verkeerd opgeschreven wordt, en om luid te falen wanneer `is_planar` of `chromatic_number` stukgaat. "De controle slaagt" verwarren met "de stelling wordt ondersteund" is precies de fout die de verificatie van dit boek niet wil aanmoedigen.

Wat de moeilijkheid betekent

De kloof tussen vijf en vier kleuren is geen kloof in inspanning. Ze is structureel:

- **Zes** volgt uit een graadgrens, die volgt uit tellen.
- **Vijf** volgt uit een lokaal verwisselingsargument dat vlakheid geldig maakt.
- **Vier** heeft geen bekend argument dat gevalsonderscheid over honderden configuraties vermijdt.

Niemand heeft een kort bewijs gevonden en er is geen reden er een te verwachten. De vierkleurenstelling is misschien gewoon een ware uitspraak waarvan het kortste bewijs groot is — en als dat zo is, is de computer geen kruk maar het enige beschikbare instrument.

Het loont te vergelijken met een nabije vraag die *wel* eenvoudig is. Op de torus is het antwoord 7, bewezen met Heawoods telargument in één bladzijde, zonder gevallen. Hoger geslacht is *eenvoudiger* dan het vlak, omdat de telgrens daar scherp is en in het vlak niet.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete, cycle, petersen
from graphs.algorithms import chromatic_number
from graphs.planar import is_planar, degeneracy

for name, g in [('K4', complete(4)), ('C5', cycle(5)), ('petersen', petersen())]:
    print(f'{name:<9} planar={is_planar(g)!s:<6} degeneracy={degeneracy(g)}
chi={chromatic_number(g)}')
```

K4	planar=True	degeneracy=3	chi=4
C5	planar=True	degeneracy=2	chi=3
petersen	planar=False	degeneracy=3	chi=3

K_4 toont dat vier kleuren soms nodig zijn, dus de stelling kan niet verbeterd worden. De Petersen-graaf toont dat het omgekeerde in de andere richting faalt: niet-vlakke grafen kunnen best 3-kleurbaar zijn, dus vlakheid is voldoende voor $\chi \leq 4$ en bij lange na niet noodzakelijk.

Oefeningen

1. Bewijs de zeskleurenstelling rechtstreeks uit $m \leq 3n - 6$.
2. Waar precies gebruikt het vijfkleurenbewijs vlakheid? Wijs de ene stap aan.
3. Geef een vlakke graaf die precies 4 kleuren nodig heeft, anders dan K_4 .
4. Kempes argument uit 1879 was fout. Wat gaat er, zonder het op te zoeken, mis wanneer je twee Kempe-verwisselingen na elkaar toepast?

Oplossingen in Bijlage E.

Kernpunten

- Zes kleuren volgen uit 5-degeneratie, die volgt uit $m \leq 3n - 6$. Twee regels.
- Vijf kleuren volgen uit een Kempe-ketenargument. Vlakheid wordt in precies één stap gebruikt: twee paden kunnen elkaar niet kruisen.
- Vier kleuren vergden 633 configuraties en een computer, en er is nog steeds geen leesbaar bewijs. Het is sindsdien machinaal gecontroleerd in Coq.
- De vierkleurencontrole van de verificatie bevestigt de *uitspraak* op kleine grafen. Ze is geen bewijsmateriaal voor de stelling, en haar als zodanig behandelen zou precies de fout zijn die deze verificatieaanpak wil vermijden.
- K_4 toont dat de grens scherp is; de Petersen-graaf toont dat vlakheid verre van noodzakelijk is.

Hoofdstuk 19 — Perfecte en koordale grafen

Hoofdstuk 15 bewees $\omega(G) \leq \chi(G)$ in één regel en liet de kloof onverklaard. Dit hoofdstuk gaat over de grafen waar er geen kloof is — een klasse groot genoeg om de meeste eenvoudige gevallen in het boek te bevatten, en gekarakteriseerd door een stelling die veertig jaar kostte.

De definitie, en waarom ze over deelgrafen kwantificeert

G is **perfect** wanneer $\chi(H) = \omega(H)$ voor elke **geïnduceerde** deelgraaf H .

De kwantor is essentieel en is de hele reden dat de definitie werkt. Alleen $\chi(G) = \omega(G)$ eisen geeft een klasse zonder bruikbare structuur — C_5 plus een disjuncte K_3 heeft $\chi = \omega = 3$ terwijl hij C_5 bevat, de canonieke imperfecte graaf. Het van elke geïnduceerde deelgraaf eisen maakt de eigenschap **erfelijk**, en dat is wat inductie toelaat; erfelijke klassen zijn precies die welke door verboden geïnduceerde deelgrafen gedefinieerd kunnen worden.

C_5 is de kleinste imperfecte graaf: $\omega = 2$ (geen driehoek) maar $\chi = 3$ (oneven cykel).

C4	chordal=False	perfect=True	chi=2	omega=2	odd_hole=False
C5	chordal=False	perfect=False	chi=3	omega=2	odd_hole=True
K4	chordal=True	perfect=True	chi=4	omega=4	odd_hole=False
P4	chordal=True	perfect=True	chi=2	omega=2	odd_hole=False

Koordale grafen

Een graaf is **koordaal** wanneer elke cykel van lengte minstens 4 een koorde heeft — equivalent: hij heeft geen geïnduceerde cykel langer dan een driehoek.

Stelling. Elke koordale graaf is perfect.

Het bewijs draait om een structureel feit dat apart vermelding verdient.

Lemma (Dirac). Elke koordale graaf heeft een **simpliciale** knoop: een knoop wiens burens een klik vormen.

Daarmee is de stelling een eenvoudige inductie: neem een simpliciale knoop v , kleur $G - v$ met $\omega(G - v)$ kleuren, en zet v terug. Zijn buurverzameling is een klik van grootte $\deg(v)$, dus samen met v een klik van grootte $\deg(v) + 1 \leq \omega(G)$. Dus op zijn burens verschijnen hoogstens $\omega(G) - 1$ kleuren, en er is er één vrij.

■

Herhaald simpliciale knopen verwijderen geeft een **perfecte eliminatievolgorde**, en een graaf is koordaal precies wanneer hij er een heeft. Dat levert een werkelijk lineaire test op — ongebruikelijk voor dit boek, want er is geen exponentiële terugval:

```
def is_chordal(g):
    order = maximum_cardinality_order(g)
    position = {v: i for i, v in enumerate(order)}
    for v in order:
        earlier = [w for w in g.neighbours(v) if position[w] < position[v]]
        if not earlier:
            continue
        parent = max(earlier, key=lambda w: position[w])
        for w in earlier:
            if w != parent and not g.has_edge(parent, w):
                return False
    return True
```

Het controleren vergde een orakel dat het idee niet deelt. De verificatie zoekt rechtstreeks naar een geïnduceerde cykel van lengte ≥ 4 :

```
held      ch19 Chordality test agrees with searching for a chordless long cycle
(52 graphs)
held      ch19 Every chordal graph is perfect (44 graphs)
```

Op koordale grafen zijn kleuring, maximale kliek, grootste onafhankelijke verzameling en minimale kliekoverdekking allemaal lineair of bijna lineair — stuk voor stuk NP-moeilijk in het algemeen. Koordale grafen zijn ook precies de doorsnedegrafen van deelbomen van een boom, en daarom duiken ze op bij ijle-matrixeliminatie en bij probabilistische grafische modellen.

De twee grote stellingen

Stelling (Lovász, 1972 — de zwakke perfecte-grafenstelling). G is perfect dan en slechts dan als zijn complement perfect is.

Een verrassende uitspraak: perfectie gaat over klieken en kleuringen, en complementeren verwisselt klieken met onafhankelijke verzamelingen, dus er is geen voor de hand liggende reden waarom de eigenschap zou overleven. Ze doet het, en de stelling verdubbelt onmiddellijk elk resultaat over perfecte grafen.

Stelling (Chudnovsky, Robertson, Seymour, Thomas, 2006 — de sterke perfecte-grafenstelling). G is perfect dan en slechts dan als noch G noch zijn complement een geïnduceerde oneven cykel van lengte minstens 5 bevat.

Vermoed door Berge in 1961, 45 jaar later bewezen in een artikel van 150 bladzijden. De verboden structuren zijn de **oneven gaten** (geïnduceerde oneven cyclen van lengte ≥ 5) en **oneven antigaten** (hun complementen). De hele klasse wordt gedefinieerd door twee oneindige families uit te sluiten, wat dezelfde vorm van antwoord is als die van Kuratowski in hoofdstuk 17 — en het is de reden dat perfecte grafen in polynomiale tijd herkenbaar zijn.

De verificatie controleert de equivalentie, en voor grafen tot vijf knopen is dat een echte controle van de *uitspraak*, ook al is het geen bewijsmateriaal voor de stelling:

```
held      ch19 Berge: perfect iff no odd hole and no odd antihole (52 graphs)
```


Merk op dat C_5 zijn eigen complement is, dus hij is tegelijk een oneven gat en een oneven antigat — de minimale belemmering in beide richtingen tegelijk.

Waar perfectie al eerder opdook

Dit hoofdstuk geeft een naam aan een patroon dat deel III steeds gebruikte zonder het te benoemen:

Klasse	Perfect?	Welke stelling eenvoudig werd
Bipartiet	ja	König (h. 14): $\chi = \omega = 2$, koppeling = overdekking
Bossen	ja	triviaal 2-kleurbaar (h. 6)
Koordaal	ja	gulzige kleuring is optimaal
Intervalgrafen	ja	plannen op vroegste eindtijd
Complementen van bipartiete	ja	via de stelling van Lovász
Oneven cykels $C_5, C_7, \dots$	nee	$\chi = 3, \omega = 2$

De stelling van König in hoofdstuk 14 is het bipartiete geval van een veel algemener verschijnsel. Kom je een grafenprobleem tegen dat op een klasse onverwacht eenvoudig is, dan is perfectie het eerste om te controleren.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import cycle, complete, path
from graphs.algorithms import chromatic_number, max_clique_size
from graphs.perfect import is_chordal, is_perfect, has_odd_hole, has_odd_antihole
for name, g in [ ('C4', cycle(4)), ('C5', cycle(5)), ('K4', complete(4)), ('P4',
path(4)) ]:
    print(f'{name:<4} chordal={is_chordal(g)!s:<6} perfect={is_perfect(g)!s:<6} '
          f'chi={chromatic_number(g)} omega={max_clique_size(g)} '
          f'odd_hole={has_odd_hole(g)!s:<6} odd_antihole={has_odd_antihole(g)!s:<6}')
"
```

C4	chordal=False	perfect=True	chi=2	omega=2	odd_hole=False	odd_antihole=False
C5	chordal=False	perfect=False	chi=3	omega=2	odd_hole=True	odd_antihole=True
K4	chordal=True	perfect=True	chi=4	omega=4	odd_hole=False	odd_antihole=False
P4	chordal=True	perfect=True	chi=2	omega=2	odd_hole=False	odd_antihole=False

C_4 is perfect maar **niet** koordaal — het is een geïnduceerde vierhoek zonder koorde. Koordaliteit is voldoende voor perfectie en niet noodzakelijk, en C_4 is het kleinste ding dat dat aantoont.

Oefeningen

1. Ga met de hand na dat C_5 $\omega = 2$ en $\chi = 3$ heeft.
2. Toon aan dat elke intervalgraaf koordaal is.

3. Waarom kwantificeert de definitie van perfect over geïnduceerde deelgrafen en niet over alle deelgrafen? Geef een graaf die het alternatief zou breken.
4. C_5 is zelfcomplementair. Vind een andere zelfcomplementaire graaf, en zeg of hij perfect is.

Oplossingen in Bijlage E.

Kernpunten

- Perfect betekent $\chi = \omega$ op **elke geïnduceerde deelgraaf**. De kwantor maakt de klasse erfelijk, en dat is wat haar bruikbaar maakt.
- C_5 is de kleinste imperfecte graaf, en hij is zijn eigen complement — tegelijk een oneven gat en een oneven antigat.
- Koordale grafen zijn perfect, via simpliciale knopen en perfecte eliminatievolgordes, en koordaliteit is in lineaire tijd testbaar.
- Koordaal is voldoende voor perfect, niet noodzakelijk: C_4 is perfect en niet koordaal.
- Lovász: perfectie overleeft complementeren. Chudnovsky–Robertson–Seymour–Thomas: perfectie is precies de afwezigheid van oneven gaten en oneven antigaten.
- Bipartiete grafen zijn het perfecte geval dat je al ontmoette. König is een gevolg van een veel groter verhaal.

Hoofdstuk 20 — Hamiltoniciteit

Een **Hamiltoniaanse cykel** bezoekt elke knoop precies één keer en keert terug naar zijn start. Vergelijk met een Euler-circuit, dat elke *kant* precies één keer gebruikt — en dat een perfecte karakterisering in één regel heeft: een samenhangende graaf heeft er een precies wanneer elke graad even is.

Voor Hamiltoniciteit is niets van dien aard bekend, en dit hoofdstuk gaat over wat je in plaats daarvan krijgt.

Waarom de analogie faalt

De voorwaarde van Euler is **lokaal**: controleer de graad van elke knoop afzonderlijk. Hamiltoniciteit is onherleidbaar **globaal** — of er een cykel bestaat hangt af van hoe de hele graaf in elkaar past, en geen enkele hoeveelheid lokale inspectie beslist het.

Dat is niet slechts een gebrek aan verbeelding. De Hamiltoniaanse cykel is **NP**-volledig (hoofdstuk 22), dus een lokaal controleerbare karakterisering zou $P = NP$ geven. De afwezigheid van een Hamiltoniaanse tegenhanger van de stelling van Euler is een stellingvormig gat met een complexiteitstheoretische verklaring.

Het vak biedt dus **voldoende** voorwaarden: hypothesen sterk genoeg om een cykel af te dwingen. Ze zeggen allemaal een variant van "genoeg kanten, gelijkmatig verspreid".

Dirac en Ore

Stelling (Dirac, 1952). Is $n \geq 3$ en heeft elke knoop graad minstens $n/2$, dan is G Hamiltoniaans.

Stelling (Ore, 1960). Is $n \geq 3$ en geldt $\deg(u) + \deg(v) \geq n$ voor elk niet-aangrenzend paar u, v , dan is G Hamiltoniaans.

Ore impliceert Dirac — is elke graad minstens $n/2$, dan sommeert elk paar tot minstens n — dus Ore is de sterkere stelling en geldt voor strikt meer grafen.

Bewijs van Ore. Stel van niet, en zij G een tegenvoorbeeld met de meeste kanten: het voldoet aan de voorwaarde van Ore, is niet Hamiltoniaans, en elke kant toevoegen maakt het Hamiltoniaans.

Neem niet-aangrenzende u, v . uv toevoegen creëert een Hamiltoniaanse cykel, die uv moet gebruiken, dus G heeft een Hamiltoniaans **pad** $u = x_1, x_2, \dots, x_n = v$.

Beschouw nu de verzamelingen $S = \{i : u \text{ grenst aan } x_{i+1}\}$ en $T = \{i : v \text{ grenst aan } x_i\}$, beide deelverzamelingen van $\{1, \dots, n-1\}$. Er geldt $|S| + |T| = \deg(u) + \deg(v) \geq n$, en beide liggen in een verzameling van grootte $n - 1$, dus per duivenhokprincipe delen ze een index i .

Maar dan zijn $u x_{i+1}$ en $x_i v$ allebei kanten, en de cykel $u, x_2, \dots, x_i, v, x_{n-1}, \dots, x_{i+1}, u$ — het pad vooruit tot x_i , spring naar v , dan achteruit tot x_{i+1} , dan terug naar u — is Hamiltoniaans. Tegenspraak. ■

De opzet met een **extremaal tegenvoorbeeld** (neem degene met de meeste kanten) is dezelfde zet als in het opspannendeboombewijs van hoofdstuk 7, en de duivenhokstap is dezelfde als in hoofdstuk 3.

Voldoende is heel ver van noodzakelijk

De voorwaarden zijn veeleisend, en de meeste Hamiltoniaanse grafen falen er ruimschoots aan:

C ₅	ham=True	dirac=False	ore=False	mindeg=2	n/2=2.5
K ₄	ham=True	dirac=True	ore=True	mindeg=3	n/2=2.0
petersen	ham=False	dirac=False	ore=False	mindeg=3	n/2=5.0
K ₃₃	ham=True	dirac=True	ore=True	mindeg=3	n/2=3.0
K ₂₃	ham=False	dirac=False	ore=False	mindeg=2	n/2=2.5

C₅ is per constructie Hamiltoniaans en faalt aan Dirac — minimale graad 2 tegen een drempel van 2,5. De verificatie registreert het omgekeerde als een stelling waarvan verwacht wordt dat ze weerlegd wordt:

```
refuted   ch20  Every Hamiltonian graph satisfies Dirac's condition  (5 graphs)
```

Vergelijk de laatste twee rijen. K_{3,3} en K_{2,3} zien er allebei niet bijzonder uit, en de ene is Hamiltoniaans en de andere niet — om een reden die Dirac niet kan zien. K_{a,b} is Hamiltoniaans precies wanneer $a = b$, want een cykel in een bipartiete graaf moet tussen de zijden afwisselen.

Bondy–Chvátal, en waarom Ore werkt

Achter de stelling van Ore zit een nettere uitspraak.

Stelling (Bondy–Chvátal, 1976). Zij de **afsluiting** van G het resultaat van herhaaldelijk niet-aangrenzende u, v met $\deg(u) + \deg(v) \geq n$ te verbinden. Dan is G Hamiltoniaans dan en slechts dan als zijn afsluiting dat is.

Dit is werkelijk verrassend: kanten toevoegen kan Hamiltoniciteit uiteraard niet vernietigen, maar dat deze bepaalde toevoegingen haar niet kunnen *creëren* is de inhoud.

De stelling van Ore is nu een gevolg. Een graaf die aan de voorwaarde van Ore voldoet heeft afsluiting K_n, en die is Hamiltoniaans, dus de graaf is het.

```
held      ch20  Bondy-Chvatal: G is Hamiltonian iff its closure is  (49 graphs)
```

Noodzakelijke voorwaarden

De andere kant op zijn de bruikbare feiten negatief — manieren om te bewijzen dat een graaf *niet* Hamiltoniaans is:

- Een Hamiltoniaanse graaf is **2-samenhangend**: de cykel geeft twee disjuncte paden tussen elk paar, dus Menger (hoofdstuk 12) is van toepassing.
- k knopen uit een Hamiltoniaanse graaf verwijderen laat hoogstens k componenten achter, want de cykel valt in hoogstens k bogen uiteen. Dit is vaak het snelste weerleggingsmiddel.
- Een Hamiltoniaanse bipartiete graaf heeft even grote delen.

De Petersen-graaf is niet Hamiltoniaans, en het standaardbewijs is een gevalsonderscheid naar hoeveel spaken de cykel gebruikt. Hij *heeft* wel een Hamiltoniaans pad, en daarom heet hij **hypohamiltoniaans**: niet

Hamiltoniaans, maar $G - v$ is Hamiltoniaans voor elke v . Hij is, voorspelbaar, ook hier het tegenvoorbeeld van dit boek.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import cycle, complete_bipartite, petersen
from graphs.hamilton import hamiltonian_cycle, hamiltonian_path, dirac_condition,
is_hamiltonian
print('C5 hamiltonian cycle:', hamiltonian_cycle(cycle(5)), ' dirac says:',
dirac_condition(cycle(5)))
print('K2,3 hamiltonian:      ', is_hamiltonian(complete_bipartite(2,3)), '(unequal
parts)')
print('K3,3 hamiltonian:      ', is_hamiltonian(complete_bipartite(3,3)), '(equal
parts)')
print('petersen cycle:        ', hamiltonian_cycle(petersen()))
print('petersen path:         ', hamiltonian_path(petersen()) is not None)
"
```

```
C5 hamiltonian cycle: [0, 1, 2, 3, 4] dirac says: False
K2,3 hamiltonian:      False (unequal parts)
K3,3 hamiltonian:      True (equal parts)
petersen cycle:        None
petersen path:         True
```

C_5 is de netste illustratie van de kloof: een vijfcykel is een Hamiltoniaanse cykel, en de best bekende voldoende voorwaarde kan dat niet zien.

Oefeningen

1. Toon aan dat $K_{\{a,b\}}$ Hamiltoniaans is dan en slechts dan als $a = b \geq 2$.
2. Bewijs dat een Hamiltoniaanse graaf geen snijknoop heeft.
3. Ga na dat de voorwaarde van Ore die van Dirac impliceert, en geef een graaf die aan Ore voldoet maar niet aan Dirac.
4. k knopen uit een Hamiltoniaanse graaf verwijderen laat hoogstens k componenten achter. Gebruik dat om van een graaf naar keuze te tonen dat hij niet Hamiltoniaans is.

Oplossingen in Bijlage E.

Kernpunten

- Euler-circuits hebben een lokale karakterisering; Hamiltoniaanse cycli geen, en NP-volledigheid verklaart waarom je er geen moet verwachten.
- Dirac ($\delta \geq n/2$) en Ore ($\deg u + \deg v \geq n$ voor niet-aangrenzende paren) zijn voldoende, niet noodzakelijk. C_5 is Hamiltoniaans en faalt aan beide.
- Ore's bewijs gebruikt een extremaal tegenvoorbeeld plus duivenhok — dezelfde twee zetten als in de hoofdstukken 7 en 3.

- De afsluitingsstelling van Bondy–Chvátal is de eigenlijke uitspraak; Ore is haar gevolg.
- Om Hamiltoniciteit te weerleggen, gebruik de negatieve feiten: 2-samenhang, de grens van k knopen tegen k componenten, en gelijke delen in het bipartiete geval.

Hoofdstuk 21 — Kliken, onafhankelijke verzamelingen, overdekkingen

Drie beroemde NP-moeilijke problemen. Het zijn hetzelfde probleem, en dat helder inzien is meer waard dan welk afzonderlijk algoritme ervoor ook.

Drie definities

- Een **klik** is een verzameling paarsgewijs aangrenzende knopen. $\omega(G)$ is de grootste.
- Een **onafhankelijke verzameling** is een verzameling paarsgewijs niet-aangrenzende knopen. $\alpha(G)$ is de grootste.
- Een **knopenoverdekking** is een verzameling die elke kant raakt. $\tau(G)$ is de kleinste.

Ze zijn één probleem

Propositie. S is een klik in G dan en slechts dan als S een onafhankelijke verzameling is in $\bar{G}$. Dus $\omega(G) = \alpha(\bar{G})$.

Bewijs. "Elk paar aangrenzend in G " en "geen paar aangrenzend in $\bar{G}$ " zijn dezelfde uitspraak, want $\bar{G}$ heeft precies de niet-kanten van G . ■

Stelling (Gallai, 1959). $\alpha(G) + \tau(G) = n$.

Bewijs. S is onafhankelijk dan en slechts dan als $V \setminus S$ een knopenoverdekking is: geen kant heeft beide eindpunten in S precies wanneer elke kant een eindpunt erbuiten heeft. Dat is een bijectie tussen onafhankelijke verzamelingen en knopenoverdekkingen die de grootte omkeert. De grootste van de ene komt dus overeen met de kleinste van de andere. ■

Beide worden gecontroleerd tegen onafhankelijke uitputtende zoektochten en niet tegen elkaar:

held	ch21	Gallai: $\alpha(G) + \tau(G) = n$ (52 graphs)
held	ch21	A clique in G is an independent set in the complement (52 graphs)

C5	n=5	alpha=2	tau=3	alpha+tau=5	omega=2	omega(comp)=2
petersen	n=10	alpha=4	tau=6	alpha+tau=10	omega=2	omega(comp)=4
K4	n=4	alpha=1	tau=3	alpha+tau=4	omega=4	omega(comp)=1

Lees de Petersen-rij: $\alpha = 4$, en de grootste klik van het complement is ook 4. De identiteit is geen toeval van kleine gevallen; het is dezelfde verzameling knopen, twee keer bekeken.

Het gevolg is praktisch. **Los er één op, en je hebt alle drie opgelost** — een exact algoritme, een benadering, of een hardheidsbewijs voor één ervan draagt onmiddellijk over. Het betekent ook dat je niet kunt hopen er één eenvoudig te vinden: had de onafhankelijke verzameling een polynomiaal algoritme, dan hadden klik en knopenoverdekking dat ook.

Waarom complementeren niet gratis is

Er zit een addertje onder het gras, en het is het soort dat asymptotiek verbergt.

De reductie van kliek naar onafhankelijke verzameling complementeert de graaf. Een ijle graaf met $m = O(n)$ heeft een complement met $\Theta(n^2)$ kanten. Een algoritme waarvan de looptijd van m afhangt, wordt na de reductie dus veel trager, ook al is de reductie "lineaire tijd" in de zin die voor de complexiteitstheorie telt.

Hier verdient de bitsetrepresentatie van hoofdstuk 2 haar plaats. Een bitmaskerrij complementeren is één `~` en een masker; buurverzamelingen snijden is één `&`. Kliekzoektocht op een dichte graaf in een adjacencielijstrepresentatie is precies het slechtste geval dat de metingen van hoofdstuk 2 aanwezen.

```
def common_neighbours(self, u, v):
    return (self.rows[u] & self.rows[v]).bit_count()
```

Elke serieuze implementatie van maximale kliek is op die operatie gebouwd.

De natuurlijke gulzige grens

De drie problemen laten alle drie een voor de hand liggende gulzige heuristiek toe, en het is nuttig te weten hoe slecht elk zich gedraagt voordat hoofdstuk 23 het echt behandelt:

- **Knopenoverdekking** heeft een 2-benadering (hoofdstuk 23), en onder het unieke-spellenvermoeden is niets beters mogelijk.
- **Onafhankelijke verzameling** en **kliek** hebben *geen* benadering met constante factor, en zijn zelfs niet binnen $n^{1-\epsilon}$ te benaderen tenzij $P = NP$. Dat is een van de sterkste bekende onbenaderbaarheidsresultaten.

Die asymmetrie is verbluffend, gegeven dat de problemen equivalent zijn. De oplossing is dat de equivalentie *exacte* antwoorden behoudt maar geen *verhoudingen*: een overdekking van grootte $\tau + 1$ is een 1,01-benadering wanneer $\tau = 100$, terwijl de bijbehorende onafhankelijke verzameling van grootte $\alpha - 1$ een 2-benadering kan zijn wanneer $\alpha = 2$. Complementeren beeldt "iets te groot" af op "verhoudingsgewijs veel te klein".

Equivalente problemen hoeven geen equivalent benaderingsgedrag te hebben, en dit is daarvan het netste voorbeeld in het boek.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import petersen, cycle, complete
from graphs.approx import max_independent_set, min_vertex_cover, max_clique
for name, g in [('C5', cycle(5)), ('K4', complete(4)), ('petersen', petersen())]:
    a = max_independent_set(g); t = min_vertex_cover(g); w = max_clique(g)
    print(f'{name:<9} alpha={len(a)} tau={len(t)} sum={len(a)+len(t)} n={g.n} '
          f'omega={len(w)} omega(complement)={len(max_clique(g.complement()))}')
"
```


C5	alpha=2	tau=3	sum=5	n=5	omega=2	omega(complement)=2
K4	alpha=1	tau=3	sum=4	n=4	omega=4	omega(complement)=1
petersen	alpha=4	tau=6	sum=10	n=10	omega=2	omega(complement)=4

Elke rij heeft $\alpha + \tau = n$, en elke $\omega(\bar{G})$ komt overeen met de α van dezelfde graaf.

Oefeningen

1. Bereken α , τ en ω voor C_6 en controleer de identiteit van Gallai.
2. Toon aan dat $\omega(G) \cdot \alpha(G) \geq n$ in het algemeen onwaar is, en vind het kleinste tegenvoorbeeld.
3. Is G bipartiet, wat is $\tau(G)$ dan uitgedrukt in zijn maximale koppeling? (Hoofdstuk 14.)
4. Leg uit waarom een ijl graaf complementeren een kliekalgoritme trager maakt, ook al is de reductie van lineaire tijd.

Oplossingen in Bijlage E.

Kernpunten

- Kliek, onafhankelijke verzameling en knopenoverdekking zijn één probleem in drie notaties: $\omega(G) = \alpha(\bar{G})$ en $\alpha(G) + \tau(G) = n$.
- Los er één op en je hebt alle drie exact opgelost. Dat snijdt aan twee kanten — geen van drieën kan eenvoudig zijn.
- De reductie is van lineaire tijd maar niet gratis: een ijl graaf complementeren geeft een dichte, en algoritmen waarvan de kosten van m afhangen lijden eronder. Bitsetrepresentaties (hoofdstuk 2) zijn het standaardantwoord.
- Equivalentie van exacte problemen draagt **niet** over op benadering. Knopenoverdekking heeft een 2-benadering; onafhankelijke verzameling in wezen geen.

Hoofdstuk 22 — NP-moeilijkheid

Aan verschillende problemen in dit boek is "en dit is NP-moeilijk" gehangen zonder verantwoording. Dit hoofdstuk maakt de bewering precies en toont de techniek, op de grafenproblemen die we al in handen hebben.

Wat de bewering betekent

Een beslissingsprobleem zit in NP wanneer een ja-geval een certificaat heeft dat in polynomiale tijd controleerbaar is. "Heeft G een onafhankelijke verzameling van grootte k ?" zit in NP: overhandig de verzameling, en controleren kost $O(k^2)$.

Een probleem is NP-moeilijk wanneer elk probleem in NP er in polynomiale tijd toe herleid kan worden, en NP-volledig wanneer het allebei is. Een reductie van A naar B is een polynomiale afbeelding f met $x \in A \Leftrightarrow f(x) \in B$, en ze betekent dat B minstens zo moeilijk is als A .

Twee dingen worden vaak door elkaar gehaald. NP-moeilijk betekent niet "niet in P" — dat is juist de open vraag. En het betekent niet "moeilijk in de praktijk": SAT-oplossers verwerken routineus gevallen met miljoenen variabelen. NP-moeilijkheid is een uitspraak over slechtste gevallen over alle invoeren, wat een veel zwakkere bewering over jouw invoer is dan het klinkt.

Het reductiepatroon

De richting die iedereen in verwarring brengt: om te tonen dat B moeilijk is, herleid je een bekend moeilijke A naar B . Je toont dat B oplossen je A zou laten oplossen, dus B erft de moeilijkheid van A . B naar A herleiden toont niets over B .

Elke reductie heeft drie verplichtingen, en de derde overslaan is de standaardfout:

1. f loopt in polynomiale tijd;
2. $x \in A \Rightarrow f(x) \in B$;
3. $f(x) \in B \Rightarrow x \in A$.

De derde is waar reducties werkelijk sneuvelen. Het is eenvoudig om een gadget te bouwen dat oplossingen in oplossingen omzet en te vergeten oplossingen van $f(x)$ uit te sluiten die nergens vandaan kwamen.

De reducties die we al hebben

De identiteiten van hoofdstuk 21 zijn reducties, en ongewoon eenvoudige:

```
def clique_to_independent_set(g):
    return g.complement()           # omega(G) = alpha(complement)

def independent_set_to_vertex_cover(g, independent):
    return set(g.vertices()) - independent # alpha(G) + tau(G) = n
```

Beide zijn $O(n^2)$ en beide behouden optima in elke richting, wat verplichting 3 naar behoren uitvoert. De verificatie controleert het:

held ch22 The complement of a maximum independent set is a minimum vertex cover (52 graphs)

Let op wat die controle wel en niet vaststelt. Ze bevestigt dat de *reductie correct is* — dat de afbeelding optima werkelijk op optima afbeeldt. Ze zegt niets over NP-moeilijkheid, wat niet het soort uitspraak is dat een eindige controle kan behandelen.

3-SAT naar onafhankelijke verzameling

Het klassieke startpunt is 3-SAT, dat Cook en Levin rechtstreeks NP-volledig bewezen. Vanaf daar is al het overige reducties.

Gegeven een 3-CNF-formule met k clausules, bouw een graaf:

- voor elke clausule een **driehoek** waarvan de drie knopen haar literalen zijn;
- een kant tussen elke twee knopen met **tegenstrijdige** literalen (x en $\neg x$).

Dan is de formule vervulbaar **dan en slechts dan als** de graaf een onafhankelijke verzameling van grootte k heeft.

Bewijs. ($\implies$) Gegeven een vervullende toekenning, kies per clausule één ware lokaal. Geen twee zijn tegenstrijdig, want ze zijn alle waar onder één toekenning, en geen twee zitten in dezelfde driehoek. Dat is een onafhankelijke verzameling van grootte k .

($\impliedby$) Gegeven een onafhankelijke verzameling van grootte k : de driehoeken dwingen hoogstens één knoop per clausule af, dus precies één uit elke. Geen twee gekozen literalen zijn tegenstrijdig, dus elke gekozen lokaal waar maken is consistent, en het vervult elke clausule. Variabelen die ongezet blijven mogen beide kanten op. ■

De twee gadgets doen elk precies één taak. De driehoek dwingt "hoogstens één per clausule" af; de tegenstrijdigheidskanten dwingen consistentie af. Die scheiding van verantwoordelijkheden is hoe een goede reductie eruitziet, en de ($\impliedby$)-richting is waar de driehoeken hun plaats verdienen.

Wat moeilijk is en wat niet

Probleem	Status
Kortste pad (niet-negatief)	P — hoofdstuk 10
Maximale stroom	P — hoofdstuk 13
Bipartiete koppeling	P — hoofdstuk 14
Algemene koppeling	P — Edmonds' bloesems, hoofdstuk 14
Vlakheid	P, zelfs $O(n)$ — hoofdstuk 17
2-kleuring	P — hoofdstuk 16
3-kleuring	NP-volledig
Onafhankelijke verzameling, kliek, knopenoverdekking	NP-volledig — hoofdstuk 21
Hamiltoniaanse cykel	NP-volledig — hoofdstuk 20
Grafenisomorfie	in NP, geen van beide bekend — hoofdstuk 5

De grens is scherp en vaak verrassend. Twee kleuren eenvoudig, drie moeilijk. Euler-circuit eenvoudig, Hamiltoniaanse cykel moeilijk. Koppeling eenvoudig, onafhankelijke verzameling moeilijk. In elk paar zien de twee problemen er vergelijkbaar moeilijk uit en zijn ze het niet, en daarom is "dit ziet er moeilijk uit" nooit een argument.

Probeer het

Kijk hoe de reductie een optimum overdraagt, in beide richtingen:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import petersen, cycle
from graphs.approx import (max_clique, max_independent_set, min_vertex_cover,
                           clique_to_independent_set,
                           independent_set_to_vertex_cover,
                           is_vertex_cover)
for name, g in [('C5', cycle(5)), ('petersen', petersen())]:
    ind = max_independent_set(g)
    cover = independent_set_to_vertex_cover(g, ind)
    print(f'{name}: alpha={len(ind)} -> cover size {len(cover)},
    valid={is_vertex_cover(g, cover)}, '
          f'optimal={len(cover) == len(min_vertex_cover(g))}')
    print(f'    omega(G)={len(max_clique(g))} equals
    alpha(complement)={len(max_independent_set(clique_to_independent_set(g)))}')
"
```

```
C5: alpha=2 -> cover size 3, valid=True, optimal=True
    omega(G)=2 equals alpha(complement)=2
petersen: alpha=4 -> cover size 6, valid=True, optimal=True
    omega(G)=2 equals alpha(complement)=2
```

Oefeningen

1. Om te bewijzen dat probleem **B** NP-moeilijk is, herleid je **B** naar een bekend moeilijk probleem of andersom? Leg uit waarom de andere richting niets bewijst.
2. Wat gaat er in de 3-SAT-reductie mis als je de tegenstrijdigheidskanten weglaat?
3. Wat gaat er mis als je de driehoeken weglaat?
4. Knopenoverdekking is NP-volledig in het algemeen maar eenvoudig op bipartiete grafen (hoofdstuk 14). Is dat in tegenspraak met NP-moeilijkheid?

Oplossingen in Bijlage E.

Kernpunten

- Om te bewijzen dat **B** moeilijk is, herleid je een bekend moeilijke **A** **naar** **B**. De andere richting bewijst niets.
- Een reductie heeft drie verplichtingen; de omgekeerde implicatie is degene die in de praktijk sneuvelt.
- De reductie van 3-SAT naar onafhankelijke verzameling gebruikt twee gadgets met elk één taak: driehoeken voor "één lokaal per clausule", tegenstrijdigheidskanten voor consistentie.
- Een reductie op voorbeelden controleren bevestigt dat ze optima juist afbeeldt. Ze kan NP-moeilijkheid niet bevestigen, want dat is niet dat soort uitspraak.
- De grens tussen **P** en NP-volledig volgt niet de schijnbare moeilijkheid: 2-kleuring eenvoudig, 3-kleuring moeilijk; Euler eenvoudig, Hamilton moeilijk; koppeling eenvoudig, onafhankelijke verzameling moeilijk.

Hoofdstuk 23 — Leven met hardheid

NP-moeilijk is niet het einde van het gesprek. Er zijn drie eerlijke antwoorden — een benaderend antwoord aanvaarden met een *bewezen* factor, exponentiële tijd aanvaarden in een parameter die klein blijft, of je beperken tot een grafenklasse waar het probleem eenvoudig is — en dit hoofdstuk gaat over de keuze ertussen.

Benaderen met garantie

Een algoritme is een ρ -**benadering** voor een minimalisatieprobleem wanneer het altijd een oplossing teruggeeft met kosten hoogstens $\rho \cdot \text{OPT}$. Het woord dat telt is *altijd*: een factor is een bewezen grens voor het slechtste geval, geen waargenomen gemiddelde.

Het standaardvoorbeeld is prachtig grof.

Stelling. Beide eindpunten nemen van elke kant van een **maximale** koppeling geeft een knopenoverdekking van grootte hoogstens $2 \cdot \text{OPT}$.

```
def vertex_cover_2approx(g):
    cover = set()
    for u, v in g.edges():
        if u not in cover and v not in cover:
            cover.add(u); cover.add(v)
    return cover
```

Bewijs. Zij M de gebouwde koppeling. Haar kanten zijn paarsgewijs disjunct, dus elke knopenoverdekking heeft minstens één knoop uit elk nodig: $\text{OPT} \geq |M|$. Het algoritme geeft $2|M|$ terug. Bovendien is M *maximaal* — er kan geen kant bij — dus elke kant van G raakt een gekoppelde knoop, en het resultaat is werkelijk een overdekking. ■

Twee dingen over dit bewijs. Het berekent OPT nooit, het begrenst het alleen van onderaf; zo werkt elk benaderingsbewijs. En het heeft alleen een **maximale** koppeling nodig, geen **maximum** koppeling — gulzig volstaat, dus het algoritme is van lineaire tijd.

held ch23 The matching heuristic returns a cover at most twice optimal (47 graphs)

De heuristiek die er beter uit ziet en slechter is

De voor de hand liggende verbetering is gulzig zijn over graad: neem herhaaldelijk de knoop die de meeste resterende kanten overdekt. Het is meer werk, het gebruikt meer informatie, en het is intuïtief slimmer.

Het heeft helemaal geen constante benaderingsfactor. Het is $\Theta(\log n)$ in het slechtste geval.

Dit zien vergt een echt geval. Neem een linkerzijde L van grootte k , en voeg voor elke $i = 2..k$ $\lfloor k/i \rfloor$ rechterknoten toe die elk met i verschillende linkerknoten verbonden zijn. L is een overdekking, dus $\text{OPT} \leq k$. Gulzig neemt eerst de rechterknoten van hoge graad en verbruikt uiteindelijk de hele rechterzijde, ongeveer $k \ln k$ knopen.

k=8	n=20	m=48	OPT<=8	greedy=12	ratio>=1.50	matching=16	ratio<=2.00
k=12	n=35	m=115	OPT<=12	greedy=23	ratio>=1.92	matching=24	ratio<=2.00
k=16	n=50	m=204	OPT<=16	greedy=34	ratio>=2.12	matching=32	ratio<=2.00
k=20	n=66	m=319	OPT<=20	greedy=46	ratio>=2.30	matching=40	ratio<=2.00

Lees de rij `k=16` twee keer. Gulzig geeft **34**; de grove koppelingsheuristiek geeft **32**. Het algoritme dat meer informatie gebruikt doet het slechter, op een geval dat gebouwd is om precies dat bloot te leggen, en het wordt slechter naarmate `k` groeit.

Lees nu de eerste twee rijen. Bij `k = 8` en `k = 12` wint gulzig. **Je hebt een geval van vijftig knopen nodig voordat het verschil überhaupt verschijnt**, en niets kleiner geeft er een hint van. Wie deze twee heuristieken op kleine grafen vergelijkt, concludeert dat gulzig beter is en levert het op.

```
refuted   ch23 The max-degree heuristic is at most twice optimal (3 graphs)
```

Daarom telt de garantie zwaarder dan de metingen. De 2 van de koppelingsheuristiek is een stelling; de schijnbare superioriteit van gulzig was een steekproefartefact.

Let ook op hoe de factor wordt vastgesteld. `OPT` berekenen op een graaf van vijftig knopen is onhaalbaar, maar `L` is per constructie een overdekking van grootte `k`, dus $OPT \leq k$ en $|gulzig| / k$ is een strikte **ondergrens** voor de werkelijke factor. Een toelaatbare oplossing aanwijzen om het optimum te begrenzen is de standaardzet, en het is dezelfde die het 2-benaderingsbewijs in de andere richting maakt.

Parameterhardheid

Het tweede antwoord: houd exactheid vast, en beperk het exponentiële tot een parameter.

Knopenoverdekking van grootte hoogstens `k` is oplosbaar in $O(2^k \cdot (n + m))$. Kies een onoverdekte kant; een van haar eindpunten moet in de overdekking zitten, dus vertak op de twee keuzes met `k - 1`. De recursiediepte is `k`, dus de boom heeft 2^k bladeren — onafhankelijk van `n`.

```
def vertex_cover_at_most_k(g, k):
    uncovered = next(((u, v) for u, v in g.edges()), None)
    if uncovered is None:
        return set()
    if k <= 0:
        return None
    u, v = uncovered
    for choice in (u, v):
        smaller = Graph(g.n, [e for e in g.edges() if choice not in e])
        rest = vertex_cover_at_most_k(smaller, k - 1)
        if rest is not None:
            return {choice} | rest
    return None
```

Vergelijk met brute kracht over $C(n, k)$ deelverzamelingen. Voor `n = 106` en `k = 10` is dat 10^{60} tegen 1024. Een probleem dat zo'n algoritme toelaat is **vast-parameter hanteerbaar** (FPT), en het onderscheid tussen $f(k) \cdot \text{poly}(n)$ en $n^{f(k)}$ is het hele vakgebied.

```
held      ch23 Bounded search finds a cover of size <= k exactly when one
exists    (52 graphs)
```

Niet elk probleem werkt mee. Onafhankelijke verzameling geparametriseerd naar oplossingsgrootte is $W[1]$ -moeilijk, de geparametriseerde tegenhanger van NP-moeilijk — dus de equivalentie van hoofdstuk 21 breekt ook hier. Knopenoverdekking is FPT; zijn exacte tweelingbroer onder complementeren niet.

De invoer beperken

Het derde antwoord, en vaak het beste: merk op dat jouw grafen niet willekeurig zijn.

Beperking	Wat eenvoudig wordt
Bipartiet	knopenoverdekking, onafhankelijke verzameling, koppeling (h. 14, 16)
Koordaal / perfect	kleuring, kliek, onafhankelijke verzameling (h. 19)
Vlak	4-kleuring gratis; PTAS voor veel problemen (h. 17)
Begrensde boombreedte	vrijwel alles, via dynamisch programmeren (h. 31)
Begrensde graad	benaderingsfactoren verbeteren

Daarom besteedde deel IV vijf hoofdstukken aan structuur. Het praktische antwoord op "dit probleem is NP-moeilijk" is heel vaak "ja, maar mijn grafen zijn vlak", en de klassen kennen is wat je dat laat opmerken.

Wat niet kan

Eerlijkheid vergt ook de negatieve resultaten:

- **Knopenoverdekking** is niet beter dan $2 - \epsilon$ te benaderen onder het unieke-spellenvermoeden, en niet beter dan 1,36 tenzij $P = NP$. Het grove algoritme hierboven is in wezen optimaal.
- **Onafhankelijke verzameling en kliek** zijn niet binnen $n^{1-\epsilon}$ te benaderen tenzij $P = NP$.
- **Kleuring** evenmin binnen $n^{1-\epsilon}$.
- **Metrische TSP** heeft een $3/2$ -benadering (Christofides); algemene TSP heeft er geen, want elke benadering met constante factor zou Hamiltoniciteit beslissen.

De 2-benadering voor knopenoverdekking is dus geen tussenoplossing die op verbetering wacht. Ze is, onder standaardaanname, het eindpunt.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.approx import (greedy_lower_bound_instance, greedy_max_degree_cover,
                           vertex_cover_2approx, is_vertex_cover)
for k in (8, 12, 16, 20):
    g, known = greedy_lower_bound_instance(k)
    gr, mm = greedy_max_degree_cover(g), vertex_cover_2approx(g)
    assert is_vertex_cover(g, gr) and is_vertex_cover(g, mm)
    print(f'k={k:3} n={g.n:3} OPT<={k:3} greedy={len(gr):3} (ratio>={len(gr)/
k:.2f}) '
          f'matching={len(mm):3} (ratio<={len(mm)/k:.2f})')
"
```


k=8	n=20	OPT≤8	greedy=12	(ratio>=1.50)	matching=16	(ratio≤2.00)
k=12	n=35	OPT≤12	greedy=23	(ratio>=1.92)	matching=24	(ratio≤2.00)
k=16	n=50	OPT≤16	greedy=34	(ratio>=2.12)	matching=32	(ratio≤2.00)
k=20	n=66	OPT≤20	greedy=46	(ratio>=2.30)	matching=40	(ratio≤2.00)

De koppelingskolom komt nooit boven 2,00, want dat kan ze niet. De gulzige kolom passeert haar bij $k = 16$ en blijft klimmen.

Oefeningen

1. Bewijs dat de koppelingsheuristiek een overdekking van grootte precies $2|M|$ teruggeeft, en dat de maximaliteit van M is wat er een overdekking van maakt.
2. Geef een graaf waarop de koppelingsheuristiek precies $2 \cdot \text{OPT}$ teruggeeft.
3. Waarom verslaat $O(2^k \cdot (n+m))$ de $O(n^k)$ voor grote n en kleine k ? Geef getallen.
4. Knopenoverdekking is FPT maar onafhankelijke verzameling is $W[1]$ -moeilijk, ook al toonde hoofdstuk 21 dat de problemen equivalent zijn. Leg uit hoe beide waar kunnen zijn.

Oplossingen in Bijlage E.

Kernpunten

- Een p -benadering is een bewezen grens voor het slechtste geval. Benaderingsbewijzen berekenen OPT nooit; ze begrenzen het, doorgaans door een toelaatbare oplossing of een disjuncte structuur aan te wijzen.
- De knopenoverdekking uit een maximale koppeling is een 2-benadering, van lineaire tijd, en onder standaardaanname in wezen optimaal.
- De maximale-graadheuristiek ziet er slimmer uit en heeft geen constante factor. Ze verslaat de koppelingsheuristiek op grafen tot ongeveer 35 knopen en verliest vanaf 50 — dus testen op kleine gevallen geeft precies het verkeerde antwoord.
- Vast-parameter hanteerbaarheid beperkt het exponentiële tot $k: 2^k \cdot (n+m)$ in plaats van n^k . Knopenoverdekking is FPT; onafhankelijke verzameling is $W[1]$ -moeilijk ondanks hetzelfde probleem te zijn.
- De invoerklasse beperken is vaak het beste van de drie antwoorden, en daarvoor diende deel IV.

Hoofdstuk 24 — De probabilistische methode

Dit is het scharnier van het boek. Elk bewijs tot hier was constructief: het bouwde het object, of gaf een algoritme dat het zou bouwen. Vanaf hier houden de argumenten daarmee op.

De methode, van Erdős, is één zin: **om te bewijzen dat een object bestaat, leg je een kansverdeling op een ruimte die het bevat en toon je dat de kans op een geschikt object positief is.** Je toont er nooit een. Je toont dat er een moet zijn.

Het kleinst mogelijke voorbeeld

Stelling. Elke graaf heeft een bipartiete deelgraaf met minstens $m/2$ kanten.

Bewijs. Kleur elke knoop onafhankelijk rood of blauw met kans $1/2$. Een kant steekt over wanneer haar eindpunten verschillen, wat met kans $1/2$ gebeurt. Per lineariteit van de verwachting is het verwachte aantal overstekende kanten $m/2$.

Een stochastische variabele is met positieve kans minstens haar verwachting. Er is dus een kleuring die minstens $m/2$ overstekende kanten haalt, en de overstekende kanten vormen een bipartiete deelgraaf. ■

Lees dat nog eens over voor wat het níet doet. Het vertelt je niet welke kleuring. Het geeft geen algoritme. Het beweert dat er onder 2^n kleuringen minstens één goede zit, op grond van het feit dat het gemiddelde goed is.

Lineariteit van de verwachting doet al het werk, en het is het meest gebruikte instrument van de methode juist omdat het geen onafhankelijkheid nodig heeft. De kanten hier zijn *niet* onafhankelijk — twee kanten die een knoop delen zijn gecorreleerd — en dat maakt niets uit.

held ch24 Every graph has a bipartite subgraph with at least $m/2$ edges (52 graphs)

Ontrandomiseren

Het bewijs is niet-constructief, maar dit bepaalde bewijs valt te repareren. Loop de knopen in volgorde af, en plaats elke knoop aan de zijde die tot dan toe de meeste overstekende kanten geeft:

```
def greedy_cut(g):
    side = {}
    for v in g.vertices():
        zero = sum(1 for w in g.neighbours(v) if side.get(w) == 0)
        one = sum(1 for w in g.neighbours(v) if side.get(w) == 1)
        side[v] = 1 if zero >= one else 0
    return sum(1 for u, v in g.edges() if side[u] != side[v])
```

Dit is de **methode van de voorwaardelijke verwachtingen**: kies bij elke stap de optie waarvan de voorwaardelijke verwachting minstens de huidige verwachting is. Omdat de verwachting bij $m/2$ begint en nooit daalt, is het uiteindelijke deterministische antwoord minstens $m/2$.

Niet elk probabilistisch bewijs laat zich zo netjes ontrandomiseren, en die welke dat niet doen zijn de interessante.

Ondergrenzen voor Ramsey

De klassieke toepassing, en degene die de werkelijke kracht van de methode toont.

Stelling (Erdős, 1947). Geldt $C(n, k) \cdot 2^{1-C(k, 2)} < 1$, dan bestaat er een 2-kleuring van K_n zonder monochromatische K_k . Dus $R(k, k) > 2^{k/2}$ voor $k \geq 3$.

Bewijs. Kleur elke kant onafhankelijk willekeurig rood of blauw. Voor een vaste verzameling van k knopen is de kans dat ze monochromatisch is $2 \cdot 2^{-C(k, 2)}$. Er zijn $C(n, k)$ zulke verzamelingen, dus per uniegrems is de kans dat *een of andere* verzameling monochromatisch is hoogstens $C(n, k) \cdot 2^{1-C(k, 2)}$.

Is dat kleiner dan 1, dan is de kans op geen monochromatische verzameling positief, dus zo'n kleuring bestaat.

■

Hier verdient de methode haar reputatie. De grens $R(k, k) > 2^{k/2}$ was in 1947 een dramatische verbetering op alles wat bekend was — en **niemand heeft ooit een kleuring geconstrueerd** die er ook maar in de buurt komt. Zestig jaar inspanning heeft expliciete constructies opgeleverd die veel zwakker zijn dan wat een alinea middelen geeft. De kloof tussen wat we kunnen bewijzen te bestaan en wat we kunnen bouwen is bij dit probleem enorm en ziet er permanent uit.

Hoofdstuk 28 geeft de bovengrens en de daaruit volgende staat van onwetendheid over $R(5, 5)$.

De drie instrumenten

Vrijwel elk argument in deze stijl gebruikt een van drie ideeën.

Lineariteit van de verwachting. $E[X + Y] = E[X] + E[Y]$, altijd, zonder enige onafhankelijkheidsaannname. Hierboven gebruikt voor de snede.

De uniegrems. $P(\cup A_i) \leq \sum P(A_i)$, altijd. Hierboven gebruikt voor Ramsey. Grof, en meestal voldoende.

De verwijderingsmethode. Toon dat een willekeurig object *gemiddeld* weinig slechte delen heeft, en verwijder dan één knoop per slecht deel. Wat overblijft is goed en nog steeds groot. Dat geeft bijvoorbeeld dat een graaf met n knopen en m kanten een onafhankelijke verzameling van grootte minstens $n^2/(4m)$ heeft — neem elke knoop met kans p , verwijder één eindpunt van elke overlevende kant, en optimaliseer p .

Wanneer deze drie falen, wordt de machinerie veel zwaarder — het lokale lemma van Lovász, martingaalconcentratie, de tweedemomentmethode — en hoofdstuk 26 heeft die laatste nodig.

Wat er vanaf hier verandert

De rest van deel VI is in deze stijl geschreven, en het loont de verschuiving expliciet te maken.

De verificatie kan niet op dezelfde manier helpen als eerder. Een bewering als "er bestaat een kleuring zonder monochromatische K_k " is alleen controleerbaar door alle kleuringen af te zoeken, en dat is precies wat de methode wil vermijden. Hoofdstuk 28 controleert $R(3, 3) = 6$ uitputtend omdat 32.768 kleuringen klein is;

$R(5, 5)$ betreft meer kleuringen dan er atomen in het waarneembare heelal zijn, en geen enkele hoeveelheid rekenkracht beslist het.

Vanaf hier zijn de beweringen van het boek dus controleerbaar bij kleine n terwijl de stellingen over grote n gaan, en het gat daartussen is een gat dat de verificatie niet kan dichten. De hoofdstukken 25 en 26 gaan daarmee om door *scherpte* te meten in plaats van waarheid — tonen dat de overgang smal is, wat een eindig feit is — en door ronduit te zeggen dat een limietuitspraak niet is wat er getest wordt.

Probeer het

```
python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.core import petersen, complete
from graphs.extremal import max_cut_bruteforce, greedy_cut, random_bipartition_cut
rng = random.Random(4)
for name, g in [('K5', complete(5)), ('petersen', petersen())]:
    trials = [random_bipartition_cut(g, rng) for _ in range(1000)]
    print(f'{name}: m={g.m}  m/2={g.m/2}')
    print(f'    random cut, mean over 1000 trials: {sum(trials)/1000:.2f}')
    print(f'    greedy (derandomised):                {greedy_cut(g)}')
    print(f'    true maximum:                            {max_cut_bruteforce(g)}')
```

```
K5: m=10  m/2=5.0
    random cut, mean over 1000 trials: 5.01
    greedy (derandomised):            6
    true maximum:                    6
petersen: m=15  m/2=7.5
    random cut, mean over 1000 trials: 7.41
    greedy (derandomised):            12
    true maximum:                    12
```

De willekeurige gemiddelden landen dicht bij $m/2$ — 5,01 tegen 5,0, en 7,41 tegen 7,5 — precies zoals lineariteit van de verwachting voorspelt. Het ontrandomiseerde gulzige algoritme overtreft de grens ruim en haalt hier op beide grafen het werkelijke maximum. Dat is normaal en het is geen garantie: de stelling belooft $m/2$ en niets meer, en een heuristiek die het meestal beter doet is precies de situatie waar hoofdstuk 23 voor waarschuwde.

Oefeningen

1. Bewijs in je eigen woorden dat elke graaf een bipartiete deelgraaf met minstens $m/2$ kanten heeft, en zeg waar onafhankelijkheid nodig geweest zou zijn als je haar gebruikt had.
2. Een toernooi is een volledige graaf waarin elke kant gericht is. Toon aan dat een of ander toernooi op n knopen minstens $n!/2^{n-1}$ Hamiltoniaanse paden heeft.
3. Gebruik de verwijderingsmethode om te tonen dat een graaf met n knopen en $m \geq n/2$ kanten een onafhankelijke verzameling van grootte minstens $n^2/(4m)$ heeft.
4. Waarom laat de $m/2$ -snedegrens zich eenvoudig ontrandomiseren en de Ramsey-ondergrens niet?

Oplossingen in Bijlage E.

Kernpunten

- Om bestaan te bewijzen, toon je dat een willekeurig object met positieve kans werkt. Je bouwt er nooit een.
- Lineariteit van de verwachting heeft geen onafhankelijkheid nodig, en daarom is ze het werkpaard.
- De uniegrens is grof en meestal voldoende; de verwijderingsmethode behandelt "grotendeels goede" objecten.
- $R(k, k) > 2^{\{k/2\}}$ volgt uit een alinea middelen, en geen expliciete constructie is ooit in de buurt gekomen. Die kloof is de duidelijkste demonstratie van de methode.
- Vanaf hier zijn de stellingen asymptotisch en de verificatie eindig. Het boek meet scherpte, en doet niet alsof dat hetzelfde is als bewijs.

Hoofdstuk 25 — Toevalsgrafen

$G(n, p)$ is de graaf op n knopen waarin elk van de $C(n, 2)$ mogelijke kanten onafhankelijk met kans p verschijnt. Het inzicht van Erdős en Rényi was dat dit niet slechts een bron van testgevallen is — er zit een rijke theorie in, en vrijwel elke eigenschap verschijnt *plotseling* naarmate p groeit.

Drempels

Een eigenschap is **monotoon stijgend** wanneer kanten toevoegen haar niet kan vernietigen: samenhang, een driehoek hebben, een Hamiltoniaanse cykel bevatten.

Stelling (Bollobás–Thomason, 1987). Elke monotoon stijgende eigenschap heeft een drempelfunctie $p^*(n)$: is $p \ll p^*$ dan geldt de eigenschap met kans naar 0, en is $p \gg p^*$ dan met kans naar 1.

Elke monotone eigenschap heeft er een. De vraag is altijd *waar*, nooit *of*.

Eigenschap	Drempel
Bevat een driehoek	$1/n$
Bevat een vaste H (gebalanceerd)	$n^{-v(H)/e(H)}$
Reuzencomponent verschijnt	$1/n$ (hoofdstuk 26)
Geen geïsoleerde knopen	$\ln n / n$
Samenhangend	$\ln n / n$
Hamiltoniaans	$\ln n / n$

Dat de laatste drie een drempel delen is geen toeval, en het is de leerzaamste rij in de tabel. Onder $\ln n / n$ heeft een toevalsgraaf geïsoleerde knopen; dat alleen al maakt hem onsamenhangend en niet-Hamiltoniaans. Erboven verdwijnen de geïsoleerde knopen — en het blijkt dat *niets anders* ooit de belemmering was. **Het moment waarop de laatste geïsoleerde knoop verdwijnt is in wezen het moment waarop de graaf Hamiltoniaans wordt**, en dat is een veel sterkere uitspraak dan samenhang en komt er gratis bij.

De eerstementmethode

Het verwachte aantal driehoeken in $G(n, p)$ is $C(n, 3) p^3 \approx n^3 p^3 / 6$.

Is $p \ll 1/n$, dan gaat dit naar 0, en omdat het aantal driehoeken een niet-negatief geheel getal is, geeft de ongelijkheid van Markov $P(\text{minstens één driehoek}) \leq E[\text{aantal}] \rightarrow 0$. Dat is de **eerstemomentmethode**: een verwachting die naar nul gaat bewijst dat het object er meestal niet is.

Het omgekeerde vergt meer. Een verwachting die naar *oneindig* gaat bewijst geen bestaan — een variabele kan een enorm gemiddelde hebben en toch bijna altijd nul zijn, als ze af en toe reusachtig is. Dat uitsluiten vergt de variantie, en dat is de **tweedementmethode**, die hoofdstuk 26 gebruikt.

Die asymmetrie is het eigen maken waard: **eerste moment voor niet-bestaan, tweede moment voor bestaan**. Ze omdraaien is de standaardfout in dit vak.

Scherpte meten

Een drempel is een asymptotische uitspraak, dus geen enkel eindig experiment kan haar bevestigen. Wat een experiment *wel* kan tonen, is dat de overgang smal is — een feit over $n = 400$ in plaats van over de limiet.

```
python scripts/random_graph_experiments.py
```

Connectivity, $n = 400$, $p = c \ln(n)/n$, 40 trials per row

c	connected fraction
0.4	0.00
0.6	0.00
0.8	0.03
1.0	0.47
1.2	0.65
1.5	0.93
2.0	1.00

Bij $c = 0,6$ is in wezen niets samenhangend; bij $c = 2,0$ alles. De hele overgang vindt plaats binnen een factor van ongeveer drie in p , en bij $c = 1$ — precies de voorspelde drempel — is het bijna kop of munt.

Dit is geen bewijsmateriaal dat de drempel $\ln n / n$ is. Het is ermee verenigbaar, bij één waarde van n . De stelling gaat over $n \rightarrow \infty$, en één n kan $\ln n / n$ niet onderscheiden van elke functie die er bij 400 toevallig dicht bij ligt. Wat de tabel eerlijk toont is scherpste, en scherpste is het verrassende deel — een eigenschap die van nooit naar altijd gaat over een smalle band is niet wat de naïeve intuïtie voorspelt.

Vrijwel elke graaf

Resultaten over toevalsgrafen worden meestal geformuleerd als feiten over "vrijwel elke graaf", en de vertaling is het expliciet maken waard. $G(n, 1/2)$ is de uniforme verdeling over alle gelabelde grafen op n knopen, want elke graaf heeft kans $2^{-\binom{n}{2}}$. Een eigenschap die **met hoge waarschijnlijkheid** geldt in $G(n, 1/2)$ is dus een eigenschap van vrijwel elke graaf.

Verschillende zulke feiten zijn verbluffend:

- Vrijwel elke graaf heeft diameter 2.
- Vrijwel elke graaf heeft een triviale automorfismegroep — helemaal geen symmetrieën.
- Vrijwel elke graaf heeft klikgetal ongeveer $2 \log_2 n$, geconcentreerd op **twee waarden**.

Die laatste is opmerkelijk. $\omega(G)$ voor een toevalsgraaf ligt niet slechts in de buurt van $2 \log_2 n$; het is een van twee specifieke gehele getallen met kans naar 1. En toch is een klik van grootte $(1+\epsilon) \log_2 n$ vinden in een toevalsgraaf een beroemd open probleem — gulzig vindt $\log_2 n$ en niets doet het beter. **We weten precies hoe groot het antwoord is en kunnen het niet vinden**, wat dezelfde kloof is die hoofdstuk 24 voor Ramsey aanwees, in een andere vermomming.

Probeer het

```
python -c "
import sys, random, math; sys.path.insert(0, '.')
from graphs.generate import random_graph
from graphs.algorithms import distances
rng = random.Random(2)
n = 200
for p in (0.02, 0.05, 0.1, 0.3):
    diam = 0
    g = random_graph(n, p, rng)
    for v in g.vertices():
        d = distances(g, v)
        if len(d) == n:
            diam = max(diam, max(d.values()))
        else:
            diam = -1; break
    print(f'p={p:<5} m={g.m:<6} diameter={diam if diam > 0 else \"disconnected\"}')
"
```

```
p=0.02  m=407    diameter=disconnected
p=0.05  m=1006   diameter=4
p=0.1   m=1925   diameter=3
p=0.3   m=5970   diameter=2
```

Bij $p = 0,3$ is de diameter al 2, en hij blijft daar — het resultaat "vrijwel elke graaf heeft diameter 2" arriveert ruim vóór $p = 1/2$.

Oefeningen

1. Bereken het verwachte aantal driehoeken in $G(n, p)$ en vind de drempel waar het ophoudt naar nul te gaan.
2. Waarom bewijst een verwacht aantal kopieën van H dat naar oneindig gaat niet dat H verschijnt? Geef de vorm van een tegenvoorbeeld.
3. Toon aan dat het verwachte aantal geïsoleerde knopen in $G(n, p)$ gelijk is aan $n(1-p)^{n-1}$, en vind waar het naar een constante gaat.
4. $G(n, 1/2)$ is uniform over gelabelde grafen. Waarom maakt dat uitspraken "met hoge waarschijnlijkheid" tot uitspraken over vrijwel elke graaf?

Oplossingen in Bijlage E.

Kernpunten

- Elke monotoon stijgende eigenschap heeft een drempel; de vraag is waar die ligt.
- Samenhang, geen geïsoleerde knopen, en Hamiltoniciteit delen de drempel $\ln n / n$. Het verdwijnen van de laatste geïsoleerde knoop is in wezen het moment waarop de graaf Hamiltoniaans wordt.
- Eerste moment (verwachting $\rightarrow 0$) bewijst niet-bestaan. Bestaan vergt het tweede moment, want een groot gemiddelde kan een variabele verbergen die meestal nul is.

- Een eindig experiment kan een asymptotische drempel niet bevestigen. Het kan tonen dat de overgang scherp is, wat een andere en nog steeds verrassende bewering is, en het boek zegt welke van beide het toont.
- Vrijwel elke graaf heeft diameter 2, geen symmetrieën, en een klikgetal geconcentreerd op twee waarden — die we niet kunnen vinden.

Hoofdstuk 26 — De reuzencomponent

Bij $p = c/n$ gebeurt er iets abrupts met $G(n, p)$ zodra c de 1 passeert. Eronder is elke component klein. Erboven is er precies één enorme component en is al het overige nog steeds klein. Dit is de best bestudeerde faseovergang in de combinatoriek, en het loont te begrijpen zowel wat er gebeurt als waarom het mechanisme zo eenvoudig is.

De stelling

Stelling (Erdős–Rényi, 1960). Zij $p = c/n$.

- Is $c < 1$, dan heeft de grootste component grootte $O(\log n)$ met hoge waarschijnlijkheid.
- Is $c > 1$, dan is er een unieke component van grootte $\approx \beta n$, waarbij β de positieve wortel is van $\beta = 1 - e^{-c\beta}$, en elke andere component is $O(\log n)$.
- Is $c = 1$, dan heeft de grootste component grootte $\Theta(n^{2/3})$ — niet logaritmisch en niet lineair.

Drie regimes, en het derde is het vreemdste: precies op het kritieke punt is het antwoord een gebroken macht, en daarom wordt het kritieke venster apart en veel gedetailleerder bestudeerd dan beide zijden.

Waarom: het vertakkingsproces

Het mechanisme is eerst een heuristiek en pas daarna een bewijs, en de heuristiek is het deel om te onthouden.

Verken een component met BFS vanuit een knoop. Elke knoop die je bereikt heeft ongeveer n mogelijke burens, elk aanwezig met kans c/n , dus hij levert ongeveer c **nieuwe knopen** in verwachting. Vroeg in de verkenning is er vrijwel niets bezocht, dus het proces lijkt op een **vertakkingsproces met gemiddeld nageslacht c** .

De theorie van vertakkingsprocessen levert dan het antwoord, en het is een tweedeling:

- $c < 1$ (subkritiek): uitsterven met kans 1, en het verwachte totale nageslacht is $1/(1-c)$, een constante. Componenten zijn klein.
- $c > 1$ (superkritiek): overleven met kans $\beta > 0$, waarbij β voldoet aan $\beta = 1 - e^{-c\beta}$. Een positieve fractie van de startknopen brengt onbegrensde groei voort, en ze belanden allemaal in dezelfde component.
- $c = 1$ (kritiek): uitsterven met kans 1, maar de verwachte tijd tot uitsterven is oneindig. Daarom is het kritieke geval delicaat.

$c = 1$ is waar elke knoop zichzelf precies één keer vervangt. Eronder sterft de verkenning uit; erboven ontsnapt ze. Alles in dit hoofdstuk is die ene zin.

De vertakkingsbenadering is niet exact — de graaf heeft maar n knopen, dus het proces moet uiteindelijk opraken — en haar tot een bewijs maken vergt de tweedemomentmethode (hoofdstuk 25) om te tonen dat de grootte van de grote component geconcentreerd is, plus een apart argument voor uniciteit. Uniciteit heeft ook een aardige heuristiek: twee componenten van grootte ϵn hebben $\epsilon^2 n^2$ mogelijke kanten ertussen, en bij $p = c/n$ zouden ze met kans naar 1 verbonden zijn.

Het meten

```
python scripts/random_graph_experiments.py
```

Giant component, $n = 400$, 5 trials per row

$c = pn$	largest/ n	2nd/ n	#comps
0.4	0.014	0.013	317.6
0.6	0.035	0.024	280.0
0.8	0.052	0.039	237.0
0.9	0.096	0.070	202.4
1.0	0.142	0.044	199.4
1.1	0.217	0.066	176.4
1.2	0.324	0.056	157.6
1.5	0.581	0.023	116.4
2.0	0.793	0.011	65.6
3.0	0.937	0.006	24.2

Twee kolommen, en de tweede is degene om te lezen.

De grootste component groeit gestaag van 1,4% naar 94% van de graaf — maar dat is een geleidelijk ogende kromme, en bij $n = 400$ is de overgang werkelijk vervaagd. De stelling is asymptotisch; $\log n \approx 6$ en $n^{2/3} \approx 54$ en $n = 400$ liggen bij deze omvang eenvoudigweg niet ver genoeg uit elkaar om op drie regimes te lijken.

De op één na grootste component is het scherpe signaal. Ze stijgt tot 7% van de graaf bij $c = 0,9$, en zakt dan weg — 4,4% bij $c = 1,0$, 2,3% bij $c = 1,5$, 1,1% bij $c = 2,0$, 0,6% bij $c = 3,0$. Die niet-monotone piek nabij $c = 1$ is de overgang die zich zichtbaar maakt: onder de drempel zijn de twee grootste componenten vergelijkbaar, erboven loopt de grootste weg en laat al het overige achter.

Vergelijk de rij $c = 0,4$ — grootste 1,4%, tweede 1,3%, in wezen gelijk — met $c = 3,0$ — grootste 94%, tweede 0,6%. De *verhouding* tussen de eerste en de tweede is de ordeparameter, en die verandert van karakter, niet de grootte van de grootste alleen.

Bij $n = 400$ met 5 herhalingen zijn deze getallen ruisig, en het gaat om de vorm en niet om enig afzonderlijk cijfer. Een eindig experiment kan een asymptotische stelling niet bevestigen; wat het wel kan, is je tonen waar te kijken.

Waarom het buiten de grafentheorie telt

Dezelfde overgang, met hetzelfde $c = 1$ -mechanisme, beheerst:

- **Percolatie:** vloeistof die zich boven een kritieke dichtheid door poreus gesteente verspreidt.
- **Epidemieën:** het basisreproductiegetal R_0 is precies c , en $R_0 = 1$ is precies deze drempel. De reuzencomponent is de uitbraak.
- **Netwerkweerbaarheid:** hoeveel knopen kunnen uitvallen voordat een netwerk versplintert.
- **Willekeurige SAT:** vervulbaarheid heeft een analoge scherpe drempel in de verhouding tussen clausules en variabelen.

De bewering dat een epidemie ofwel uitdooft ofwel een constante fractie van de bevolking bereikt, met niets ertussenin, is deze stelling. De vertakkingsheuristiek — elk geval brengt R_0 nieuwe gevallen voort — is dezelfde, en het is waarom $R_0 = 1$ het getal is waar iedereen naar kijkt.

Probeer het

Kijk hoe de op één na grootste component piekt en terugvalt:

```
python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.generate import random_graph
from graphs.algorithms import components
rng = random.Random(11)
n, trials = 800, 15
for c in (0.5, 0.9, 1.0, 1.1, 1.5, 2.5):
    first = second = 0
    for _ in range(trials):
        s = sorted((len(x) for x in components(random_graph(n, c/n, rng))),
reverse=True)
        first += s[0]; second += s[1] if len(s) > 1 else 0
    a, b = first/trials, second/trials
    print(f'c={c:<4} largest={a:<7.1f} second={b:<6.1f} ratio={a/max(b,1):.1f}')
"
```

c=0.5	largest=11.2	second=8.4	ratio=1.3
c=0.9	largest=51.3	second=26.0	ratio=2.0
c=1.0	largest=106.5	second=29.9	ratio=3.6
c=1.1	largest=128.1	second=45.1	ratio=2.8
c=1.5	largest=470.1	second=11.9	ratio=39.6
c=2.5	largest=717.0	second=3.3	ratio=219.5

De verhouding is het verhaal: nabij 1 onder de drempel, en onbegrensd klimmend erboven.

Dit vergt middelen over vijftien herhalingen, en dat is zelf de bevinding. Eén run per rij levert een niet-monotone warboel — bij één poging gaf $c = 1,1$ een *lagere* verhouding dan $c = 1,0$. Dat is geen experimentele slordigheid; nabij het kritieke punt hebben de componentgroottes werkelijk een enorme variantie, en dat is juist waarom het geval $c = 1$ zijn eigen $\Theta(n^{\{2/3\}})$ -stelling en zijn eigen literatuur heeft. Zien je metingen bij een faseovergang er ruisig uit, dan kan de ruis het verschijnsel zijn.

Oefeningen

1. Los $\beta = 1 - e^{-c\beta}$ numeriek op voor $c = 1,5$ en vergelijk met de gemeten 0,581.
2. Waarom breekt de vertakkingsheuristiek af zodra de verkende verzameling een constante fractie van de graaf is?
3. Geef het argument dat twee reuzencomponenten niet naast elkaar kunnen bestaan.
4. Wat betekent $c < 1$ in epidemiologische termen, en waarmee komt de op één na grootste component overeen?

Oplossingen in Bijlage E.

Kernpunten

- Bij $p = c/n$: alle componenten $O(\log n)$ voor $c < 1$; een unieke component van βn voor $c > 1$; $\theta(n^{2/3})$ precies bij $c = 1$.
- Het mechanisme is een vertakkingsproces met gemiddeld nageslacht c . $c = 1$ is waar elke knoop zichzelf precies één keer vervangt — die ene zin is het hele hoofdstuk.
- β voldoet aan $\beta = 1 - e^{-c\beta}$, wat volgt uit de uitsterfkans van dat proces.
- Bij $n = 400$ ziet de groei van de grootste component er geleidelijk uit. De **verhouding tussen de grootste en de op één na grootste** is het scherpe signaal, en de piek van de tweede nabij $c = 1$ is de overgang die bij eindige omvang zichtbaar wordt.
- Dezelfde overgang is percolatie, $R_0 = 1$ bij epidemieën, en de drempel bij willekeurige SAT.

Hoofdstuk 27 — Extremale grafentheorie

Eén vraag, keer op keer gesteld: **hoeveel kanten kan een graaf hebben voordat een of andere structuur gedwongen wordt te verschijnen?** Hoofdstuk 16 toonde dat driehoekvrij niet bipartiet impliceert; dit hoofdstuk vraagt hoeveel kanten driehoekvrijheid je kost.

Mantel

Stelling (Mantel, 1907). Een driehoekvrije graaf op n knopen heeft hoogstens $n^2/4$ kanten, en de volledig bipartiete graaf $K_{\lfloor n/2 \rfloor, \lfloor n/2 \rfloor}$ is het unieke extremale voorbeeld.

Bewijs. Zij uv een kant. Omdat G driehoekvrij is, hebben u en v geen gemeenschappelijke buur, dus $\deg(u) + \deg(v) \leq n$. Sommeren over alle kanten geeft

$$\sum_{uv \in E} (\deg(u) + \deg(v)) \leq mn.$$

De linkerkant telt elke knoop v één keer per kant eraan, wat $\deg(v)^2$ bijdraagt. Dus $\sum_v \deg(v)^2 \leq mn$. Per Cauchy–Schwarz geldt $\sum \deg(v)^2 \geq (\sum \deg(v))^2/n = 4m^2/n$. Samengevoegd: $4m^2/n \leq mn$, dus $m \leq n^2/4$. ■

Twee technieken in vijf regels: **dubbeltelling** (hoofdstuk 3, vierde optreden) en **Cauchy–Schwarz**, de standaardmanier om een som van kwadraten in een grens om te zetten. Beide keren in dit deel van het boek terug.

held ch27 Mantel: a triangle-free graph has $m \leq n^2/4$ (27 graphs)

Turán

De veralgemening vervangt de driehoek door K_{r+1} .

Stelling (Turán, 1941). Een graaf zonder K_{r+1} heeft hoogstens $(1 - 1/r) n^2/2$ kanten, en de unieke extremale graaf is de **Turán-graaf** $T(n, r)$: de volledig r -delige graaf met zo gelijk mogelijke delen.

Mantel is het geval $r = 2$.

Het extremale voorbeeld is het bekijken waard. Om K_{r+1} te vermijden met zoveel mogelijk kanten, splits je de knopen in r groepen en verbind je alles *tussen* de groepen. Elke klik kiest hoogstens één knoop per groep, dus klikken hebben grootte hoogstens r . De delen gelijk maken maximaliseert het aantal kanten — een toepassing van dezelfde convexiteit die Cauchy–Schwarz codeert.

De verificatie controleert dit door **elke** graaf op n knopen op te sommen en het werkelijke maximum te vinden, in plaats van de formule te vertrouwen:

```
def max_edges_without_clique(n, k):
    pairs = list(itertools.combinations(range(n), 2))
    best = 0
    for mask in range(1 << len(pairs)):
        chosen = [p for i, p in enumerate(pairs) if mask >> i & 1]
        if len(chosen) > best and not has_clique(Graph(n, chosen), k):
            best = len(chosen)
    return best
```

```
held      ch27 Turan: the K_{r+1}-free maximum is exactly the Turan graph's edge
count (8 graphs)
```

$2^{\binom{n}{2}}$ grafen, dus dit stopt bij $n = 5$. Het volstaat om een verkeerd opgeschreven formule te vangen, en daarvoor dient het.

Erdős–Stone: het algemene antwoord

Turán behandelt volledige grafen. En als je een willekeurige H verbiedt?

Stelling (Erdős–Stone, 1946). Voor elke graaf H met chromatisch getal $\chi(H) = r + 1$ is het maximale aantal kanten in een H -vrije graaf op n knopen

$$(1 - 1/r) \frac{n^2}{2} + o(n^2).$$

Dit wordt soms de hoofdstelling van de extremale grafentheorie genoemd, en de reden is haar reikwijdte: **het antwoord hangt van H alleen af via zijn chromatisch getal**. Een driehoek verbieden en de Petersen-graaf verbieden geven dezelfde hoofdterm, want beide hebben $\chi = 3$. Elk detail van de structuur van H is bij deze resolutie onzichtbaar.

Er is één enorme uitzondering, en daar liggen de moeilijke open problemen van het vak. Is $\chi(H) = 2$ — dat wil zeggen, is H **bipartiet** — dan geeft de stelling $o(n^2)$ en zegt ze verder niets. De werkelijke orde bepalen voor bipartiete H is het **probleem van Zarankiewicz**, en dat is in het algemeen open. Zelfs $H = C_8$ is niet volledig opgelost. De bekende gevallen:

- $H = C_4$: het antwoord is $\frac{1}{2} n^{\frac{3}{2}}(1 + o(1))$, en de extremale grafen komen uit de eindige meetkunde.
- $H = K_{\{3,3\}}$: $\Theta(n^{\frac{5}{3}})$, met boven- en ondergrenzen die alleen op constanten na overeenkomen.

Extremale grafentheorie is dus in wezen opgelost voor niet-bipartiete H en in wezen open voor bipartiete H . Dat is een ongebruikelijke vorm voor een vakgebied, en het is goed te weten voordat je op zoek gaat naar open problemen.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.extremal import max_edges_without_clique, turan_bound, turan_graph
print(f\" {n':>3} {r':>3} {'exhaustive max':>15} {'Turan bound':>12} {'(1-1/
r)n^2/2':>14}\")
for n in range(3, 6):
    for r in (2, 3):
        exact = max_edges_without_clique(n, r + 1)
        print(f' {n:>3} {r:>3} {exact:>15} {turan_bound(n, r):>12} {(1 - 1/r) * n
* n / 2:>14.2f}')
"
```

n	r	exhaustive max	Turan bound	$(1-1/r)n^2/2$
3	2	2	2	2.25
3	3	3	3	3.00
4	2	4	4	4.00
4	3	5	5	5.33
5	2	6	6	6.25
5	3	8	8	8.33

Het uitputtende maximum komt elke keer exact overeen met het aantal kanten van de Turán-graaf. De laatste kolom — de nette formule $(1 - 1/r)n^2/2$ — is een *boven grens*, alleen scherp wanneer r een deler van n is. Bij $n = 4$, $r = 2$ is ze exact op 4; bij $n = 5$, $r = 2$ zegt ze 6,25 tegen een echte 6, en bij $n = 5$, $r = 3$ zegt ze 8,33 tegen een echte 8.

De verschillen zijn hier klein, en ze zijn de reden dat extremale resultaten geformuleerd worden in termen van het extremale *object* en niet van de afgeronde formule. Het aantal kanten van de Turán-graaf is de stelling; $(1 - 1/r)n^2/2$ is een handige asymptotiek die nooit kleiner en zelden gelijk is.

Oefeningen

1. Verifieer de grens van Mantel voor $n = 5$ door een driehoekvrije graaf met 6 kanten te vinden.
2. Waarom maximaliseert het gelijk maken van de delen van de Turán-graaf het aantal kanten? Gebruik convexiteit.
3. Erdős–Stone zegt dat het antwoord van H alleen afhangt via $\chi(H)$. Bepaal de chromatische getallen van K_3 , K_4 en de Petersen-graaf, en zeg welke twee van de drie dezelfde hoofdterm geven.
4. Waarom zegt Erdős–Stone niets bruikbaar wanneer H bipartiet is?

Oplossingen in Bijlage E.

Kernpunten

- Mantel: driehoekvrij impliceert $m \leq n^2/4$, extremaal bij de gebalanceerde volledig bipartiete graaf. Bewezen met dubbeltelling plus Cauchy–Schwarz.
- Turán: geen K_{r+1} impliceert $m \leq (1 - 1/r)n^2/2$, extremaal bij de gebalanceerde volledig r -delige graaf.

- De nette formule is een bovengrens; het exacte antwoord is het aantal kanten van de Turán-graaf, en de twee verschillen wanneer r geen deler van n is.
- Erdős–Stone: voor elke H hangt het antwoord alleen af van $\chi(H)$. Alle structuur voorbij het chromatisch getal is onzichtbaar.
- Is $\chi(H) = 2$, dan zwijgt de stelling, en dat zwijgen is het probleem van Zarankiewicz — waar de open vragen van het vak liggen.

Hoofdstuk 28 — Ramsey-theorie

Hoofdstuk 27 vroeg hoeveel kanten een structuur afdwingen. De Ramsey-theorie vraagt iets sterkers: hoe groot moet een graaf zijn voordat **elke** 2-kleuring van zijn kanten monochromatische orde bevat. Het antwoord is altijd "eindig", en vrijwel nooit "bekend".

$R(3,3) = 6$

Stelling. Elke 2-kleuring van de kanten van K_6 bevat een monochromatische driehoek, en een of andere 2-kleuring van K_5 doet dat niet.

Bewijs. Neem een willekeurige knoop v in K_6 . Hij heeft 5 kanten, dus per duivenhokprincipe delen er minstens 3 een kleur — zeg dat v rood verbonden is met a , b , c . Is een van ab , bc , ac rood, dan vormt die kant met v een rode driehoek. Is er geen rood, dan is abc een blauwe driehoek. ■

Dat is het hele bewijs, en het is de vriendelijkste stelling in dit deel van het boek: één duivenhokstap en één gevalsonderscheid. Het is ook het bekende "feestpuzzeltje" — onder elke zes mensen zijn er drie die elkaar allemaal kennen of drie die elkaar geen van allen kennen.

Voor de ondergrens: kleur K_5 rood op een vijfcykel en blauw op de complementaire vijfcykel. Geen van beide kleurklassen bevat een driehoek, want C_5 is driehoekvrij en zelfcomplementair.

Beide helften zijn uitputtend controleerbaar, en dit is een van de weinige plaatsen in deel VI waar de verificatie een echte vraag kan afdoen in plaats van een uitspraak steekproefsgewijs te controleren:

```
held      ch28  R(3,3) = 6: every 2-colouring of K_6 has a monochromatic
triangle  (3 graphs)
```

32.768 kleuringen van K_6 , allemaal gecontroleerd; 1.024 van K_5 , en een tegenvoorbeeld gevonden. Samen stellen die $R(3,3) = 6$ volledig vast. **Dit is een bewijs, geen steekproef** — de zoektocht is uitputtend over de volledige ruimte waarover de stelling kwantificeert, en het is de enige situatie in dit boek waar een eindige berekening een stelling rechtstreeks afdoet.

De algemene stelling

Stelling (Ramsey, 1930). Voor alle s , t bestaat er een kleinste $R(s,t)$ zodat elke 2-kleuring van $K_{R(s,t)}$ een rode K_s of een blauwe K_t bevat.

Het bestaansbewijs geeft $R(s,t) \leq R(s-1,t) + R(s,t-1)$, dus $R(s,s) \leq 4^s$. De probabilistische methode van hoofdstuk 24 geeft $R(s,s) > 2^{\lfloor s/2 \rfloor}$.

Beide grenzen stammen uit de jaren dertig en veertig. **Het grondtal van de exponentiële functie is nog steeds onbekend.** In tachtig jaar is de kloof tussen $\sqrt{2}$ en 4 alleen in de termen van lagere orde versmald — een resultaat uit 2023 verbeterde de bovengrens tot $3,993^s$, wat groot nieuws was en het beeld niet veranderde.

De bekende waarden

	$t=3$	$t=4$	$t=5$	$t=6$
$s=3$	6	9	14	18
$s=4$		18	25	36–40
$s=5$			43–46	58–85

$R(5, 5)$ is onbekend. Het is bekend dat het tussen 43 en 46 ligt, en daar houdt de tabel op een tabel te zijn.

De reden is de schaal. Beslissen of $R(5, 5) = 43$ betekent elke 2-kleuring van K_{43} controleren, en daarvan zijn er 2^{903} . Dat is geen kwestie van wachten op snellere computers: 2^{903} overtreft het aantal atomen in het waarneembare heelal tot een aanzienlijke macht. Geen denkbare berekening raakt eraan, en er is geen beter idee gevonden.

De opmerking van Erdős is de standaardsamenvatting, en ze is niet echt een grap: eisten buitenaardse wezens $R(5, 5)$ op straffe van vernietiging, dan zouden we al onze computers en wiskundigen moeten mobiliseren om het te vinden; eisten ze $R(6, 6)$, dan zouden we moeten proberen de buitenaardse wezens te vernietigen.

Het patroon voorbij grafen

De stelling van Ramsey is één geval van een algemeen verschijnsel: **volledige wanorde is onmogelijk**. Elke voldoende grote structuur bevat een grote geordende deelstructuur, wat "geordend" in die context ook betekent.

- **Van der Waerden:** elke 2-kleuring van $\{1, \dots, N\}$ bevat een monochromatische rekenkundige rij van lengte k , voor N groot genoeg.
- **Szemerédi:** elke deelverzameling van de gehele getallen met positieve bovendichtheid bevat willekeurig lange rekenkundige rijen.
- **Green–Tao:** de priemgetallen bevatten willekeurig lange rekenkundige rijen — ondanks dichtheid nul, zodat Szemerédi niet rechtstreeks van toepassing is.
- **Hales–Jewett:** hoogdimensionaal boter-kaas-en-eieren kan niet in remise eindigen.

Ze delen alle dezelfde vorm, en ze delen alle hetzelfde gebrek: de grenzen zijn astronomisch slecht. De oorspronkelijke grens van Van der Waerden was niet primitief recursief. Het regelmatigheidslemma van Szemerédi — het hoofdinstrument — heeft grenzen die torens van exponenten zijn, en Gowers bewees dat dat noodzakelijk is en geen artefact.

Ramsey-achtige stellingen vertellen je dat een structuur bestaat en geven je geen enkele manier om haar te vinden. Dat is dezelfde klacht die hoofdstuk 24 over de probabilistische methode uitte, en dat is geen toeval: de ondergrenzen zijn probabilistisch en de bovengrenzen Ramsey-achtig, en geen van beide is constructief.

Probeer het

```
python -c "
import sys, time; sys.path.insert(0, '.')
from graphs.extremal import ramsey_holds, ramsey_counterexample
t0 = time.perf_counter()
print('every colouring of K_6 has a mono triangle:', ramsey_holds(6, 3, 3))
witness = ramsey_counterexample(5, 3, 3)
print('K_5 counterexample exists: ', witness is not None)
print('  its red edges:', sorted(witness.edges()))
print(f'  ({time.perf_counter() - t0:.1f}s for 32768 + 1024 colourings)')
"
```

```
every colouring of K_6 has a mono triangle: True
K_5 counterexample exists: True
  its red edges: [(0, 3), (0, 4), (1, 2), (1, 4), (2, 3)]
  (0.5s for 32768 + 1024 colourings)
```

Een halve seconde doet $R(3,3)$ af. Die aanpak opschalen naar $R(5,5)$ zou 2^{903} kleuringen vergen, en het verschil tussen die twee getallen is de volledige moeilijkheid van het vak.

Oefeningen

1. Bewijs $R(3,3) \leq 6$ met het duivenhokargument, in je eigen woorden.
2. Toon aan dat het tegenvoorbeeld op K_5 werkelijk geen monochromatische driehoek heeft, door beide kleurklassen na te gaan.
3. Gebruik $R(s,t) \leq R(s-1,t) + R(s,t-1)$ om $R(3,4) \leq 10$ af te leiden. (De werkelijke waarde is 9.)
4. Waarom kan geen berekening $R(5,5)$ door uitputtend zoeken afdoen? Geef het aantal kleuringen.

Oplossingen in Bijlage E.

Kernpunten

- $R(3,3) = 6$, bewijsbaar met één duivenhokstap, en hier afgedaan door uitputtend zoeken over alle 32.768 kleuringen — de enige plaats in dit boek waar een eindige berekening *een bewijs is*.
- $2^{\lfloor s/2 \rfloor} < R(s,s) < 4^s$, beide grenzen uit de jaren dertig en veertig, en het grondtal is na tachtig jaar nog steeds onbekend.
- Van $R(5,5)$ is alleen bekend dat het in $[43, 46]$ ligt. Het door zoeken afdoen vergt 2^{903} kleuringen, dus het zal niet door zoeken afgedaan worden.
- Het algemene patroon — Van der Waerden, Szemerédi, Green–Tao — is dat volledige wanorde onmogelijk is. Ze hebben alle astronomisch slechte grenzen, en aantoonbaar zo.
- Net als de probabilistische methode bewijst de Ramsey-theorie bestaan en biedt ze geen constructie.

Hoofdstuk 29 — Spectrale grafentheorie

Hoofdstuk 4 merkte op dat A^k wandelingen telt en liet de matrix daar liggen. Dit hoofdstuk neemt de matrix serieus: de adjacentiematrix van een graaf is reëel en symmetrisch, dus ze heeft n reële eigenwaarden, en die getallen coderen een verrassende hoeveelheid structuur.

Veertig regels eigenwaarde-oplosser

Dit boek heeft geen dependencies, dus het heeft zijn eigen oplosser nodig. Voor reële symmetrische matrices is de **Jacobi-rotatiemethode** kort, onvoorwaardelijk convergent, en leesbaar.

Het idee: zoek de grootste plaats buiten de diagonaal en roteer in dat vlak om haar op nul te zetten. Elke rotatie verkleint de som van de kwadraten van de niet-diagonale plaatsen, dus de matrix convergeert naar een diagonale matrix, waarvan de plaatsen de eigenwaarden zijn.

```
for _ in range(max_sweeps):
    off, p, q = 0.0, 0, 0
    for i in range(n):
        for j in range(i + 1, n):
            if abs(a[i][j]) > off:
                off, p, q = abs(a[i][j]), i, j
    if off < tolerance:
        break
    theta = (math.pi / 4 if a[p][p] == a[q][q]
            else 0.5 * math.atan2(2 * a[p][q], a[p][p] - a[q][q]))
    c, s = math.cos(theta), math.sin(theta)
    # rotate rows p, q and then columns p, q
```

Trager dan wat een bibliotheek gebruikt — $O(n^3)$ per veegbeurt — en het is degene die je kunt lezen. Ze is bovendien alleen geldig voor symmetrische invoer; er iets anders in stoppen levert onzin op in plaats van een fout, en de moduledocumentatie zegt dat expliciet.

Een eigenwaarde-oplosser verifiëren vergt zorg: haar vergelijken met een andere oplosser die je ook zelf schreef bewijst niets. De verificatie gebruikt in plaats daarvan **spooridentiteiten**, die feiten over de graaf zijn en niet over de lineaire algebra:

- $\sum \lambda_i = \text{spoor}(A) = 0$, want de diagonaal is helemaal nul;
- $\sum \lambda_i^2 = \text{spoor}(A^2) = 2m$, want $(A^2)_{\{vv\}}$ is het aantal gesloten wandelingen van lengte 2, en dat is $\deg(v)$;
- $\sum \lambda_i^3 = \text{spoor}(A^3) = 6 \cdot (\text{aantal driehoeken})$, want een gesloten wandeling van lengte 3 is een driehoek doorlopen vanaf een van 3 startpunten in een van 2 richtingen.

held	ch29	The adjacency spectrum sums to 0 and its squares sum to 2m (52 graphs)
held	ch29	The cube of the spectrum sums to six times the triangle count (52 graphs)

De derde identiteit is de goede: het aantal driehoeken rechts wordt door directe opsomming berekend, dus een spectrum dat eraan voldoet wordt tegen combinatoriek gecontroleerd en niet tegen nog meer lineaire algebra.

Wat het spectrum weet

K4	spectrum [-1.0, -1.0, -1.0, 3.0]
C4	spectrum [-2.0, -0.0, 0.0, 2.0]
C5	spectrum [-1.618, -1.618, 0.618, 0.618, 2.0]
petersen	spectrum [-2.0, -2.0, -2.0, -2.0, 1.0, 1.0, 1.0, 1.0, 1.0, 3.0]
K33	spectrum [-3.0, -0.0, 0.0, 0.0, 0.0, 3.0]

Lees de Petersen-rij: 3 één keer, 1 vijf keer, -2 vier keer. Een graaf op 10 knopen met slechts drie verschillende eigenwaarden, wat uiterst ongewoon is en een van de redenen is dat deze graaf bijzonder is — grafen met drie verschillende adjacentie-eigenwaarden zijn **sterk regulier**, en de Petersen-graaf is de kleinste interessante.

Verschillende structurele feiten lees je rechtstreeks van het spectrum af:

- $\lambda_{\max}$ ligt tussen de gemiddelde en de maximale graad: $2m/n \leq \lambda_{\max} \leq \Delta$. Voor een d -reguliere graaf vallen beide grenzen samen, dus $\lambda_{\max} = d$ precies, met de al-enen-eigenvector.
- De graaf is bipartiet dan en slechts dan als het spectrum symmetrisch is om 0. C_4 geeft $\{-2, 0, 0, 2\}$ en $K_{3,3}$ geeft $\{-3, 0, 0, 0, 0, 3\}$; C_5 geeft een asymmetrisch spectrum en is niet bipartiet.
- Het aantal samenhangscomponenten is de multipliciteit van $\lambda_{\max}$ voor een reguliere graaf.

held	ch29	A d -regular graph has spectral radius exactly d (12 graphs)
held	ch29	The spectral radius lies between the average and maximum degree (52 graphs)

Wat het spectrum niet weet

Het spectrum is een invariant, dus het kan bewijzen dat twee grafen niet isomorf zijn. Zoals elke invariant in hoofdstuk 5 kan het niet bewijzen dat ze het wel zijn.

Twee grafen met hetzelfde spectrum heten **cospectraal**. Het kleinste cospectrale paar is $K_{1,4}$ en $C_4 + K_1$ — een ster en een vierhoek-plus-losse-knoop, beide met spectrum $\{-2, 0, 0, 0, 2\}$. Ze zijn duidelijk niet isomorf; de ene is samenhangend.

Erger nog, dit is niet zeldzaam. **Vrijwel alle bomen zijn cospectraal met een andere boom**, dus voor bomen is het spectrum als onderscheidingsmiddel bijna nutteloos. De precieze fractie van alle grafen die door hun spectrum bepaald wordt is een open probleem, en de numerieke aanwijzingen suggereren dat ze dicht bij 1 ligt voor algemene grafen — wat bomen tot een ongewoon slecht geval maakt.

Dit is hetzelfde verhaal als kleurverfijning in hoofdstuk 5: een goedkope invariant, correct in één richting, en blind voor een familie gevallen die je kunt karakteriseren.

Verstrengeling

Het diepste elementaire instrument hier is **eigenwaardeverstrengeling**.

Stelling (verstrengeling van Cauchy). Is H een geïnduceerde deelgraaf van G op $n - 1$ knopen, met eigenwaarden $\mu_1 \leq \dots \leq \mu_{n-1}$ en heeft G $\lambda_1 \leq \dots \leq \lambda_n$, dan geldt $\lambda_1 \leq \mu_1 \leq \lambda_2 \leq \mu_2 \leq \dots \leq \mu_{n-1} \leq \lambda_n$.

Een knoop verwijderen kan de eigenwaarden niet langs elkaar heen laten schuiven. Dit levert grenzen die combinatorisch moeilijk te krijgen zijn — bijvoorbeeld de **verhoudingsgrens**, dat een onafhankelijke verzameling in een d -reguliere graaf grootte hoogstens $n \cdot (-\lambda_{\min})/(d - \lambda_{\min})$ heeft. Voor de Petersen-graaf: $10 \cdot 2/(3 + 2) = 4$, en dat is precies $\alpha(\text{Petersen}) = 4$ zoals hoofdstuk 21 berekende.

Een scherpe grens op een NP-moeilijke grootheid, uit vier eigenwaarden, is een eerlijke aanbeveling voor de methode.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete, cycle, petersen, complete_bipartite, Graph
from graphs.spectral import adjacency_spectrum

for name, g in [('C4', cycle(4)), ('C5', cycle(5)), ('K3,3',
complete_bipartite(3,3)),
                ('petersen', petersen())]:
    sp = [round(x, 3) for x in adjacency_spectrum(g)]
    symmetric = all(abs(sp[i] + sp[-1-i]) < 1e-6 for i in range(len(sp)))
    print(f'{name:<9} {sp}')
    print(f'{"":<9} symmetric about 0: {symmetric} (bipartite iff this)')

print()
star = Graph(5, [(0,1),(0,2),(0,3),(0,4)])
other = Graph(5, [(0,1),(1,2),(2,3),(3,0)])
print('cospectral pair:')
print(' K_1,4      ', [round(x,3) for x in adjacency_spectrum(star)])
print(' C_4 + K_1    ', [round(x,3) for x in adjacency_spectrum(other)])
"
```

```
C4      [-2.0, -0.0, 0.0, 2.0]
        symmetric about 0: True (bipartite iff this)
C5      [-1.618, -1.618, 0.618, 0.618, 2.0]
        symmetric about 0: False (bipartite iff this)
K3,3    [-3.0, -0.0, 0.0, 0.0, 0.0, 3.0]
        symmetric about 0: True (bipartite iff this)
petersen [-2.0, -2.0, -2.0, -2.0, 1.0, 1.0, 1.0, 1.0, 1.0, 3.0]
        symmetric about 0: False (bipartite iff this)

cospectral pair:
K_1,4    [-2.0, -0.0, 0.0, 0.0, 2.0]
C_4 + K_1 [-2.0, -0.0, 0.0, 0.0, 2.0]
```

Identieke spectra; de ene graaf is samenhangend en de andere niet. Het spectrum weet heel veel en niet alles.

Oefeningen

1. Bereken het adjacentiespectrum van K_n met de hand. (Hint: $A = J - I$.)
2. Toon aan dat $\sum \lambda_i^2 = 2m$ volgt uit het tellen van gesloten wandelingen van lengte 2.
3. Verifieer de verhoudingsgrens op de Petersen-graaf en vergelijk met $\alpha = 4$.
4. Waarom kan het spectrum bewijzen dat twee grafen niet isomorf zijn maar nooit dat ze het wel zijn?

Oplossingen in Bijlage E.

Kernpunten

- De adjacenciematrix van een graaf is reëel symmetrisch, dus het spectrum is reëel. Veertig regels Jacobi-rotatie berekenen het zonder dependencies.
- Verifieer een eigenwaarde-oplosser tegen spooridentiteiten — $\sum \lambda = 0$, $\sum \lambda^2 = 2m$, $\sum \lambda^3 = 6 \cdot \text{driehoeken}$ — en niet tegen een andere oplosser.
- $\lambda_{\max}$ ligt tussen de gemiddelde en de maximale graad, met gelijkheid voor reguliere grafen. Bipartiet precies wanneer het spectrum symmetrisch om 0 is.
- Cospectrale niet-isomorfe grafen bestaan en zijn gewoon onder bomen. Het spectrum is een eenzijdige invariant, zoals alles in hoofdstuk 5.
- Verstrengeling geeft grenzen op combinatorische grootheden uit eigenwaarden; de verhoudingsgrens is op de Petersen-graaf precies scherp.

Hoofdstuk 30 — De Laplaciaan

De adjacentiematrix is de voor de hand liggende matrix om aan een graaf te hangen. De **Laplaciaan** $L = D - A$, met D de diagonale graadmatrix, is de bruikbare — en dit hoofdstuk gaat over waarom die wissel alles verandert.

De kwadratische vorm

De reden dat L beter is, is één identiteit:

Lemma. Voor elke vector x geldt $x^T L x = \sum_{uv \in E} (x_u - x_v)^2$.

Bewijs. Uitwerken: $x^T D x = \sum_v \deg(v) x_v^2$ en $x^T A x = 2 \sum_{uv} x_u x_v$. Aftrekken geeft $x^T L x = \sum_v \deg(v) x_v^2 - 2 \sum_{uv} x_u x_v = \sum_{uv} (x_u^2 + x_v^2 - 2x_u x_v)$, want elke knoop komt in $\deg(v)$ kanttermen voor. ■

Alles volgt hieruit. $x^T L x \geq 0$ altijd, dus L is positief semidefiniet en alle eigenwaarden zijn ≥ 0 . De vorm is nul precies wanneer x constant is op elke samenhangscomponent. Dus:

Stelling. De multipliciteit van eigenwaarde 0 in L is gelijk aan het aantal samenhangscomponenten.

```
K4      L-spectrum [0.0, 4.0, 4.0, 4.0]
C5      L-spectrum [0.0, 1.382, 1.382, 3.618, 3.618]
petersen L-spectrum [0.0, 2.0, 2.0, 2.0, 2.0, 5.0, 5.0, 5.0, 5.0]
P4      L-spectrum [0.0, 0.5858, 2.0, 3.4142]
two triangles L-spectrum [0.0, 0.0, 3.0, 3.0, 3.0, 3.0]
```

De laatste rij heeft twee keer 0, en de graaf heeft twee componenten. Samenhang — een combinatorische eigenschap — wordt van een eigenwaardemultipliciteit afgelezen.

```
held      ch30 The multiplicity of Laplacian eigenvalue 0 is the number of
components (52 graphs)
```

De matrix-boomstelling

Hoofdstuk 7 beloofde dit en kon het niet waarmaken.

Stelling (Kirchhoff, 1847). Het aantal opspannende bomen van G is gelijk aan elke cofactor van L — schrap een willekeurige rij en de bijbehorende kolom, en neem de determinant.

```
def spanning_tree_count(g):
    lap = laplacian_matrix(g)
    minor = [row[1:] for row in lap[1:]]
    return round(determinant(minor))
```

$O(n^3)$, tegen de opsomming van $C(m, n-1)$ uit hoofdstuk 7. De verificatie controleert de een tegen de ander, en dat is een determinant tegen een uitputtende zoektocht — zo onafhankelijk als twee berekeningen maar kunnen zijn:

```
held      ch30 Matrix-tree: a Laplacian cofactor counts spanning trees (52
graphs)
```

En de belofte is nu ingelost. Hoofdstuk 7 kon de formule van Cayley alleen tot K_6 verifiëren:

```
K10 spanning trees: 100000000      10^8 = 100000000
```

K_{10} heeft 10^8 opspannende bomen, ogenblikkelijk bevestigd uit een 9×9 -determinant. De Petersen-graaf heeft er 2000.

Algebraïsche samenhang

De op één na kleinste eigenwaarde λ_2 is de **algebraïsche samenhang**, of Fiedler-waarde. Ze is nul precies wanneer de graaf onsamenvhangend is, en groter naarmate de graaf moeilijker te splitsen is.

```
held      ch30 Algebraic connectivity is positive exactly when the graph is
connected (51 graphs)
```

Vergelijk P_4 bij 0,586 met K_4 bij 4,0. Beide zijn samenhangend; de ene is een pad dat één verwijderde kant breekt, de andere is zo robuust samenhangend als vier knopen toelaten. De Fiedler-waarde meet dat verschil op een continue schaal, waar $\kappa(G)$ en $\lambda(G)$ uit hoofdstuk 12 gehele getallen geven.

De ongelijkheid van Cheeger

Het precieze verband tussen het spectrum en het doorknippen van de graaf is het diepste resultaat in dit boek met een korte formulering.

Definieer de **Cheeger-constante** — het isoperimetrisch getal:

```
h(G) = min over S met |S| ≤ n/2 van |kanten die S verlaten| / |S|
```

Haar berekenen betekent elke deelverzameling nagaan, dus ze is per definitie exponentieel. En toch:

Stelling (Cheeger; Alon–Milman). $\lambda_2/2 \leq h(G) \leq \sqrt{(2 \cdot \Delta \cdot \lambda_2)}$.

Een tweezijdige grens op een exponentieel moeilijke grootte, uit één eigenwaarde die in $O(n^3)$ berekenbaar is. Dat is de volledige theoretische grondslag van spectraal clusteren.

```
held      ch30 Cheeger: a(G)/2 ≤ h(G) ≤ sqrt(2 * Delta * a(G)) (30 graphs)
```

De verificatie berekent $h(G)$ via haar exponentiële definitie en λ_2 uit de Laplaciaan, dus de twee zijden komen uit ongerelateerde berekeningen. Dat is de enige eerlijke manier om een ongelijkheid tussen beide te controleren.

De grens is niet scherp — er zit een vierkantswortel tussen de twee zijden — en die kloof is echt, geen artefact van het bewijs. Ze is de reden dat spectraal clusteren goede partities geeft en geen optimale.

Spectraal clusteren

Het algoritme volgt vanzelf. Om een graaf in tweeën te splitsen:

1. bereken de eigenvector bij λ_2 — de **Fiedler-vector**;
2. sorteer de knopen op hun plaats erin;
3. knip bij de beste van de $n - 1$ resulterende splitsingen.

De rechtvaardiging is opnieuw de kwadratische vorm. $x^T L x$ minimaliseren onder $x \perp 1$ en $\|x\| = 1$ geeft precies de Fiedler-vector, en $x^T L x = \sum (x_u - x_v)^2$ is klein juist wanneer aangrenzende knopen soortgelijke waarden krijgen. De Fiedler-vector is dus de toekenning van getallen aan knopen die aangrenzende knopen zo dicht mogelijk bij elkaar brengt — een **relaxatie** van het 0/1-waardige snedeprobleem, met de geheeltalligheid weggelaten.

Dat is het algemene recept om mee te nemen: **laat de geheeltalligheidseis vallen, los het continue probleem exact op, en rond af**. De ongelijkheid van Cheeger is precies de uitspraak dat afronden niet te veel verliest.

Probeer het

```
python -c "
import sys, math; sys.path.insert(0, '.')
from graphs.core import complete, cycle, path, petersen, Graph
from graphs.spectral import (algebraic_connectivity, cheeger_constant,
                             laplacian_spectrum, spanning_tree_count)
print(f"\n {'graph':<10} {'lambda2':>9} {'h(G)':>7} {'lambda2/2':>10}
{'sqrt(2*D*l2)':>13} {'trees':>7}\n")
for name, g in [('P4', path(4)), ('C5', cycle(5)), ('K4', complete(4)),
('petersen', petersen())]:
    l2 = algebraic_connectivity(g); h = cheeger_constant(g)
    D = max(g.degree(v) for v in g.vertices())
    print(f'  {name:<10} {l2:>9.4f} {h:>7.4f} {l2/2:>10.4f}
{math.sqrt(2*D*l2):>13.4f} {spanning_tree_count(g):>7}')
print()
two = Graph(6, [(0,1),(1,2),(2,0),(3,4),(4,5),(5,3)])
print(' disconnected graph, lambda2 =', round(algebraic_connectivity(two), 10))
"
```

graph	lambda2	h(G)	lambda2/2	sqrt(2*D*l2)	trees
P4	0.5858	0.5000	0.2929	1.5307	1
C5	1.3820	1.0000	0.6910	2.3511	5
K4	4.0000	2.0000	2.0000	4.8990	16
petersen	2.0000	1.0000	1.0000	3.4641	2000

disconnected graph, lambda2 = 0.0

Elke rij voldoet aan $\lambda_2/2 \leq h(G) \leq \sqrt{(2D\lambda_2)}$, en de twee grenzen liggen ver uiteen — voor **P₄** omsluiten 0,29 en 1,53 een werkelijke waarde van 0,50. Dat venster met factor vijf is wat "spectraal clusteren werkt maar is niet optimaal" kwantitatief betekent.

Merk op dat K_4 het scherpe geval aan de linkerkant is: $\lambda_2/2 = 2,0 = h(G)$.

Oefeningen

1. Bewijs $x^T L x = \sum_{uv \in E} (x_u - x_v)^2$ door uit te werken, en leid af dat L positief semidefiniet is.
2. Bereken het Laplace-spectrum van K_3 met de hand en controleer dat de matrix-boomstelling 3 geeft.
3. Waarom is de al-enen-vector altijd een eigenvector van L met eigenwaarde 0?
4. P_4 heeft $\lambda_2 = 0,586$ en K_4 heeft $\lambda_2 = 4$. Beide zijn samenhangend — wat meet het verschil?

Oplossingen in Bijlage E.

Kernpunten

- $L = D - A$, en $x^T L x = \sum (x_u - x_v)^2$ is de identiteit waaruit alles volgt.
- De multipliciteit van eigenwaarde nul is het aantal componenten. Samenhang wordt lineaire algebra.
- Matrix-boom telt opspannende bomen met één $O(n^3)$ -determinant en maakt de exponentiële opsomming van hoofdstuk 7 overbodig: K_{10} heeft 10^8 opspannende bomen, ogenblikkelijk berekend.
- λ_2 is nul precies wanneer de graaf onsaamenhangend is, en meet hoe moeilijk hij te doorknippen is.
- Cheeger omsluit een exponentieel moeilijke grootheid tussen twee functies van één eigenwaarde. De kloof is een echte vierkantswortel, en daarom is spectraal clusteren goed en niet optimaal.
- Het recept veralgemeent: laat de geheeltalligheid vallen, los exact op, rond af.

Hoofdstuk 31 — Minoren en boombreedte

Hoofdstuk 17 karakteriseerde vlakheid met twee verboden minoren. Dit hoofdstuk gaat over hoe ver dat idee reikt, en het reikt opmerkelijk ver — tot een stelling die oneindig veel vragen tegelijk beslecht en je het antwoord op geen enkele ervan vertelt.

Minoren

H is een **minor** van G wanneer hij verkregen kan worden door knopen te verwijderen, kanten te verwijderen, en kanten samen te trekken. Een klasse grafen is **minor-gesloten** wanneer ze onder die operaties gesloten is: vlakke grafen, bossen, grafen inbedbaar op een vast oppervlak, grafen van boombreedte hoogstens k .

Kuratowski en Wagner gaven vlakheid twee verboden minoren. De voor de hand liggende vraag is of elke minor-gesloten klasse een eindige belemmeringsverzameling heeft. Dat is zo.

Stelling (Robertson–Seymour, 1983–2004). Elke minor-gesloten klasse grafen wordt gekarakteriseerd door een **eindige** verzameling verboden minoren.

Twintig artikelen over twintig jaar, en het is een van de diepste resultaten in de combinatoriek. Het heeft een onmiddellijk algoritmisch gevolg, want testen op een vaste minor H kost $O(n^3)$:

Gevolg. Lidmaatschap van elke minor-gesloten klasse is beslisbaar in $O(n^3)$.

En hier komt het vreemdste. De stelling vertelt je niet wat de verboden minoren *zijn*. Ze bewijst dat de verzameling eindig is zonder haar te identificeren — dus voor een gegeven minor-gesloten klasse weet je dat er een kubisch algoritme bestaat en heb je mogelijk geen enkele manier om het op te schrijven. Voor grafen inbedbaar op de torus zijn meer dan 17.000 verboden minoren bekend en de volledige lijst is nog steeds onbekend.

Dit is niet-constructiviteit van een andere orde dan die van hoofdstuk 24. De probabilistische methode bewijst dat een object bestaat zonder het te produceren; Robertson–Seymour bewijst dat een *algoritme* bestaat zonder het te produceren. Weten dat je probleem in kubische tijd beslisbaar is terwijl je geen middel hebt om het te beslissen, is een ongewone positie.

Boombreedte

De andere helft van de theorie is een parameter die meet hoe boomachtig een graaf is.

Een **boomdecompositie** kent elke knoop van een boom een **zak** knopen toe zodat:

1. elke knoop in een zak zit;
2. elke kant beide eindpunten in een zak heeft;
3. voor elke knoop de zakken die hem bevatten een samenhangende deelboom vormen.

De **breedte** is de grootste zakgrootte min één; de **boombreedte** is de minimale breedte over alle decomposities. De min één is een conventie zó gekozen dat bomen boombreedte 1 hebben in plaats van 2.

Voorwaarde 3 draagt de inhoud. Zonder haar kon je elke knoop in één zak stoppen en was de definitie leeg.

Boombreedte berekenen gaat eenvoudiger via **eliminatievolgordes**:

```
def eliminate(g, order):
    adjacency = {v: set(g.neighbours(v)) for v in g.vertices()}
    remaining, width = set(g.vertices()), 0
    for v in order:
        nbrs = adjacency[v] & remaining - {v}
        width = max(width, len(nbrs))
        for a, b in itertools.combinations(nbrs, 2): # the fill edges
            adjacency[a].add(b); adjacency[b].add(a)
        remaining.discard(v)
    return width
```

$\text{treewidth}(G)$ is het minimum hiervan over alle $n!$ volgordes. Dat is exact en hopeloos voorbij $n = 8$, wat passend is — boombreedte berekenen is NP-moeilijk, al is het vast-parameter hanteerbaar in de breedte zelf (Bodlaender).

P5	n=5	treewidth=1
tree	n=7	treewidth=1
C5	n=5	treewidth=2
K4	n=4	treewidth=3
K5	n=5	treewidth=4
K33	n=6	treewidth=3
grid2x3	n=6	treewidth=2
grid3x3	n=9	treewidth=3

Bossen hebben boombreedte 1, cykels 2, K_n precies $n - 1$. Het $r \times c$ -rooster heeft boombreedte $\min(r, c)$ — dus **vlakke grafen hebben onbegrensde boombreedte**, en vlakheid en begrensde boombreedte zijn werkelijk verschillende beperkingen.

held	ch31	Treewidth is 1 exactly for forests with at least one edge (51 graphs)
held	ch31	Treewidth of K_n is $n-1$, and of a cycle is 2 (7 graphs)
held	ch31	Treewidth never increases when passing to a subgraph (51 graphs)
held	ch31	Chordal graphs have treewidth = max clique size - 1 (44 graphs)

De laatste regel verbindt met hoofdstuk 19. Koordale grafen zijn precies de grafen waarvan de eliminatie geen opvulkanten produceert, dus hun boombreedte is rechtstreeks van hun grootste kliek af te lezen — en daarom komen koordaliteit en boombreedte steeds in dezelfde zinnen voor.

De stelling van Courcelle

Begrensde boombreedte is de moeite waard vanwege wat je ermee kunt.

Stelling (Courcelle, 1990). Elke grafeneigenschap uitdrukbaar in monadische tweede-orde logica is in lineaire tijd beslisbaar op grafen van begrensde boombreedte.

Dat omvat 3-kleurbaarheid, Hamiltoniciteit, onafhankelijke verzameling, dominerende verzameling, en in wezen elk probleem in deel V. Alle NP-moeilijk in het algemeen; alle lineaire tijd zodra de boombreedte begrensd is.

Het mechanisme is dynamisch programmeren over de decompositieboom: verwerk de zakken van de bladeren omhoog en houd voor elke zak een tabel van deeloplossingen bij, geïndexeerd op de inwendige configuratie van de zak. De tabel heeft grootte exponentieel in de **breedte** en het aantal zakken is lineair in n , wat $f(k) \cdot n$ geeft.

Dit boek implementeert dat dynamisch programmeren niet. Het correct doen betekent de decompositieboom bouwen en introduceer-, vergeet- en samenvoegknopen behandelen, en een fout erin zou onzichtbaar zijn tegen de brute-force-antwoorden die de verificatie overal elders gebruikt. Dezelfde keuze werd gemaakt voor Hopcroft–Tarjan-vlakheid in hoofdstuk 17 en voor de vierkleurenstelling in hoofdstuk 18: waar de eerlijke opties een lange correcte implementatie of een korte foute zijn, beschrijft dit boek en zegt het dat het beschrijft.

Het praktische voorbehoud is dat $f(k)$ vaak wreed is — voor de stelling van Courcelle in volle algemeenheid is het een toren van exponenten in de formulegrootte. "Lineaire tijd" verbergt een constante die de leeftijd van het heelal kan overtreffen. Met de hand geschreven DP's voor specifieke problemen doen het veel beter, en dat is wat men in de praktijk gebruikt.

De structuurstelling voor grafenminoren

Onder Robertson–Seymour ligt een structurele beschrijving waarvan het bestaan het weten waard is.

Ruwweg: voor elke vaste H kan elke H -minorvrije graaf opgebouwd worden door grafen aan elkaar te lijmen die "bijna inbedbaar" zijn op oppervlakken van begrensd geslacht.

Een minor uitsluiten dwingt dus bijna-topologische structuur af. Dit is de motor achter zowel de welgeordendheidsstelling als de algoritmische resultaten, en de volledige formulering beslaat een bladzijde. Het is de reden dat "uitgesloten minor" en "begrensd boombreedte" de twee ordenende ideeën van de structurele grafentheorie zijn.

Probeer het

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph, complete, cycle, path, complete_bipartite
from graphs.treewidth import treewidth, tree_decomposition, is_tree_decomposition,
grid

for name, g in [('P5', path(5)), ('C5', cycle(5)), ('K4', complete(4)),
                ('K3,3', complete_bipartite(3,3)), ('2x3 grid', grid(2,3))]:
    print(f'{name:<9} treewidth = {treewidth(g)}')
print()
g = cycle(5)
bags = tree_decomposition(g, list(g.vertices()))
print('C5 bags from the natural elimination order:', [sorted(b) for b in bags])
print('is a valid tree decomposition:', is_tree_decomposition(g, bags))
print('width of THIS ordering:', max(len(b) for b in bags) - 1, ' optimal:',
      treewidth(g))
"
```

```
P5      treewidth = 1
C5      treewidth = 2
K4      treewidth = 3
K3,3    treewidth = 3
2x3 grid treewidth = 2

C5 bags from the natural elimination order: [[0, 1, 4], [1, 2, 4], [2, 3, 4], [3,
4], [4]]
is a valid tree decomposition: True
width of THIS ordering: 2  optimal: 2
```

De zakken krimpen naarmate de eliminatie vordert, en de grootste heeft drie knopen — breedte 2, wat optimaal is voor een cykel. Let op de opvulkant: knoop 0 elimineren verbindt zijn burens 1 en 4, die in C_5 niet aangrenzend waren.

Oefeningen

1. Geef een boomdecompositie van K_4 van breedte 3, en beargumenteer dat er geen betere bestaat.
2. Waarom trekt de definitie van breedte er één af?
3. Toon aan dat het $2 \times n$ -rooster boombreedte 2 heeft voor elke n .
4. Vlakke grafen hebben onbegrensde boombreedte. Welke familie toont dat, en waarom doet het ertoe voor algoritmen?

Oplossingen in Bijlage E.

Kernpunten

- Minor-gesloten klassen hebben eindige karakteriseringen met verboden minoren (Robertson–Seymour), dus lidmaatschap is beslisbaar in $O(n^3)$ — zonder dat de stelling je vertelt welke minoren, en dus ook niet hoe.

- Dat is niet-constructiviteit één niveau boven hoofdstuk 24: een *algoritme* waarvan bewezen is dat het bestaat en dat niet geproduceerd wordt.
- Boombreedte meet boomachtigheid: 1 voor bossen, 2 voor cykels, $n - 1$ voor K_n , $\min(r, c)$ voor het $r \times c$ -rooster.
- Vlakke grafen hebben onbegrensde boombreedte, dus de twee beperkingen zijn onafhankelijk.
- Courcelle: begrensde boombreedte maakt elke MSO-uitdrukbare eigenschap lineaire tijd. De verborgen constante kan een toren van exponenten zijn, en dit boek beschrijft het DP in plaats van het te implementeren.

Hoofdstuk 32 — Expanders, en waar je hierna heen gaat

Het laatste idee in dit boek is een graaf die ijl is en toch buitengewoon goed samenhangt. Die twee eisen klinken tegenstrijdig, en dat ze het niet zijn, is een van de nuttigste ontdekkingen in de moderne wiskunde.

De definitie

Een **expander** is een familie d -reguliere grafen op groeiende n waarvan de Cheeger-constante $h(G)$ van onderen begrensd blijft door een constante. Elke verzameling van hoogstens de helft van de knopen heeft een rand evenredig met haar grootte, uniform, voor willekeurig grote grafen — met slechts $dn/2$ kanten om mee te werken.

De ongelijkheid van Cheeger uit hoofdstuk 30 maakt hiervan een spectrale voorwaarde. Goede expansie is equivalent met een **grote spectrale kloof** $d - \lambda$, waarbij λ de grootste niet-triviale eigenwaarde in absolute waarde is. Expanders construeren wordt dus een eigenwaardeprobleem.

Twée lambda's, en ze verwarren

Er zit hier een subtiliteit die dit boek bij de eerste poging fout had, en die de alinea waard is.

Voor een d -reguliere graaf is d altijd een eigenwaarde (de al-enen-eigenvector). Is de graaf **bipartiet**, dan is $-d$ het ook. Er zijn twee grootheden in omloop:

- **spectral_expansion** — sluit zowel d als $-d$ uit. Dit is de λ van de Ramanujan-voorwaarde. Een bipartiete graaf heeft $-d$ juist *omdat* hij bipartiet is, en die behouden zou elke bipartiete graaf brandmerken als slechte expander om een reden die niets met samenhang te maken heeft.
- **mixing_lambda** — sluit alleen d uit. Dit is de λ van het mengingslemma hieronder, en $-d$ moet behouden blijven.

Eén functie voor beide gebruiken liet het mengingslemma onmiddellijk falen op $K_{3,3}$. Neem $S = \{0\}$ en $T = \{1\}$, beide aan dezelfde zijde: $e(S, T) = 0$ terwijl $d|S||T|/n = 0,5$, dus het verschil is $0,5$. Met -3 uitgesloten is de grens 0 , en het lemma is geschonden. De eigenwaarde -3 is precies wat dat verschil verantwoordt — het is geen ruis.

Hetzelfde symbool, twee definities, en telkens is er maar één juist.

Het mengingslemma

Stelling (mengingslemma voor expanders). Voor een d -reguliere graaf met $\lambda = \text{mixing_lambda}(G)$ en willekeurige knopenverzamelingen S, T :

$$|e(S, T) - d|S||T|/n| \leq \lambda \sqrt{(|S||T|)}.$$

De middelste term is wat je zou verwachten als de kanten willekeurig geplaatst waren. Het lemma zegt dus dat een graaf met kleine λ een kantverdeling heeft die **niet te onderscheiden is van toeval** op de resolutie van

verzamelingsgroottes. Dat is wat "pseudotoevallig" precies betekent, en het is waarom expanders op zoveel plaatsen toeval vervangen.

held ch32 Expander mixing lemma: $|e(S,T) - d|S||T|/n| \leq \lambda \sqrt{|S||T|}$ (11 graphs)

Gecontroleerd over **elk** paar deelverzamelingen, tegen een spectrum uit de Jacobi-oplosser van hoofdstuk 29 — combinatoriek aan de ene kant, lineaire algebra aan de andere.

Hoe goed kan expansie worden?

Stelling (Alon–Boppana). Voor d -reguliere grafen geldt $\lambda \geq 2\sqrt{d-1} - o(1)$ als $n \rightarrow \infty$.

Er is een bodem. Geen oneindige familie d -reguliere grafen kan λ onder $2\sqrt{d-1}$ hebben.

Een graaf die haar haalt heet **Ramanujan**: $\lambda \leq 2\sqrt{d-1}$, optimale expansie. Ze bestaan — Lubotzky–Phillips–Sarnak en Margulis construeerden in 1988 oneindige families met diepe getaltheorie, en de naam komt van het Ramanujan-vermoeden waarop hun bewijs steunt. Marcus, Spielman en Srivastava gaven in 2013 een heel ander bestaansbewijs voor bipartiete Ramanujan-grafen van elke graad, met verstrengelde veeltermen.

De Petersen-graaf is Ramanujan: $d = 3$, $\lambda = 2$, en $2\sqrt{2} \approx 2,83$.

held ch32 The Petersen graph is Ramanujan (1 graphs)

Waarom het iemand kan schelen

Expanders zijn ijle objecten met de samenhang van dichte grafen, dus ze duiken op waar je goedkoop robuustheid nodig hebt:

- **Ontrandomiseren.** Een toevalswandeling op een expander raakt elke grote verzameling snel, dus $O(\log n)$ toevalsbits kunnen $O(n)$ vervangen in veel algoritmen.
- **Foutcorrigerende codes.** Expandercodes decoderen in lineaire tijd en benaderen de Shannon-limiet.
- **Netwerkontwerp.** Constante graad, constante diameter, en bestand tegen knooppuitval.
- **Eigenschapstesten.** Een graaf onderscheiden van een die ver van een eigenschap af ligt.
- **De PCP-stelling.** Expanders worden gebruikt in de kloofversterkingsstap, en dus in de onbenaderbaarheidsresultaten uit hoofdstuk 23.

Het terugkerende thema: **expliciete constructies zijn veel moeilijker dan bestaansbewijzen**. Een willekeurige d -reguliere graaf is met kans naar 1 een expander — een argument van één alinea in de stijl van hoofdstuk 24 — terwijl expliciete families tot 1988 duurden en gereedschap uit een ander vakgebied vergden. Dat is dezelfde kloof als bij de Ramsey-ondergrenzen in hoofdstuk 28, en het is de karakteristieke vorm van de nalatenschap van de probabilistische methode.

Probeer het

```
python -c "
import sys, math; sys.path.insert(0, '.')
from graphs.core import complete, cycle, petersen, complete_bipartite
from graphs.spectral import (spectral_expansion, mixing_lambda, is_ramanujan,
                             cheeger_constant, algebraic_connectivity)
print(f\" {'graph':<10} {'d':>2} {'expansion l':>12} {'mixing l':>9}
{'2sqrt(d-1)':>11} {'ram':>5} {'h(G)':>6}\")
for name, g in [('C6', cycle(6)), ('K4', complete(4)), ('K3,3',
complete_bipartite(3,3)),
                ('petersen', petersen())]:
    d = g.degree(0)
    print(f'  {name:<10} {d:>2} {spectral_expansion(g):>12.4f}
{mixing_lambda(g):>9.4f} '
          f' {2*math.sqrt(d-1):>11.4f} {str(is_ramanujan(g)):>5}
{cheeger_constant(g):>6.3f}')
"
```

graph	d	expansion l	mixing l	2sqrt(d-1)	ram	h(G)
C6	2	1.0000	2.0000	2.0000	True	0.667
K4	3	1.0000	1.0000	2.8284	True	2.000
K3,3	3	0.0000	3.0000	2.8284	True	1.667
petersen	3	2.0000	2.0000	2.8284	True	1.000

Kijk naar de rij $K_{3,3}$: expansie $\lambda = 0$ en menging $\lambda = 3$. Dezelfde graaf, hetzelfde spectrum, en de twee kolommen verschillen de hele graad — omdat de ene -3 uitsluit en de andere dat niet mag. Die ene rij is het onderscheid waarmee dit hoofdstuk opende.

Vergelijk C_6 en de Petersen-graaf. Beide hebben menging $\lambda = 2$ en beide heten formeel "Ramanujan", maar een cykel is in geen enkele nuttige zin een expander — en de tabel met één graaf per rij kan dat niet tonen. Expansie is een eigenschap van een *groeïende familie*, dus je moet haar zien groeien:

C_4	$h = 1.0000$	$(4/n = 1.0000)$
C_8	$h = 0.5000$	$(4/n = 0.5000)$
C_{12}	$h = 0.3333$	$(4/n = 0.3333)$
C_{14}	$h = 0.2857$	$(4/n = 0.2857)$

$h(C_n) = 4/n \rightarrow 0$. Je kunt een cykel altijd met twee kanten doorknippen, hoe groot hij ook wordt, dus de verhouding tussen rand en grootte verdwijnt. Een expanderfamilie houdt h van *onderen* begrensd door een constante, en geen enkele hoeveelheid inspectie van één eindige cykel onthult dat deze dat niet doet.

Dat is de valstrik die de tabel van dit hoofdstuk zet en de reden dat de definitie over een familie kwantificeert: **geen enkele eindige graaf is of is geen expander**. C_6 Ramanujan noemen is technisch juist en zegt niets over de familie waartoe hij behoort.

Waar je hierna heen gaat

Dit boek stopt hier. De natuurlijke vervolgen:

Structurele grafentheorie. Diestel, *Graph Theory*, is de standaard moderne referentie en behandelt minoren, samenhang en extremale theorie grondig.

Extremaal en probabilistisch. Alon en Spencer, *The Probabilistic Method*, is hét boek over het onderwerp van hoofdstuk 24 en gaat er ver voorbij — het lokale lemma, martingalen, entropie.

Spectraal. Spielmans collegenotities over spectrale en algebraïsche grafentheorie, en Chung, *Spectral Graph Theory*, voor de genormaliseerde Laplaciaan die dit boek oversloeg.

Algoritmen. Williamson en Shmoys over benaderen; Cygan e.a. over geparametriseerde algoritmen, de moderne behandeling van de tweede helft van hoofdstuk 23.

Open problemen die je nu kunt lezen: de waarden van $R(5,5)$ en $R(6,6)$; het probleem van Zarankiewicz voor bipartiete H (hoofdstuk 27); of $P = NP$; het reconstructievermoeden, dat een graaf op minstens drie knopen bepaald wordt door de multiverzameling van zijn knoop-verwijderde deelgrafen — open sinds 1942 en g nant eenvoudig te formuleren.

Oefeningen

1. Waarom is een cykel geen expander, ondanks dat hij samenhangend en regulier is?
2. Verifieer dat de Petersen-graaf Ramanujan is vanuit zijn spectrum $\{3, 1^5, (-2)^4\}$.
3. Leg uit waarom $-d$ behouden moet blijven voor het mengingslemma maar uitgesloten voor de Ramanujan-voorwaarde.
4. Geef het argument van   n alinea dat een willekeurige d -reguliere graaf waarschijnlijk een expander is, en zeg waarom dat er geen produceert.

Oplossingen in Bijlage E.

Kernpunten

- Een expander is een *familie* ijle reguliere grafen met van onderen begrensde expansie. Geen enkele eindige graaf is een expander.
- Cheeger (hoofdstuk 30) maakt expansie tot een spectrale voorwaarde: goede expansie is een grote spectrale kloof.
- Er zijn twee verschillende λ 's in omloop. Het mengingslemma behoudt $-d$; de Ramanujan-voorwaarde laat haar vallen. Ze verwarren breekt het mengingslemma onmiddellijk op $K_3, 3$.
- Het mengingslemma zegt dat kleine λ betekent dat de kanten verdeeld zijn alsof bij toeval — de precieze inhoud van "pseudotoevallig".
- Alon–Boppana legt een bodem bij $2\sqrt{d-1}$; Ramanujan-grafen halen haar, en de Petersen-graaf is er een.
- Willekeurige d -reguliere grafen zijn vrijwel zeker expanders; expliciete constructies duurden tot 1988 en vergden getaltheorie. Bestaan is eenvoudig, construeren is moeilijk — de vorm van dit hele laatste deel van het boek.

Bijlage A — Notatie

Elk symbool dat in dit boek gebruikt wordt, met het hoofdstuk dat het invoert.

De graaf zelf

Symbool	Betekenis	H.
$G = (V, E)$	een graaf: knopenverzameling en kantenverzameling	1
n	aantal knopen, $ V $	1
m	aantal kanten, $ E $	1
uv	de kant die u en v verbindt	1
$\bar{G}$	het complement	1
$G[S]$	de deelgraaf geïnduceerd op knopenverzameling S	1
$G - v, G - e$	een knoop verwijderen, een kant verwijderen	4
G / e	een kant samentrekken	17

Benoemde grafen

Symbool	Betekenis	H.
K_n	volledige graaf, $n(n-1)/2$ kanten	1
P_n	pad op n knopen, $n-1$ kanten	1
C_n	cykel op n knopen, n kanten, $n \geq 3$	1
$K_{\{a,b\}}$	volledig bipartiete graaf, ab kanten	1
$T(n, r)$	Turán-graaf: volledig r -delig, zo gelijk mogelijke delen	27

Knopen en graden

Symbool	Betekenis	H.
$N(v)$	de burens van v	1
$\deg(v)$	de graad van v , $ N(v) $	3
$\Delta(G)$	maximale graad	3
$\delta(G)$	minimale graad	3
$d(G)$	degeneratie	15

Afstand en samenhang

Symbool	Betekenis	H.
$d(u, v)$	afstand: lengte van een kortste $u-v$ -pad	4
$\kappa(G)$	knopensamenhang	12
$\lambda(G)$	kantensamenhang	12
INF	onbereikbaar, <code>float("inf")</code> in de code	10

Optimalisatieparameters

Symbool	Betekenis	H.
$\chi(G)$	chromatisch getal	15
$\omega(G)$	kliekgetal	15
$\alpha(G)$	onafhankelijkheidsgetal	21
$\tau(G)$	knopenoverdekkingsgetal	21
$h(G)$	Cheeger-constante / isoperimetrisch getal	30
$\text{tw}(G)$	boombreedte	31

De identiteit van Gallai verbindt er twee: $\alpha(G) + \tau(G) = n$ (hoofdstuk 21).

Matrices en spectra

Symbool	Betekenis	H.
A	adjacentiematrix	2
D	diagonale graadmatrix	30
$L = D - A$	de Laplaciaan	30
$\lambda_1 \leq \dots \leq \lambda_n$	eigenwaarden, oplopend	29
λ_2	algebraïsche samenhang, voor L	30
λ	grootste niet-triviale λ -eigenwaarde — twee definities, zie h. 32	32

Die laatste rij is een echte valstrik. `mixing_lambda` sluit alleen d uit; `spectral_expansion` sluit voor bipartiete grafen ook $-d$ uit. Hoofdstuk 32 legt uit waarom het mengingslemma de eerste nodig heeft en de Ramanujan-voorwaarde de tweede.

Asymptotiek en kans

Symbool	Betekenis	H.
O, Ω, Θ	boven-, onder- en scherpe asymptotische grenzen	2
$o(f)$	strikt kleinere orde dan f	27
$G(n, p)$	toevalsgraaf: elke kant onafhankelijk met kans p	25
$E[X]$	verwachting	24
mhw	met hoge waarschijnlijkheid: naar 1 als $n \rightarrow \infty$	25

Conventies die dit boek vastlegt

Teksten verschillen hierin, en een bewijs kan er stilzwijgend van afhangen.

- **Enkelvoudig als standaard.** Geen lussen, geen herhaalde kanten, geen richtingen, tenzij een hoofdstuk anders zegt. Richtingen komen in hoofdstuk 10, gewichten in hoofdstuk 9.
- **De lege graaf is samenhangend.** Hij heeft nul componenten, dus dit is een conventie, gekozen zodat "elke graaf is de disjuncte vereniging van zijn componenten" geen uitzondering nodig heeft (hoofdstuk 4).
- **Boombreedte trekt er één af,** zodat bomen boombreedte 1 hebben in plaats van 2 (hoofdstuk 31).
- **Eigenwaarden staan oplopend,** dus λ_n is de grootste.
- $e(S, T)$ telt **geordende** paren, dus een kant binnen $S \cap T$ wordt twee keer geteld — de conventie waarin het mengingslemma geformuleerd is (hoofdstuk 32).

Bijlage B — Woordenlijst

Definities zoals dit boek ze gebruikt, met het hoofdstuk dat elk begrip invoert. Waar een term concurrerende definities in de literatuur heeft, wordt het meningsverschil vermeld.

Aangrenzend. Twee knopen verbonden door een kant. → H. 1

Algebraïsche samenhang. De op één na kleinste Laplace-eigenwaarde λ_2 , ook Fiedler-waarde genoemd. Nul precies wanneer de graaf onsamenvhangend is. → H. 30

Alternerend pad. Een pad dat afwisselt tussen kanten buiten een koppeling en kanten erin. → H. 14

Bipartiet. Knopen gesplitst in twee verzamelingen met elke kant oversteken. Equivalent: geen oneven cykel; equivalent $\chi \leq 2$. → H. 16

Blad. Een knoop van graad 1. Elke boom op $n \geq 2$ knopen heeft er minstens twee. → H. 6

Boom. Vijf equivalente definities; samenhangend en cykelvrij is de gebruikelijke. → H. 6

Boombreedte. Minimale breedte over alle boomdecomposities, waarbij breedte de grootste zakgrootte min één is. NP-moeilijk te berekenen. → H. 31

Boomdecompositie. Een boom van zakken die aan drie voorwaarden voldoet; de derde — dat de zakken die een knoop bevatten een samenhangende deelboom vormen — draagt alle inhoud. → H. 31

Brug. Een kant waarvan verwijdering het aantal componenten verhoogt. Equivalent: een kant op geen enkele cykel. → H. 4, 12

Cheeger-constante. Het isoperimetrisch getal $h(G)$: het minimum over verzamelingen S van hoogstens halve grootte van het aantal kanten dat S verlaat gedeeld door $|S|$. → H. 30

Complement $\bar{G}$. Dezelfde knopen, kanten precies waar G er geen heeft. → H. 1

Component. Een equivalentieklasse van de relatie "verbonden door een pad". → H. 4

Cospectraal. Twee grafen met hetzelfde adjacentspectrum. Ze hoeven niet isomorf te zijn; $K_{1,4}$ en $C_4 + K_1$ zijn het kleinste paar. → H. 29

Degeneratie $d(G)$. De grootste k zodat elke deelgraaf een knoop van graad hoogstens k heeft. Geeft $\chi \leq d + 1$, nooit slechter dan $\Delta + 1$. → H. 15

Enkelvoudig. Geen lussen, geen herhaalde kanten. De standaard overal. → H. 1

Expander. Een *familie* d -reguliere grafen waarvan de Cheeger-constante van onderen begrensd blijft. Geen enkele eindige graaf is een expander. → H. 32

Geïnduceerde deelgraaf. $G[S]$: houd S en *alle* kanten tussen zijn leden. Verschilt van een deelgraaf, die kanten mag weglaten. Het onderscheid doet ertoe voor perfectie. → H. 1, 19

Gerichte graaf (digraaf). Kanten zijn geordende paren, **bogen** genoemd. Ingevoerd wanneer negatieve gewichten ongerichte kanten betekenisloos maken. → H. 10

Graad. Het aantal buuren. → H. 3

Graadrij. Graden in niet-stijgende volgorde. Een invariant, dus ze kan isomorfie weerleggen en nooit bewijzen. → H. 3, 5

Grafisch. Een rij gehele getallen die de graadrij van een enkelvoudige graaf is. Beslist door Havel–Hakimi of Erdős–Gallai. → H. 3

Hamiltoniaans. Heeft een cykel die elke knoop precies één keer bezoekt. NP-volledig, en zonder bekende goede karakterisering. → H. 20

Hypohamiltoniaans. Niet Hamiltoniaans, maar $G - v$ is Hamiltoniaans voor elke v . De Petersen-graaf is het standaardvoorbeeld. → H. 20

Invariant. Alles wat door isomorfie behouden blijft. Eenzijdig: verschillen weerleggen, overeenstemming bewijst niets. → H. 5

Isomorfie. Een bijectie van knopen die aangrenzendheid behoudt. In NP, niet bekend in P, en vermoedelijk niet NP-volledig. → H. 5

Kliek. Een verzameling paarsgewijs aangrenzende knopen. $\omega(G)$ is de grootste. → H. 21

Koordaal. Elke cykel van lengte minstens 4 heeft een koorde. Equivalent: heeft een perfecte eliminatievolgorde. Koordale grafen zijn perfect. → H. 19

Koppeling. Kanten waarvan er geen twee een knoop delen. → H. 14

Min-max-stelling. Een resultaat dat een maximum en een minimum gelijkstelt. Zwakke dualiteit ($\max \leq \min$) is gratis; de inhoud is altijd de andere richting. Menger, max-stroom min-snedes, König en Hall zijn één stelling in vier vermommingen. → H. 12–14

Minor. Verkrijgbaar door knopen te verwijderen, kanten te verwijderen, en kanten samen te trekken. → H. 17, 31

Omtrek. De lengte van een kortste cykel. De Petersen-graaf heeft omtrek 5. → H. 17

Onafhankelijke verzameling. Paarsgewijs niet-aangrenzende knopen. $\alpha(G)$ is de grootste. → H. 21

Perfect. $\chi(H) = \omega(H)$ voor elke geïnduceerde deelgraaf H . De kwantor maakt de klasse erfelijk. C_5 is de kleinste imperfecte graaf. → H. 19

Ramanujan. Een d -reguliere graaf met $\lambda \leq 2\sqrt{d-1}$ — optimale expansie volgens Alon–Boppana. De Petersen-graaf is er een. → H. 32

Regulier. Elke knoop heeft dezelfde graad. d -reguliere grafen hebben spectraalstraal precies d . → H. 29

Restnetwerk. Het stroomnetwerk uitgebreid met omgekeerde bogen die de huidige stroom dragen. De omgekeerde bogen zijn wat het algoritme een beslissing laat herzien. → H. 13

Rotatiesysteem. Een cyclische volgorde van burens bij elke knoop — de volledige combinatorische inhoud van een inbedding. → H. 17

Samenhangend. Precies één component. Dit boek noemt de lege graaf samenhangend — een conventie, geen gevolg. → H. 4

Simpliciale knoop. Een knoop wiens burens een kliek vormen. Elke koordale graaf heeft er een. → H. 19

Snede. Voor stroom een verdeling (S, T) met $s \in S$, $t \in T$; haar capaciteit telt alleen bogen van S naar T . Voor samenhang een verzameling waarvan verwijdering twee knopen scheidt. → H. 12, 13

Snede-eigenschap. De strikt lichtste kant over een willekeurige snede zit in elke minimale opspannende boom. "Strikt" draagt gewicht. → H. 9

Opspannende boom. Een deelgraaf die een boom is en elke knoop bevat. K_n heeft er $n^{(n-2)}$ gelabelde. → H. 7

Vlak. Tekenbaar zonder kruisingen. Een eigenschap van de graaf, want ze kwantificeert over alle tekeningen. → H. 17

Drempel. Voor een monotone eigenschap in $G(n, p)$ de functie p^* waaronder ze vrijwel nooit geldt en waarboven vrijwel altijd. Elke monotone eigenschap heeft er een. → H. 25

Wandeling / pad / cykel. Een wandeling herhaalt; een pad niet; een cykel sluit. Elke wandeling bevat een pad met dezelfde eindpunten, en dat maakt samenhang tot een equivalentierelatie. → H. 4

Weisfeiler–Leman (kleurverfijning). Een snelle, correcte, onvolledige isomorfieheuristiek. Blind voor elke twee reguliere grafen van dezelfde grootte en graad. → H. 5

Bijlage C — Verder lezen

Waar je heen gaat voor elk deel van dit boek, en waar elke bron werkelijk goed in is.

Algemene naslagwerken

Diestel, *Graph Theory*. Het standaard moderne tekstboek voor gevorderden, en degene om te bezitten als je er één bezit. Volledig en streng over samenhang, minoren, extremale theorie en het Robertson–Seymour-programma. Vrij beschikbaar via de site van de auteur. → Delen I–IV, VII

Bondy en Murty, *Graph Theory*. Breder en vriendelijker dan Diestel, met meer algoritmische inhoud en een grote voorraad oefeningen. → Delen I–V

West, *Introduction to Graph Theory*. De toegankelijkste van de drie, en de beste bron van uitgewerkte voorbeelden als de bewijzen in dit boek te snel gingen. → Delen I–IV

Per onderwerp

Alon en Spencer, *The Probabilistic Method*. Hét boek over het onderwerp van hoofdstuk 24, en het gaat ver voorbij wat dit boek behandelt — het lokale lemma van Lovász, martingaalconcentratie, entropiemethoden, en de tweedemomentargumenten waarnaar hoofdstuk 26 alleen gebaarde. → H. 24–26, 28

Bollobás, *Random Graphs*. Het naslagwerk voor de hoofdstukken 25 en 26. Drempels, het kritieke venster bij $c = 1$, en het $\Theta(n^{2/3})$ -gedrag dat dit boek alleen vermeldde. → H. 25, 26

Bollobás, *Extremal Graph Theory*. Turán-achtige problemen in de diepte, inclusief de bipartiete gevallen waar Erdős–Stone zwijgt. → H. 27

Graham, Rothschild en Spencer, *Ramsey Theory*. Van der Waerden, Hales–Jewett, en het algemene programma "volledige wanorde is onmogelijk". → H. 28

Godsil en Royle, *Algebraic Graph Theory*. De grondige behandeling van de hoofdstukken 29 en 30 — verstrengeling, sterk reguliere grafen, en het materiaal over automorfismegroepen dat dit boek volledig oversloeg. → H. 29, 30

Chung, *Spectral Graph Theory*. Opgebouwd rond de **genormaliseerde** Laplaciaan $L = I - D^{-1/2} A D^{-1/2}$, die dit boek niet gebruikte en die het juiste object is voor irreguliere grafen. → H. 30

Spielman, collegenotities over spectrale en algebraïsche grafentheorie. Vrij beschikbaar, modern, en de helderste route naar de ongelijkheid van Cheeger en spectraal clusteren. → H. 30, 32

Hoory, Linial en Wigderson, "Expander graphs and their applications" (*Bulletin of the AMS*, 2006). Een overzichtsartikel, en het beste startpunt voor hoofdstuk 32. → H. 32

Algoritmen

Cormen, Leiserson, Rivest en Stein, *Introduction to Algorithms*. Het naslagwerk voor alles in deel III. Het max-stroomnetwerk in hoofdstuk 13 komt daarvandaan. → H. 8–14

Williamson en Shmoys, *The Design of Approximation Algorithms*. De eerste helft van hoofdstuk 23, naar behoren: LP-afronding, primaal-duaal, en de onbenaderbaarheidsresultaten die hier alleen geciteerd werden. Vrij beschikbaar. → H. 23

Cygan e.a., *Parameterized Algorithms*. De tweede helft van hoofdstuk 23. Begrensde zoekbomen, kernelisatie, boombreedte-DP naar behoren gedaan — inclusief het dynamisch programmeren dat hoofdstuk 31 beschreef maar niet implementeerde. Vrij beschikbaar. → H. 23, 31

Kleinberg en Tardos, *Algorithm Design*. De beste uitleg die er is over waarom stroomreducties werken, als de tabel met coderingen in hoofdstuk 13 aanvoelde als een lijst trucs. → H. 13

Wat dit boek bewust oversloeg

Elk hiervan is een echt gat, geen vergetelheid:

- **Hopcroft–Tarjan-vlakheidstest** in $O(n)$. Hoofdstuk 17 doorzoekt in plaats daarvan rotatiesystemen, wat exponentieel is en zichtbaar de stelling zelf.
- **Edmonds' bloesemalgoritme** voor algemene koppeling. Hoofdstuk 14 geeft alleen het bipartiete geval en toont waar het breekt.
- **Het algoritme van Johnson** voor alle-parenkortstepaden met negatieve bogen op ijle grafen. Hoofdstuk 11 beschrijft de herwegingstruc zonder haar te implementeren.
- **Boombreedte-DP** achter de stelling van Courcelle. Hoofdstuk 31 zegt waarom.
- **De genormaliseerde Laplaciaan**, het juiste spectrale object voor irreguliere grafen.
- **Gerichte grafentheorie** voorbij kortste paden en stroom: sterke samenhang, toernooien, en de gerichte tegenhangers van het grootste deel van de delen IV–VI.

De vierkleurenstelling

Een eigen vermelding waard, aangezien hoofdstuk 18 haar niet kon bewijzen.

- De oorspronkelijke artikelen van Appel en Haken uit 1976, en de vereenvoudiging uit 1997 van Robertson, Sanders, Seymour en Thomas, die het aantal configuraties van 1936 tot 633 terugbracht.
- **Gonthier, "Formal Proof — The Four-Color Theorem"** (*Notices of the AMS*, 2008), over de volledig machinaal gecontroleerde Coq-versie. Het interessantste van de drie om te lezen, omdat het gaat over wat het betekent dat een bewijs geverifieerd is in plaats van begrepen.

Open problemen uit dit boek

Vermeld in de hoofdstukken, en alle nog open:

- $R(5, 5)$, waarvan alleen bekend is dat het in $[43, 46]$ ligt. → H. 28
- Het probleem van Zarankiewicz: het extremale getal voor bipartiete H . → H. 27
- Het reconstructievermoeden: een graaf op ≥ 3 knopen wordt bepaald door de multiverzameling van zijn knoop-verwijderde deelgrafen. Open sinds 1942, en gênant eenvoudig te formuleren. → H. 5
- Of grafenisomorfie in P zit. → H. 5

- Een klik van grootte $(1+\varepsilon) \log_2 n$ vinden in een toevalsgraaf, terwijl we precies weten hoe groot het antwoord is. → H. 25
- P tegenover NP. → H. 22

Reeksen

De aantallen in dit boek komen uit de OEIS, en ertegen controleren is de goedkoopst mogelijke verificatie van een opsommer:

- **A000088** — grafen op n knopen op isomorfie na: 1, 2, 4, 11, 34, 156, 1044. → H. 5
- **A000272** — gelabelde bomen, n^{n-2} : 1, 1, 3, 16, 125, 1296. → H. 7

Bijlage E — Oplossingen bij de oefeningen

Elk numeriek antwoord hier is tegen de eigen code van het boek gedraaid en niet beweed. Waar een oefening je iets laat zoeken, wordt één getuigenis gegeven; er bestaan er meestal meer.

Hoofdstuk 1 — Wat een graaf is

1. K_6 heeft $6 \cdot 5 / 2 = 15$ kanten: elk van de 6 knopen ontmoet er 5, en elke kant wordt twee keer geteld. `complete(6).m` geeft 15.
2. Tel de graden op: $3 \times 10 = 30$, en het handdruklemma (hoofdstuk 3) zegt dat dit $2m$ is, dus $m = 15$. Tellen is niet nodig.
3. $[(0,1), (1,2), (2,0)]$ en $[(2,0), (0,1), (1,2)]$ zijn dezelfde gelabelde graaf — de volgorde in de kantenlijst doet er niet toe, want E is een verzameling. $[(0,1), (1,2), (2,3)]$ op vier knopen is een andere graaf, en $[(0,2), (2,1), (1,0)]$ op drie is *weer dezelfde* graaf, want dezelfde paren zijn verbonden.
4. Drie paarsgewijze kanten zijn niet te onderscheiden van drie afzonderlijke samenwerkingen tussen twee auteurs die elkaar nooit ontmoetten. De informatie "deze drie werkten *als groep* samen" is niet uit de driehoek terug te halen. Een **hypergraaf**, waarvan de kanten willekeurige deelverzamelingen zijn, behoudt haar.

Hoofdstuk 2 — Representaties

1. Verwacht aantal kanten $= p \cdot C(1000, 2) = 0,01 \times 499500 \approx 4995$. De matrix heeft $1000^2 = 1.000.000$ plaatsen, dus ongeveer 99,5% ervan is nul.
2. De adjacentiematrix. Driehoeken door v tellen betekent buurverzamelingen snijden, en met bitmaskerrijen is dat één & plus een popcount — 64 knopen per machinewoord. Hoofdstuk 2 mat een winst van factor 48 bij $p = 0,5$.
3. De rij is een Python-geheel getal van willekeurige precisie, dus `rows[u] >> v` schuift een getal van 600 bits of meer en raakt elk cijfer onder v . Dat is $O(n)$, niet $O(1)$. De meting toonde de kloof groeien van $1,7\times$ bij $n = 64$ tot ongeveer $6\times$ bij $n = 16384$.
4. Een Python-set draagt ruwweg twee kilobyte overhead voordat hij iets bevat, en er zijn er n ; een geheel getal van 600 bits is ongeveer 100 byte. In C zou een adjacentielijst bij $p = 0,05$ werkelijk kleiner zijn — het resultaat is een feit over de containers van Python, niet over adjacentielijsten.

Hoofdstuk 3 — Graad

1. Nee. Negen knopen van graad 3 geven een graadsom van 27, wat oneven is, in tegenspraak met $\sum \deg(v) = 2m$. Equivalent: het aantal knopen van oneven graad moet even zijn, en 9 is oneven.
2. Niet grafisch. Havel–Hakimi: verwijder de 4 en trek één af van de volgende vier plaatsen, wat $[2, 1, 0, -1]$ geeft — negatief, dus het faalt. Intuïtief moet een knoop van graad 4 in een graaf met 5 knopen aan alles grenzen, dus niets kan graad 0 hebben.
3. $\sum \deg(v) = 3 \times 10 = 30$, dus $m = 15$.

4. Op één knoop is er geen paar om te vergelijken, dus de conclusie is niet eens betekenisvol. De duivenhokstap van het bewijs heeft n graden nodig uit een verzameling van grootte $n - 1$, en dat vergt $n \geq 2$ om de verzameling niet-leeg te laten zijn. De verificatie geeft `None` terug voor $n < 2$ in plaats van `True`, want de stelling zegt daar niets.

Hoofdstuk 4 — Wandelingen, paden, samenhang

1. In `C5` met knopen `0..4`: de wandeling `0,1,0,1` herhaalt knopen en kanten, dus ze is geen pad. Het pad `0,1,2` is geen cykel — het keert niet terug naar zijn start.
2. $(A^2)_{\{vv\}} = \deg(v)$. Een gesloten wandeling van lengte 2 vanuit v gaat naar een buur en meteen terug, dus er is er precies één per buur.
3. $n - 1$, precies bereikt door de bomen (hoofdstuk 6).
4. Twee paden aan elkaar plakken geeft een *wandeling*, die knopen kan herhalen. Het wandeling-naar-padlemma — een kortste wandeling tussen twee knopen herhaalt niets — zet haar terug om in een pad. Zonder dat lemma faalt transitiviteit en is "samenhangscomponent" niet welgedefinieerd.

Hoofdstuk 5 — Isomorfie

1. Nee. `C6` is samenhangend en twee disjuncte driehoeken niet, en samenhang blijft onder isomorfie behouden. Geen berekening nodig.
2. Isomorfe grafen moeten in elke invariant overeenkomen, dus onenigheid bewijst niet-isomorfie; maar veel niet-isomorfe grafen komen in een gegeven invariant overeen, dus overeenstemming bewijst niets.
3. Verfijning herkleurt een knoop met zijn eigen kleur plus de multiverzameling kleuren van zijn burens. In een d -reguliere graaf begint elke knoop identiek en heeft hij d identieke burens, dus de partitie is al stabiel bij ronde nul en splitst nooit.
4. `len(all_graphs_up_to_iso(4))` geeft 11 — de rij is 1, 2, 4, 11, 34, 156 (OEIS A000088).

Hoofdstuk 6 — Bomen

1. 11 kanten: een boom heeft $m = n - 1$.
2. Twee, bereikt door het pad `Pn`. Het langste-padargument toont dat beide uiteinden van een maximaal pad bladeren zijn, dus er zijn er altijd minstens twee.
3. Precies één. De twee eindpunten hadden al een uniek pad ertussen, en de nieuwe kant sluit precies dat pad tot een cykel.
4. Nee. `C4` is bipartiet en heeft een cykel, dus hij is geen boom. Bipartiet is veel zwakker: het verbiedt alleen oneven cycli.

Hoofdstuk 7 — Opspannende bomen en de formule van Cayley

1. 16, en Cayley geeft $4^{4-2} = 4^2 = 16$. `len(spanning_trees(complete(4)))` bevestigt het.

2. $[1, 2]$. Verwijder het kleinste blad (0) en noteer zijn buur 1; dan is het kleinste blad 1, met buur 2. Er blijven twee knopen over, dus we stoppen — de rij heeft lengte $n - 2 = 2$.
3. Dat een knoop $\deg(v) - 1$ keer voorkomt betekent dat een blad, met $\deg = 1$, nul keer voorkomt. De labels die in de rij ontbreken zijn dus precies de bladeren.
4. 1296 telt bomen op een *vaste gelabelde* knopenverzameling; 6 telt isomorfieklassen. Veel verschillende labelingen geven dezelfde vorm — het onderscheid van hoofdstuk 5 tussen gelijkheid en isomorfie, toegepast op tellen.

Hoofdstuk 8 — Doorlopen

1. BFS. DFS is niet slechts trager — het geeft *een* pad terug, dat willekeurig veel langer kan zijn dan het kortste. In het voorbeeld van hoofdstuk 8 bereikt DFS een aangrenzende knoop als laatste, na een omweg van zeven kanten.
2. Beide zijn $O(n + m)$: elke knoop komt één keer in de wachtrij of op de stapel, en elke kant wordt twee keer bekeken, één keer vanaf elk eindpunt. De houder verandert het aantal operaties niet, alleen hun volgorde.
3. Een oneven cykel. De twee eindpunten liggen op gelijke afstand van de bron, dus de twee boompaden plus deze kant sluiten een cykel van oneven lengte. Dit is precies het algoritme van hoofdstuk 16.
4. De standaardrecursielimiet van Python is 1000 frames, dus een recursieve DFS crasht op een padgraaf van tienduizend knopen — een graaf die enkel lang is en niet groot.

Hoofdstuk 9 — Minimale opspannende bomen

1. De **strikt** lichtste kant over een willekeurige snede behoort tot elke minimale opspannende boom. "Strikt" is het dragende woord: bij gelijke gewichten zit elke gelijk gewogen kant in *een* MOB en hoeft geen ervan in *alle* te zitten.
2. C_4 met alle gewichten 1 en een koorde — of de vierknoopsgraaf $\{(0,2), (0,3), (1,2), (1,3)\}$ met eenheidsgewichten, die hoofdstuk 9 door uitputtend zoeken als het kleinste geval vond. Kruskal geeft $\{(0,2), (0,3), (1,2)\}$ en Prim vanaf knoop 1 geeft $\{(0,2), (1,2), (1,3)\}$, beide van gewicht 3.
3. Een minimaal opspannend **bos** — één boom per component. Het faalt niet; de union-find voegt eenvoudigweg nooit over componenten heen samen.
4. Wanneer alle kantgewichten verschillend zijn. Dan kan geen gelijkstand optreden, geldt de snede-eigenschap overal met striktheid, en is de MOB uniek.

Hoofdstuk 10 — Kortste paden

1. De laatste stap, waar gezegd wordt dat de rest van een pad vanaf de nieuw bereikte knoop naar het doel "er alleen bij optelt". Met een negatieve boog kan die rest het totaal verlagen, en de ongelijkheid faalt.
2. $O(nm)$, tegen Dijkstra's $O(m \log n)$. Het relaxeert elke boog $n - 1$ keer omdat het niets kan afhandelen: zonder niet-negativiteit is er geen knoop die het definitief kan verklaren.

3. Een negatieve cykel bereikbaar vanuit de bron. Na $n - 1$ ronden kan geen enkelvoudig pad meer verbeteren, dus elke verdere verbetering vergt een wandeling die met winst meer dan $n - 1$ bogen gebruikt.
4. Die bestaat niet. Nog eens rond de cykel gaan verlaagt het totaal onbegrensd, dus het infimum is $-\infty$ en de vraag is slecht gesteld in plaats van slechts moeilijk.

Hoofdstuk 11 — Afstand tussen alle paren

1. $O(n^3)$, en helemaal geen datastructuur — drie geneste lussen over een vlakke array, zonder prioriteitswachtrij en zonder allocatie.
2. k indexeert de inductie op welke knopen een pad mag doorkruisen; i en j doorlopen binnen een vaste fase. Met i buitenste kan $\text{dist}[0][1]$ definitief worden terwijl $\text{dist}[3][1]$ nog oneindig is. Hoofdstuk 11 mat dat de kapotte versie afweek op 942 van de 4000 willekeurige digrafen terwijl ze op een padgraaf correct was.
3. Die knoop ligt op een negatieve cykel. Merk op dat dit verschilt van de test van Bellman–Ford, die vraagt of er een bereikbaar is vanuit een gegeven bron.
4. $\text{Digraph}(2, [(0,1,1)])$: $d(0,1) = 1$ en $d(1,0) = \infty$. Gerichte afstand is een quasimetriek, geen metriek.

Hoofdstuk 12 — Samenhang en de stelling van Menger

1. Elk s – t -pad ontmoet de snede, en inwendig disjuncte paden kunnen geen snedeknoop delen, dus $\#\text{paden} \leq |\text{snede}|$. Ze bewijst op zichzelf niets omdat ze geen reden geeft dat de grens bereikt wordt — genoeg natuurlijke min-max-paren hebben een echte kloof, $\omega \leq \chi$ onder meer.
2. Zijn s en t aangrenzend, dan kan geen verzameling andere knopen ze scheiden, dus er is helemaal geen s – t -knopensnede. De grootte rechts is ongedefinieerd, niet groot.
3. Beide ongelijkheden kunnen strikt zijn, al niet in de eenvoudigste voorbeelden. Twee driehoeken verbonden door een lang pad geven $\delta = 2$, $\lambda = 1$, $\kappa = 1$ — geverifieerd met `edge_connectivity` en `vertex_connectivity` — dus $\kappa = \lambda < \delta$. Voor $\kappa < \lambda$ neem je twee kopieën van K_4 die één knoop delen: die snijknoop geeft $\kappa = 1$, terwijl $\lambda = 3$ want je moet alle drie de kanten bij een knoop vernietigen, en $\delta = 3$.
4. De inwendige boog begrenst de stroom door de knoop op 1, dus twee paden kunnen hem niet delen — en dat is wat de paden inwendig disjunct maakt. Kanten krijgen oneindige capaciteit omdat de stelling knopen telt en geen kanten, en een kant mag nooit de flessenhals zijn.

Hoofdstuk 13 — Max-stroom min-snede

1. Ze laten het algoritme een eerdere beslissing terugdraaien. Zonder ze kan een vroeg gulzig pad een beter later pad blokkeren en heeft de zoektocht geen manier om te herleiden — gulzig paden duwen is dan eenvoudigweg fout, niet slechts suboptimaal.
2. Met willekeurige paden en irrationale capaciteiten kan Ford–Fulkerson eeuwig doorlopen en naar een waarde onder het maximum convergeren. Met gehele capaciteiten termineert het maar kan het tijd kosten

evenredig met de *waarde* van de stroom, wat exponentieel is in de invoergrootte. Kortste paden (BFS) geven $O(nm^2)$ ongeacht.

3. Gehele capaciteiten geven een gehele maximale stroom, want elke vermeerdering duwt een gehele flessenhals. Het doet ertoe omdat Menger en koppeling nodig hebben dat het antwoord een *verzameling* paden of kanten is; een fractioneel optimum kon niet als zodanig afgelezen worden.

4. Niets. De capaciteit van een snede telt alleen bogen van de bronzijde naar de putzijde. In het voorbeeld van hoofdstuk 13 loopt de boog $(3, 2)$ met capaciteit 9 achterwaarts over de snede en draagt ze nul bij.

Hoofdstuk 14 — Koppeling

1. Een alternerend pad — kanten afwisselend buiten en binnen M — waarvan beide eindpunten ongekoppeld zijn. Omkeren verhoogt $|M|$ met precies één, want het heeft één niet-koppelingskant meer dan koppelingskanten.

2. M is maximaal dan en slechts dan als er geen M -vermeerderend pad bestaat. Het bewijs gebruikt **symmetrisch verschil**: $M \Delta N$ heeft maximale graad 2, dus zijn componenten zijn paden en even cykels, en is $|N| > |M|$ dan moet een component een vermeerderend pad zijn.

3. Een oneven cykel. Een alternerende wandeling kan terugkeren naar een knoop met tegengestelde pariteit, en de zoektocht kan een vermeerderend pad niet van een lus onderscheiden. Edmonds' bloesemalgoritme lost het op door oneven cykels samen te trekken. Het getuigenis van hoofdstuk 14 is C_7 ; C_3 en C_5 leggen de fout niet bloot.

4. Maximale koppeling 1, minimale knopenoverdekking 2 — geverifieerd door uitputtend zoeken. Ze verschillen omdat een driehoek niet bipartiet is, en de stelling van König bipartietheid vereist. Dit is het kleinste tegenvoorbeeld.

Hoofdstuk 15 — Kleuring

1. Een geldige kleuring verdeelt de knopen in kleurklassen, elk een onafhankelijke verzameling van grootte hoogstens $\alpha(G)$. n knopen bedekken vergt dus minstens $n / \alpha(G)$ klassen.

2. C_5 : $\omega = 2$ (driehoekvrij) en $\chi = 3$ (oneven cykel). Het is de kleinste zo'n graaf — elke graaf op hoogstens 4 knopen is perfect, zoals de uitputtende controle van hoofdstuk 19 bevestigt.

3. Orden de knopen volgens de degeneratie-eliminatie en tel: elke knoop heeft hoogstens d burens die al geplaatst zijn wanneer hij verwijderd wordt, dus sommeren over de knopen geeft $m \leq d \cdot n$.

4. De kroongraaf op $2n = 8$ knopen heeft degeneratie 3, dus de grens garandeert slechts $d + 1 = 4$ kleuren — en gulzig in de afwisselende volgorde gebruikt er precies 4. De grens wordt gerespecteerd; ze is alleen veel zwakker dan $\chi = 2$.

Hoofdstuk 16 — Bipartiete grafen

1. Is hij bipartiet, dan is de grootste zijde een onafhankelijke verzameling met minstens de helft van de knopen, en dat erft elke deelgraaf. Omgekeerd, heeft een deelgraaf geen zo'n onafhankelijke verzameling, dan bevat die deelgraaf een oneven cykel, dus is de graaf niet bipartiet.

2. $\lfloor n^2/4 \rfloor$, bereikt door de gebalanceerde volledig bipartiete graaf $K_{\lfloor n/2 \rfloor, \lfloor n/2 \rfloor}$. Voor $n = 6$ is dat $K_{3,3}$ met 9 kanten. Dit is de stelling van Mantel (hoofdstuk 27), want bipartiete grafen zijn driehoekvrij.
3. De Petersen-graaf heeft omtrek 5 en is niet bipartiet. Elke graaf die een geïnduceerde C_5 bevat en geen kortere cykel werkt, bijvoorbeeld C_7 , of de vijfcykel met een hangende knoop.
4. Driehoekvrijheid is *lokaal* — bekijk elk drietal knopen, $O(n^3)$ — terwijl bipartietheid *globaal* is, maar BFS berekent haar in $O(n + m)$ omdat de niveauctuur al het werk in één keer doet. De goedkopere test levert de sterkere eigenschap omdat één doorloop een globale beperking voortplant.

Hoofdstuk 17 — Vlakheid

1. Was elke graad minstens 6, dan $2m = \sum \deg(v) \geq 6n$, dus $m \geq 3n$, in tegenspraak met $m \leq 3n - 6$.
2. K_5 min een kant heeft $n = 5$, $m = 9 \leq 3 \cdot 5 - 6 = 9$, en `is_planar` bevestigt dat hij vlak is. Eén kant weghalen is precies genoeg.
3. De Petersen-graaf heeft omtrek 5, dus elk vlakdeel in elke inbedding zou minstens 5 kanten nodig hebben, wat $2m \geq 5f$ geeft. Met $n = 10$, $m = 15$ dwingt Euler $f = 7$ af, maar $2 \cdot 15 = 30 < 35 = 5 \cdot 7$. Tegenspraak.
4. Eén. Een boom heeft $m = n - 1$, dus Euler geeft $f = 2 - n + (n-1) = 1$ — alleen het buitenvlak. `planar_face_count(path(5))` geeft 1.

Hoofdstuk 18 — De vijf- en de vierkleurenstelling

1. $m \leq 3n - 6$ dwingt een knoop van graad hoogstens 5 af (oefening 17.1), en dat geldt voor elke deelgraaf omdat deelgrafen van vlakke grafen vlak zijn. Vlakke grafen zijn dus 5-degeneraat, en de grens van hoofdstuk 15 geeft $\chi \leq d + 1 = 6$.
2. De laatste stap, waar gezegd wordt dat een pad van v_2 naar v_4 het v_1-v_3 -pad niet kan kruisen. Kruisingen bestaan niet in een vlakke inbedding; dat is de enige topologische invoer in het argument.
3. De graaf K_4 met één knoop verdubbeld — bijvoorbeeld K_5 min één kant — heeft $\chi = 4$ en is vlak, zoals hierboven geverifieerd. Elke vlakke triangulatie die K_4 bevat werkt ook.
4. Na de eerste Kempe-verwisseling zijn de kleuren van de burens van v veranderd, dus de configuratie die de tweede verwisseling rechtvaardigde hoeft niet meer te gelden. De twee verwisselingen kunnen op elkaar inwerken: de ene uitvoeren kan de belemmering hercreëren die de andere moest wegnemen. Kempe nam onafhankelijkheid aan en Heawood vond het geval waarin dat faalt.

Hoofdstuk 19 — Perfecte en koordale grafen

1. C_5 heeft geen driehoek, dus $\omega = 2$. Hij is een oneven cykel, dus 2 kleuren zijn onmogelijk en 3 volstaan, wat $\chi = 3$ geeft. De uitputtende routines van het boek geven precies dit.
2. De knopen van een intervalgraaf zijn intervallen, aangrenzend wanneer ze overlappen. Neem in een cykel van lengte ≥ 4 het interval met het meest linkse rechteruiteinde; zijn twee cykelburen overlappen het allebei, dus ze overlappen elkaar — een koorde.

3. Omdat de klasse **erfelijk** moet zijn om bruikbaar te zijn, en erfelijke klassen zijn precies die welke door verboden geïnduceerde deelgrafën definieerbaar zijn. Alleen $\chi(G) = \omega(G)$ eisen laat C_5 plus een disjuncte K_3 toe, die $\chi = \omega = 3$ heeft en toch de canonieke imperfecte graaf bevat.
4. P_4 (het pad op 4 knopen) is zelfcomplementair, en hij is perfect — zelfs koordaal, dus de stelling van hoofdstuk 19 is van toepassing. C_5 is de zelfcomplementaire graaf die *niet* perfect is.

Hoofdstuk 20 — Hamiltoniciteit

1. Een Hamiltoniaanse cykel in een bipartiete graaf wisselt tussen de zijden, dus hij bezoekt er evenveel van elk — wat $a = b$ afdwingt. Omgekeerd heeft $K_{\{a,a\}}$ met $a \geq 2$ een voor de hand liggende alternerende cykel. `is_hamiltonian(complete_bipartite(2,3))` is `False` en `(3,3)` is `True`.
2. Een Hamiltoniaanse cykel geeft twee inwendig disjuncte paden tussen elk paar, dus de graaf is 2-samenhangend per Menger (hoofdstuk 12), en een 2-samenhangende graaf heeft geen snijknoop.
3. Is elke graad minstens $n/2$, dan heeft elk tweetal knopen graadsom minstens n , dus de hypothese van Ore geldt. Uitputtend zoeken geeft het kleinste voorbeeld dat aan Ore voldoet en niet aan Dirac: op 5 knopen, kanten $\{(0,2), (0,3), (0,4), (1,2), (1,3), (1,4), (2,3)\}$ met graadrij $[3,3,3,3,2]$. Knoop 4 heeft graad $2 < 5/2$, dus Dirac faalt; elk niet-aangrenzend paar sommeert nog altijd tot minstens 5, dus Ore geldt — en de graaf is inderdaad Hamiltoniaans.
4. k knopen uit een Hamiltoniaanse graaf verwijderen breekt de cykel in hoogstens k bogen, dus er blijven hoogstens k componenten over. De ster $K_{1,3}$ faalt hieraan: verwijder het middelpunt en er blijven drie componenten over uit één verwijdering, dus hij is niet Hamiltoniaans.

Hoofdstuk 21 — Kliken, onafhankelijke verzamelingen, overdekkingen

1. C_6 : $\alpha = 3$, $\tau = 3$, $\alpha + \tau = 6 = n$. $\omega = 2$, want C_6 is driehoekvrij.
2. C_5 heeft $\omega = 2$ en $\alpha = 2$, dus $\omega \cdot \alpha = 4 < 5 = n$. Dit is het kleinste tegenvoorbeeld; elke graaf op hoogstens 4 knopen voldoet aan de ongelijkheid.
3. $\tau(G)$ is gelijk aan de grootte van een maximale koppeling, per de stelling van König (hoofdstuk 14).
4. Het complement van een ijle graaf is dicht: $m = O(n)$ wordt $\Theta(n^2)$. Een algoritme waarvan de looptijd van m afhangt vertraagt dus met een factor n , ook al is de reductie zelf polynomiaal. De complexiteitstheorie rekent de reductie als goedkoop; je processor niet.

Hoofdstuk 22 — NP-moeilijkheid

1. Herleid een bekend moeilijke A naar B . Dat toont dat B oplossen A zou oplossen, dus B erft de moeilijkheid van A . B naar A herleiden toont slechts dat B niet moeilijker is dan iets moeilijks, en dat geldt ook voor elk probleem in P .
2. Zonder tegenstrijdigheidskanten kon een onafhankelijke verzameling x uit de driehoek van de ene clause kiezen en $\neg x$ uit die van een andere. De gekozen literalen kwamen dan met geen enkele consistente toekenning overeen, dus de ($\Rightarrow$)-richting faalt.

3. Zonder de driehoeken kon een onafhankelijke verzameling van grootte k verschillende literalen uit één clause nemen en geen uit een andere, waardoor die clause onvervuld blijft. De driehoeken zijn wat precies één lokaal per clause afdwingt.
4. Nee. NP-moeilijkheid is een uitspraak over het slechtste geval over *alle* invoeren. Beperken tot bipartiete invoer is een ander probleem, en dat het eenvoudig is zegt niets over het algemene — dit is het derde antwoord uit de lijst van hoofdstuk 23.

Hoofdstuk 23 — Leven met hardheid

1. Het algoritme voegt beide eindpunten toe van elke kant die het neemt, en het neemt $|M|$ kanten, dus het geeft precies $2|M|$ knopen terug. Dat M **maximaal** is betekent dat er geen kant bij kan, dus elke kant van G raakt een gekoppelde knoop — en dat is precies wat het resultaat tot een overdekking maakt.
2. Eén kant: $OPT = 1$ en de heuristiek geeft beide eindpunten, dus 2. Algemener elke perfecte koppeling op $2k$ knopen zonder andere kanten.
3. Voor $n = 10^6$ en $k = 10$: $2^{k(n+m)} \approx 1024 \cdot 10^6 \approx 10^9$ operaties, tegen $n^k = 10^{60}$. De eerste is in seconden klaar; de tweede overtreft het aantal atomen in het waarneembare heelal.
4. De equivalentie van hoofdstuk 21 behoudt *exacte* antwoorden, geen *parameters*. Een knopenoverdekking van grootte k komt overeen met een onafhankelijke verzameling van grootte $n - k$, dus de kleine parameter aan de ene kant wordt een enorme aan de andere. Geparametriseerde complexiteit is gevoelig voor welke grootte k heet, en de reductie behoudt haar niet.

Hoofdstuk 24 — De probabilistische methode

1. Kleur elke knoop onafhankelijk rood of blauw met kans $1/2$; elke kant steekt over met kans $1/2$; per lineariteit van de verwachting is het verwachte aantal oversteken $m/2$; een of andere uitkomst haalt minstens het gemiddelde. Onafhankelijkheid wordt nergens gebruikt — lineariteit geldt hoe dan ook, en de kantgebeurtenissen zijn werkelijk gecorreleerd wanneer kanten een knoop delen.
2. Richt elke kant willekeurig. Elk van de $n!$ knoopordeningen is een Hamiltoniaans pad met kans $2^{-(n-1)}$, dus het verwachte aantal is $n!/2^{n-1}$, en een of ander toernooi haalt minstens het gemiddelde.
3. Neem elke knoop onafhankelijk met kans p op. Het verwachte aantal overlevende knopen is pn en van overlevende kanten p^2m . Verwijder één eindpunt per overlevende kant: de rest is onafhankelijk en heeft verwachte grootte $pn - p^2m$. Optimaliseren bij $p = n/(2m)$ geeft $n^2/(4m)$.
4. De snedegrens laat zich ontrandomiseren met voorwaardelijke verwachtingen omdat de verwachting *lokaal* berekend en vergeleken kan worden, knoop voor knoop. Voor Ramsey is er geen bekende manier om de voorwaardelijke verwachting van "geen monochromatische K_k " efficiënt te evalueren, dus de gulzige stap is niet te zetten.

Hoofdstuk 25 — Toevalsgrafen

1. $E[\text{driehoeken}] = C(n,3)p^3 \approx n^3p^3/6$. Dit gaat naar nul wanneer $p \ll 1/n$ en naar oneindig wanneer $p \gg 1/n$, dus de drempel is $1/n$.

2. Een variabele kan een groot gemiddelde hebben en toch bijna altijd nul zijn, als ze af en toe reusachtig is — bijvoorbeeld een variabele gelijk aan n^2 met kans $1/n$ en anders nul heeft gemiddelde n en is meestal nul. Dat uitsluiten vergt de variantie.
3. Een gegeven knoop is geïsoleerd wanneer geen van zijn $n - 1$ mogelijke kanten verschijnt, met kans $(1-p)^{n-1}$; vermenigvuldig met n . Met $p = c \ln n / n$ wordt dit $\approx n^{1-c}$, wat bij $c = 1$ naar een constante gaat.
4. In $G(n, 1/2)$ heeft elke gelabelde graaf op n knopen dezelfde kans $2^{-\binom{n}{2}}$, dus de verdeling is uniform. Een eigenschap die met kans naar 1 geldt, geldt dus voor een fractie van alle grafen die naar 1 gaat.

Hoofdstuk 26 — De reuzencomponent

1. $\beta = 1 - e^{-1,5\beta}$ itereren convergeert naar $\beta \approx 0,5828$. De gemeten grootste component bij $c = 1,5$, $n = 400$ was $0,581$ van de graaf — overeenstemming tot ongeveer drie duizendsten.
2. De benadering neemt aan dat elke nieuw bereikte knoop $\approx n$ onverkende mogelijke burens heeft. Zodra een constante fractie verkend is, daalt dat aantal wezenlijk, het effectieve gemiddelde nageslacht zakt onder c , en de groei vertraagt — en dat is wat de reuzencomponent bij βn stopt in plaats van bij n .
3. Twee componenten van grootte εn hebben $\varepsilon^2 n^2$ mogelijke kanten ertussen. Bij $p = c/n$ is de kans dat er geen verschijnt $(1 - c/n)^{\varepsilon^2 n^2} \approx e^{-c\varepsilon^2 n}$, wat naar nul gaat. Ze zouden dus vrijwel zeker verbonden zijn, in tegenspraak met dat het verschillende componenten zijn.
4. $c < 1$ betekent $R_0 < 1$: elk geval brengt gemiddeld minder dan één nieuw geval voort, en de uitbraak dooft uit. De op één na grootste component komt overeen met de grootste van de kleine zelfbeperkende clusters — de lokale uitbraken die nooit epidemieën werden.

Hoofdstuk 27 — Extremale grafentheorie

1. $K_{2,3}$ heeft 6 kanten, is driehoekvrij (bipartiet), en $n^2/4 = 6,25$, dus hij haalt de grens zo scherp als een geheel getal bij $n = 5$ kan.
2. Het aantal kanten is $\sum_{i < j} a_i a_j$, wat bij vaste som $\sum a_i = n$ gemaximaliseerd wordt wanneer de delen zo gelijk mogelijk zijn. Een knoop van een groter naar een kleiner deel verplaatsen verhoogt de producttermen strikt — het standaard convexiteitsargument, hetzelfde dat Cauchy–Schwarz codeert.
3. $\chi(K_3) = 3$, $\chi(K_4) = 4$, $\chi(\text{Petersen}) = 3$ — alle drie bevestigd met `chromatic_number`. Dus K_3 en de Petersen-graaf delen $r = 2$ en geven dezelfde hoofdterm $(1 - 1/2)n^2/2 = n^2/4$; K_4 heeft $r = 3$ en geeft $n^2/3$. De Petersen-graaf heeft tien knopen en vijftien kanten tegen de drie en drie van de driehoek, en een van beide verbieden kost asymptotisch hetzelfde — wat het hele punt van de stelling is.
4. Is $\chi(H) = 2$, dan is $r = 1$ en is de hoofdterm $(1 - 1/1)n^2/2$ nul, dus Erdős–Stone reduceert tot $o(n^2)$ en bepaalt niets over de werkelijke orde. Die vinden is het probleem van Zarankiewicz, in het algemeen open.

Hoofdstuk 28 — Ramsey-theorie

1. Neem een willekeurige knoop van K_6 . Zijn 5 kanten dragen 2 kleuren, dus minstens 3 delen er een — zeg dat v rood verbonden is met a , b , c . Is een van ab , bc , ac rood, dan geeft die kant met v een rode driehoek; is er geen rood, dan is abc een blauwe driehoek.
2. Het getuigenis heeft rode kanten $\{(0,3), (0,4), (1,2), (1,4), (2,3)\}$, en dat is een vijfcykel. Zijn complement is eveneens een vijfcykel. C_5 is driehoekvrij, dus geen van beide kleurklassen bevat een driehoek — geverifieerd met `canonical` tegen `cycle(5)` in beide richtingen.
3. $R(2,4) = 4$ (met 2 aan de ene kant volstaat één kant van die kleur, dus je hebt alleen genoeg knopen nodig om 4 onderling niet-aangrenzende af te dwingen), en $R(3,3) = 6$. De recursie geeft $R(3,4) \leq 4 + 6 = 10$. De werkelijke waarde is 9.
4. K_{43} heeft $C(43,2) = 903$ kanten, elk onafhankelijk rood of blauw, wat 2^{903} kleuringen geeft. Dat ligt ver buiten elke denkbare berekening — het aantal atomen in het waarneembare heelal is ongeveer 2^{266} .

Hoofdstuk 29 — Spectrale grafentheorie

1. $A(K_n) = J - I$ met J overal enen. J heeft eigenwaarden n (één keer) en 0 ($n-1$ keer), dus A heeft $n - 1$ één keer en -1 met multipliciteit $n - 1$. Voor K_4 geeft de oplosser $[-1, -1, -1, 3]$.
2. $\sum \lambda_i^2 = \text{spoor}(A^2)$, en $(A^2)_{vv}$ telt gesloten wandelingen van lengte 2 vanuit v , en dat is $\deg(v)$. Sommeren geeft $\sum \deg(v) = 2m$.
3. $n(-\lambda_{\min})/(d - \lambda_{\min}) = 10 \cdot 2/(3 + 2) = 4$, en $\alpha(\text{Petersen}) = 4$ per uitputtend zoeken. De grens is precies scherp.
4. Het spectrum is een invariant, dus isomorfe grafen delen het en een verschil weerlegt isomorfie. Maar cospectrale niet-isomorfe grafen bestaan — $K_{1,4}$ en $C_4 + K_1$ geven beide $\{-2, 0, 0, 0, 2\}$ — dus overeenstemming stelt niets vast.

Hoofdstuk 30 — De Laplaciaan

1. $x^T D x = \sum_v \deg(v) x_v^2$ en $x^T A x = 2 \sum_{uv \in E} x_u x_v$. Aftrekken en per kant hergroeperen geeft $\sum_{uv \in E} (x_u^2 + x_v^2 - 2x_u x_v) = \sum_{uv \in E} (x_u - x_v)^2$. Een som van kwadraten is niet-negatief, dus L is positief semidefiniet.
2. $L(K_3)$ heeft eigenwaarden $\{0, 3, 3\}$. Rij en kolom 0 schrappen laat $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ over met determinant 3, en K_3 heeft inderdaad 3 opspannende bomen — één voor elke weggelaten kant. Cayley is het ermee eens: $3^{3-2} = 3$.
3. Elke rij van L sommeert tot nul, want de diagonaalplaats $\deg(v)$ wordt opgeheven door de $\deg(v)$ plaatsen -1 . Dus $L \cdot \mathbf{1} = \mathbf{0}$, wat de al-enen-vector tot eigenvector met eigenwaarde 0 maakt.
4. Hoe moeilijk de graaf in tweeën te knippen is. P_4 valt uiteen door één kant te verwijderen; K_4 is helemaal niet goedkoop te splitsen. De ongelijkheid van Cheeger maakt dit precies, door het isoperimetrisch getal $h(G)$ tussen $\lambda_2/2$ en $\sqrt{2\Delta\lambda_2}$ in te sluiten.

Hoofdstuk 31 — Minoren en boombreedte

1. Eén zak met alle vier de knopen: breedte $4 - 1 = 3$. Er bestaat geen betere, want elke kant moet binnen een zak liggen, en de kanten van K_4 dwingen elk zakkenstelsel dat ze overdekt om alle vier de knopen samen te bevatten — een klik belandt altijd in één zak.
2. Zodat bomen boombreedte 1 hebben in plaats van 2. De decompositie van een boom heeft zakken van grootte 2 (één per kant), en er één aftrekken laat de conventie overeenkomen met de intuïtie dat bomen het eenvoudigste niet-triviale geval zijn.
3. Verwerk het rooster kolom voor kolom: elke zak bevat de twee knopen van één kolom plus de twee van de volgende, dus zakken hebben grootte hoogstens 3 en de breedte is 2. `treewidth(grid(2,n))` geeft 2 voor $n = 2, 3, 4$.
4. Het $k \times k$ -rooster, met boombreedte k en vlak. Het doet ertoe omdat begrensde boombreedte via de stelling van Courcelle lineaire-tijdalgoritmen geeft en vlakheid alleen niet — dus de twee beperkingen zijn werkelijk onafhankelijk en vlakke problemen blijven moeilijk.

Hoofdstuk 32 — Expanders, en waar je hierna heen gaat

1. Omdat expansie een eigenschap van een groeiende familie is, en $h(C_n) = 4/n \rightarrow 0$. Je kunt een cykel altijd met twee kanten doorknippen, hoe groot hij ook wordt, dus de verhouding tussen rand en grootte verdwijnt. Eén C_6 die als "Ramanujan" gerapporteerd wordt zegt niets over de familie.
2. $d = 3$ en de niet-triviale eigenwaarden zijn 1 en -2 , dus $\lambda = 2$. De Ramanujan-grens is $2\sqrt{(d-1)} = 2\sqrt{2} \approx 2,828$, en $2 \leq 2,828$.
3. Een bipartiete graaf heeft $-d$ in zijn spectrum juist *omdat* hij bipartiet is, dus die behouden zou elke bipartiete graaf brandmerken als slechte expander om een reden die niets met samenhang te maken heeft — vandaar dat de Ramanujan-voorwaarde haar laat vallen. Het mengingslemma moet haar behouden: in $K_{3,3}$ geeft $S = \{0\}$ en $T = \{1\}$ aan dezelfde zijde $e(S, T) = 0$ tegen een verwachte $0,5$, en met -3 uitgesloten zou de grens 0 zijn.
4. Trek een willekeurige d -reguliere graaf en begrens, met de uniegrens, de kans dat een of andere verzameling van hoogstens $n/2$ knopen een kleine rand heeft; het aantal slechte verzamelingen weegt niet op tegen hoe onwaarschijnlijk elke afzonderlijk is. Dat bewijst dat de familie bestaat zonder er een te noemen — dezelfde kloof als bij de Ramsey-ondergrens in hoofdstuk 28, en het duurde tot 1988 en getaltheorie om haar expliciet te dichten.